



University of Bahrain
College of Information Technology
Department of Information Systems
B.Sc. In Cyber security

CYBER HYGIENE EDUCATOR & ASSISTANT

Prepared by

Student 1: Shatha Ebrahim Moosa - 202103023

Student 2: Hiba Barkat Abbas Nakade- 202201477

Student 3: Shahad Mustafa Ali- 202201096

For

ITCY 499

Senior Project

Academic Year 2025-2026-Semester 2

Project Supervisor:

Dr. Yaqoob Salman Mohamed Alsais

Date of Submission

Abstract

The education sector has experienced a 300% rise in cyberattacks, leaving students aged 9–15 increasingly vulnerable. Lacking routine cyber hygiene and finding professional tools overly complex, this group requires accessible solutions. The Cyber Hygiene Educator & Assistant (CHEA) was developed as a unified Windows application that merges active protection with gamified learning. Its five core modules- a Password Vault, Malware Scanner, Encryption tools, AI learning assistant, and Phishing Dojo- are complemented by AI features including scan analysis, attack narratives, phishing generators, scenario simulators, daily challenges, and posture assessments. User evaluations confirm CHEA’s ability to simplify enterprise-grade security concepts, with praise for its intuitive interface, accurate threat detection, and engaging gamified design. By bridging awareness and practice, CHEA equips students to secure their digital identities and build lifelong defensive habits.

Acknowledgments

First and foremost, we extend our gratitude to Almighty Allah for giving us strength, patience, and guidance to complete this project successfully.

This report could not have been produced without the dedication and help from our project supervisor, Dr. Yaqoob Salman Mohamed Alslais. Throughout the stages of the project, we have received advice and guidance from him, which we greatly appreciate. Any feedback on our work has always been received well, and we would like to thank him for being an all-round great supervisor.

We are deeply thankful to University of Bahrain, especially College of Information Technology for their full support, resources, facilities, and academic environment where this research has been conducted.

We would also like to thank the students, and parents who along with their children participated in our surveys and gave us valuable insights for developing this application. We wish to thank the respondents for their time and valuable opinions. Their input was helpful in shaping the requirements and features of our applications.

Lastly, we would like to thank our families and our friends for all their support throughout the year. We also wish to thank them for their encouragement throughout the semester.

Table of Contents

ABSTRACT	II
ACKNOWLEDGMENTS	III
LIST OF TABLES	IX
LIST OF FIGURES	X
CHAPTER 1: INTRODUCTION	1
1.1 PROBLEM STATEMENT	1
1.2 PROJECT OBJECTIVES	2
1.2.1 EDUCATIONAL PLATFORM DEVELOPMENT	2
1.2.2 AI- BASED ASSISTANT INTEGRATION	2
1.2.3 GAMIFICATION IMPLEMENTATION	3
1.2.4 USER-FRIENDLY INTERFACE DESIGN	3
1.2.5 BEHAVIORAL CHANGE PROMOTION	3
1.2.6 SYSTEM EVALUATION	3
1.3 RELEVANCE/SIGNIFICANCE OF THE PROJECT	4
1.4 REPORT OUTLINE	4
CHAPTER 2 LITERATURE REVIEW	5
2.1 INTRODUCTION	5
2.2 UNDERSTANDING STUDENT VULNERABILITY	5
2.3 EXISTING TOOLS AND APPLICATIONS	6
2.3.1 PASSWORD MANAGEMENT TOOLS	6
2.3.2 SECURITY SCANNING TOOLS	7
2.3.3 EDUCATIONAL PLATFORMS	7
2.3.4 COMPARATIVE ANALYSIS	7
2.4 ACADEMIC LITERATURE ANALYSIS	8
2.4.1 SYSTEMATIC REVIEWS ON CYBERSECURITY EDUCATION	8
2.4.2 STUDIES ON STUDENT AWARENESS	9
2.4.3 GAMIFICATION IN CYBERSECURITY EDUCATION	9
2.5 RESEARCH GAPS IDENTIFIED	10

2.5.1 TOOL INTEGRATION GAP	10
2.5.2 AGE-APPROPRIATENESS GAP.....	10
2.5.3 EDUCATION-PROTECTION GAP	10
2.5.4 ENGAGEMENT AND MOTIVATION GAP	10
2.5.5 ACCESSIBILITY GAP	11
2.6 HOW CHEA ADDRESSES THE GAPS	11
2.6.1 INTEGRATED PLATFORM APPROACH.....	11
2.6.2 AGE-APPROPRIATE DESIGN	11
2.6.3 EDUCATION + PROTECTION INTEGRATION	12
2.6.3.1 Six AI-Powered Features	12
2.6.4 GAMIFICATION STRATEGY	13
2.6.5 FREE AND ACCESSIBLE.....	13
2.6.6 CONTRIBUTION TO THE FIELD.....	13
2.7 CONCLUSION	15
CHAPTER 3 PROJECT MANAGEMENT	16
3.1 PROCESS MODELS.....	16
3.2 SCRUM IMPLEMENTATION FOR CHEA	16
3.2 RISK MANAGEMENT	17
3.3 PROJECT ACTIVITIES PLAN	19
CHAPTER 4 REQUIREMENTS COLLECTION.....	21
4.1 REQUIREMENT ELICITATION.....	21
4.1.1 LITERATURE REVIEW FINDINGS	21
4.1.2 ONLINE SURVEY	22
4.1.2.1 Summary of Survey Findings:.....	22
4.2 SYSTEM REQUIREMENTS.....	24
4.2.1 FUNCTIONAL REQUIREMENTS	24
4.2.2 NON-FUNCTIONAL REQUIREMENTS.....	27
4.3 SYSTEM MODELS	28
4.3.1 SYSTEM ARCHITECTURE	28
4.3.2 USE CASE MODEL	30
4.3.3 DATA FLOW MODEL	31
4.3.4 ENTITY RELATIONSHIP MODEL.....	34
CHAPTER 5 SYSTEM DESIGN	36

5.1 INTRODUCTION	36
5.2 DATABASE SCHEMA DESIGN.....	36
5.2.1 ENCRYPTION IN THE SCHEMA.....	39
5.2.2 GAMIFICATION DATA MODEL	39
5.2.3 FIRESTORE REST API WORKAROUND	40
5.2.4 ROOT USER DOCUMENT.....	40
5.2.5 SECURITY BUDDY CLIENT-SIDE CACHE	41
5.3 USER INTERFACE DESIGN	41
5.3.1 LOGIN INTERFACE.....	41
5.3.2 REGISTRATION INTERFACE.....	42
5.3.3 FORGOT PASSWORD INTERFACE	43
5.3.4 DASHBOARD INTERFACE.....	44
5.3.5 PASSWORD GENERATOR INTERFACE	45
5.3.6 LINK SCANNER INTERFACE	46
5.3.7 ENCRYPTION LAB INTERFACE	48
5.3.8 PHISHING DOJO INTERFACE	49
5.3.9 CREDENTIAL VAULT INTERFACE	51
5.3.10 AI BUDDY INTERFACE.....	53
5.3.11 NAVIGATION AND LAYOUT	55
5.3.12 INTERNATIONALIZATION AND ACCESSIBILITY.....	56
5.3.13 FILE SCANNER INTERFACE.....	57
5.3.14 PASSWORD CHECKER INTERFACE	59
5.3.15 QUIZ ARENA INTERFACE	61
5.3.16 IMAGE PRIVACY (PHOTO SECRETS) INTERFACE	62
5.3.17 AI SCENARIO SIMULATOR INTERFACE	64
5.3.18 AI SECURITY POSTURE ASSESSMENT INTERFACE	65
5.3.19 SETTINGS INTERFACE.....	67
5.4 APPLICATION ARCHITECTURE	68
5.4.1 ROUTING AND NAVIGATION ARCHITECTURE	68
5.4.2 STATE MANAGEMENT ARCHITECTURE.....	69
5.4.3 AI SERVICE LAYER.....	70
5.4.4 REUSABLE UI COMPONENT LIBRARY.....	71
5.5 ALGORITHM DESIGN	72
5.5.1 ENCRYPTION ALGORITHM	72
5.5.2 DECRYPTION ALGORITHM	73

5.5.3 PASSWORD GENERATION ALGORITHM.....	74
5.5.4 PASSWORD STRENGTH ANALYSIS ALGORITHM.....	75
5.5.5 URL SCANNING WITH VIRUSTOTAL POLLING ALGORITHM	77
5.5.6 FILE SCANNING WITH HASH-FIRST CHECK ALGORITHM	78
5.5.7 IMAGE EXIF METADATA SCANNING ALGORITHM	80
5.5.8 AI CHATBOT WITH STREAMING RESPONSE ALGORITHM.....	82
5.5.9 XP CALCULATION AND LEVEL PROGRESSION ALGORITHM	84
5.5.10 AI SCENARIO SIMULATOR ALGORITHM.....	88
5.5.11 AI SECURITY POSTURE ASSESSMENT ALGORITHM.....	89
5.6 SUMMARY	90
CHAPTER 6SYSTEM IMPLEMENTATION AND TESTING	92
6.1 SYSTEM IMPLEMENTATION	92
6.1.1 DEVELOPMENT ENVIRONMENT SETUP	92
6.1.2 FRONTEND IMPLEMENTATION.....	94
6.1.2.1 Project Structure	94
6.1.2.2 Routing and Navigation	94
6.1.2.3 State Management	95
6.1.2.4 Activity Tracking Hook.....	96
6.1.2.5 Firebase Integration.....	96
6.1.2.6 Theme System.....	97
6.1.2.7 Key UI Components	97
6.1.2.8 AI Service Layer	98
6.1.3 BACKEND IMPLEMENTATION	99
6.1.3.1 Command Registration and IPC Architecture	99
6.1.3.2 Cryptography Module (crypto.rs)	99
6.1.3.3 VirusTotal Module (virustotal.rs)	100
6.1.3.4 Image Privacy Module (image_privacy.rs).....	101
6.1.3.5 AI Buddy Module (ai_agent.rs)	101
6.1.3.6 Terminal Module (terminal.rs)	102
6.1.4 DATABASE IMPLEMENTATION	103
6.1.4.1 Firebase Project Configuration	103
6.1.4.2 Collection and Subcollection Structure	103
6.1.4.3 Firestore REST API Workaround.....	104
6.1.4.4 Security Rules	104
6.1.4.5 Data Encryption Approach	105
6.2 EVALUATION AND TESTING	105

6.2.1 FUNCTIONAL TESTING.....	105
6.2.2 SECURITY TESTING.....	110
6.2.3 PERFORMANCE TESTING.....	111
6.2.4 USABILITY TESTING.....	114
6.2.4.1 Testing Approach.....	114
6.2.4.2 Task Scenarios.....	115
6.2.4.3 Survey Results.....	115
6.2.4.4 Key Findings.....	116
6.2.4.5 User Feedback Summary.....	116
6.2.5 STRENGTHS AND LIMITATIONS.....	117
6.2.5.1 System Strengths.....	117
6.2.5.2 Known Limitations.....	118
6.2.5.3 Comparison with Existing Tools.....	119
CHAPTER 7 CONCLUSION AND FUTURE WORK.....	121
7.1 INTRODUCTION.....	121
7.2 SUMMARY OF PROJECT WORK.....	121
7.3 KEY FEATURES IMPLEMENTED.....	121
7.4 TESTING RESULTS AND FINDINGS.....	122
7.5 SIGNIFICANCE AND IMPACT.....	122
7.6 LIMITATIONS.....	123
7.7 FUTURE WORK.....	123
7.8 CONCLUSION.....	124
REFERENCES.....	125

List of Tables

Table 1: Comparative Analysis of Existing Cybersecurity Tools.....	8
Table 2: Summary of Research Gaps and solution	14
Table 3: CHEA Risk Management Plan.....	17
Table 4: Findings from Literature Review.....	21
Table 5: Survey Demographics	22
Table 6: Functional Requirements.....	25
Table 7: Non-Functional Requirements	27
Table 8: Complete Firestore Database Schema	37
Table 9: Main User profile fields.....	40
Table 10: Route Tree Structure	68
Table 11: Library of reusable UI components	71
Table 12: Reusable AI Component	72
Table 13: Activity XP Rewards	84
Table 14: Functional Testing Results.....	105
Table 15: Performance Testing Results.....	112
Table 16: Survey Results.....	115
Table 17: Comparision with existing tools	119
Table 18: Summary of Security Test Results	122
Table 19: The standards for Page and margin setup.....	خطأ! الإشارة المرجعية غير معرّفة.
Table 20: Fonts and Styles.	خطأ! الإشارة المرجعية غير معرّفة.

[11]: To update the list of figures, right click → Update Field. Make sure each figure has a caption

List of Figures

Figure 1: Fragmented and Integrated Security Landscape 11

Figure 2: Scrum Process Model 17

Figure 3: Gantt Chart 19

Figure 4: Online Threats Encountered 23

Figure 5: Students Password Habits 23

Figure 6: Students Learning Preference..... 24

Figure 7: Students interest in AI assistant feature 24

Figure 8: System Architecture..... 30

Figure 9: Use Case Model..... 31

Figure 10: Data Flow Model 34

Figure 11: Entity Relationship Model..... 35

Figure 12: Login Interface (Light and dark Mode)..... 42

Figure 13: Registration Interface (Light and Dark Mode) 43

Figure 14: Forget Password Interface 44

Figure 15: Dashboard Interface 45

Figure 16: Password Generator Interface 46

Figure 17: Link Scanner Interface (Light Mode) 48

Figure 18: Encryption Lab Interface (Light Mode) 49

Figure 19: Phishing Dojo Interface (Light Mode) 51

Figure 20: Credential Vault Interface (Light Mode)..... 53

Figure 21: AI Agent Interface (Light Mode)..... 55

Figure 22: Navigation Layout..... 56

Figure 23: File Scanner Interface (Light Mode) 59

Figure 24: Password Checker Interface 60

Figure 25: AI attack Narative..... 61

Figure 26: Quiz Arena Interface 62

Figure 27: Image Privacy Interface 64

Figure 28: Scenario Simulator Interface 65

Figure 29: Security Posture Assessment Interface 67

Figure 30: Settings Interface 68

Figure 31: Main body chapter titleخطأ! الإشارة المرجعية غير معروفة.

Chapter 1: Introduction

The fast transformation of education digitally has brought new learning opportunities to students worldwide, but it has also put the younger generation at risk. Students, who are typically around the age of 9-15 years old, will be using technologies for learning a variety of contexts. They are going to access school portals, use social media platforms where they publish digitally, and share information online with schoolmates and educators.

Compared to adult internet users who have developed security instincts through years of digital engagement, children and young people lack such instincts. Therefore, they could be exploited via phishing tactics, easily compromised through the use of weak passwords, and tricked into spreading malicious software.

There is an urgency regarding this situation which is underscored by alarming industry data. Research from Check Point Research (2022) found that cyberattack activity on educational institutions increased by 300% since 2020, where students were the primary victims. Verizon Data Breach Investigations Report (2021) found that 81% of security breaches stem from weak or stolen credentials, while 94% of malware entered an organization via email. Today's basic cybersecurity tools are largely inaccessible to children and students because they are either too complex or adult oriented.

This project addresses these gaps by creating a comprehensive Windows desktop application, the Cyber Hygiene Educator & Assistant (CHEA), which will incorporate tools for active protection as well as playful and engaging education to teach individuals Cyber Hygiene. The application shall incorporate five security features including a Password Vault, Malware Scanner, file and folder Encryption Tools, and an AI-powered learning assistant. In addition, the CHEA application shall incorporate 6 AI-powered features that will engage users in fun, interactive scenarios, including a scenario simulator, as well as feature a daily Security Buddy challenge and a monthly Security Posture Assessment. CHEA aims to ensure students have a strong theoretical foundation of security best practices, and more importantly, gain the practical tools and knowledge to defend themselves on the Internet

This chapter introduces the project's problem statement of the project, outlines its objectives, highlights the significance of the study, and provide a structured overview of the report.

1.1 Problem Statement

Due to fast digitalization of the education sector, an important security crisis for students aged 9-15 has been created. The problem is divided into three sub-problems which are listed below:

1. **Increase in Targeted Attacks:** Since 2020, the educational institutions have seen a 300% increase in cyberattacks since 2020. Students have often been found to be the main targets of these threats.
2. **The Cyber Hygiene Gap:** Many students lack fundamental skills like strong password management and phishing detection. According to research, 81% breaches are because of weak credentials.
3. **Fragmented and Complex Tools:** The security solutions that exist in today's digital time are designed for adult professionals. These solutions are either too technical or too fragmented for the younger generation to adopt.

1.2 Project Objectives

The main objective of this project is to design and develop an interactive system called "Cyber Hygiene Educator & Assistant" (CHEA) that aims to improve cybersecurity awareness among students in a simple and engaging way. The main objectives of this project are:

1.2.1 Educational Platform Development

To develop an educational platform that teaches basic cybersecurity concepts such as strong passwords, phishing awareness, URL safety, file security, and safe internet usage. This objective was achieved through the implementation of:

- Password Generator and Strength Analyzer
- Link Scanner (URL Safety Checker)
- Smart File Scanner
- Image Privacy (Metadata Wiper)
- Encryption/Decryption Tools
- Quick Guide educational content

1.2.2 AI- Based Assistant Integration

To integrate an AI-based assistant (chatbot) that guide students, answer their questions, and provide instant feedback in a simple and understandable manner. This objective is achieved through:

- AI Agent (main chatbot) with contextual awareness
- AI Security Buddy (personalized daily challenges)
- AI Scenario Simulator (interactive decision-making environment)
- AI Attack Narrative Generator (immersive security incident stories)
- AI Scan Analysis (plain-language threat explanations)
- AI Phishing Email Generator (dynamic phishing training)
- AI Security Posture Insights (personalized security recommendation)

1.2.3 Gamification Implementation

To apply gamification techniques (such as quizzes, rewards, challenges, and progression systems) to make the learning process more fun and motivating for young users. This objective was achieved through:

- XP (Experience Points) system: earn points through genuine security behaviours.
- 10-level ranking system: Rookie → Scout → Shield → Guardian → Cyber Ninja → Security Ranger → Knight → Cyber Legend → Cyber Master → Cyber Supreme
- Daily tasks and streak tracking
- Achievement badges
- Quiz Arena with 3 difficulty tiers (Cadet, Analyst, Operator)
- Phishing Dojo with AI-generated phishing scenarios

1.2.4 User-Friendly Interface Design

To design a user-friendly interface suitable for school students (ages 9-15), ensuring the content is clear, interactive, and easy to navigate. This objective is achieved through:

- Cyberpunk/neon aesthetic with modern visual appeal
- React/Tauri desktop application (cross-platform)
- Visual feedback indicators (red/green for password strength)
- Age-appropriate language (non-technical vocabulary)
- RTL support with Arabic and English localization
- Responsive design with intuitive navigation
- Dark/Light theme support

1.2.5 Behavioral Change Promotion

To promote positive cyber hygiene behaviors by encouraging students to practice safe digital habits in their daily online activities. This objective is achieved through:

- Security Posture page with personalized recommendations
- AI Security Posture Insights analyzing user habits
- Daily challenges encouraging consistent security practices
- Streak system rewarding continued engagement
- Educational content explaining "why" behind security practices

1.2.6 System Evaluation

To evaluate the effectiveness of the system in improving students' cybersecurity knowledge and awareness. This objective is achieved through:

- Usability testing with target demographic (students aged 9-15)

- Survey-based assessment comparing pre/post usage knowledge
- Qualitative feedback collection on user experience
- Analysis of engagement metrics (XP earned, levels gained, streaks maintained)

1.3 Relevance/Significance of the project

The Cyber Hygiene Educator & Assistant addresses critical gaps in K-12 cybersecurity education:

- **Tool Integration:** Consolidates multiple security tools into a unified platform, eliminating "security fatigue" from managing separate applications.
- **Age-Appropriate Design:** Specifically targets ages 9-15 with interfaces respecting developmental capabilities, unlike adult-oriented tools retrofitted for children.
- **Education-Protection Integration:** Combines practical security tools with educational content, rejecting the false dichotomy between theoretical learning and practical protection.
- **Engagement Enhancement:** Implements evidence-based gamification (validated by Pramod, 2025 and Khairallah & Abu-Naseer, 2024) to improve knowledge retention and behavioral change.

1.4 Report Outline

This report is organized as follows:

- **Chapter 2: Literature Review** – Details a comprehensive review of existing cybersecurity education tools, academic literature, and identified research gaps.
- **Chapter 3: Project Management** - Describes the Agile (Scrum) methodology, risk management plan, and project timeline.
- **Chapter 4: Requirements Collection and Analysis** - Details the requirement elicitation process, functional and non-functional requirements, and system models.
- **Chapter 5: System Design** - Presents the system architecture, database design, and user interface design.
- **Chapter 6: System Implementation and Testing** - Describes the implementation of all system components and the testing procedures.
- **Chapter 7: Conclusion and Future Work** - Summarizes the project achievements and suggests potential enhancements.

Chapter 2

Literature Review

2.1 Introduction

Understanding why this project matters requires looking beyond the surface of existing solutions to examine the limitations that have left K-12 students vulnerable to cyber threats. This literature review aims not to catalogue prior work, but to position our study within a broader academic conversation about gaps between threat realities and educational responses.

This study is motivated by the disconnect between the increasing cyber threats targeting K-12 students and the educational responses designed to address them. Check Point Research (2022) documented a significant 300% surge in cyberattacks against educational institutions since 2020. This is not viewed as an abstract statistic, but as evidence of a vulnerability that highlights the need for academic attention. Notably, the Verizon Data Breach Investigations Report (2021) indicates the human element behind these breaches: 81% stem from weak passwords, while 94% of malware arrives via email. These are not sophisticated zero-day exploits targeting infrastructure; rather, they appear to be preventable attacks exploiting gaps in user knowledge and behavior. Despite passwords and phishing representing the primary attack vectors, these topics tend to receive the least attention in educational literature. This suggests that the field may have divergent priorities, focusing on broad concepts while potentially overlooking the specific technical vulnerabilities that often compromise student safety.

In the following sections, we examine existing tools not merely to describe them, but to understand their pedagogical limitations. We synthesize academic literature to identify the gaps between theoretical frameworks and practical implementation, and we position the Cyber Hygiene Educator & Assistant as a proposed solution to address these identified limitations. Our approach combines insights from peer-reviewed sources, industry threat intelligence, and market analysis to construct an argument for integrated, gamified security education as a constructive response to the persistent opportunities for improvement in K-12 cyber hygiene instruction.

2.2 Understanding Student Vulnerability

Students occupy a unique position in the cybersecurity ecosystem simultaneously highly connected and profoundly vulnerable. Unlike adults who may have developed security instincts through years of professional digital engagement, students navigate increasingly complex online environments without adequate preparation or protection. Almomani et al. (2021) highlight that higher education often lacks maturity in cybersecurity practices, with students routinely depending on weak, easily guessable credentials and demonstrating limited capacity to recognize social engineering tactics. We interpret this not as individual negligence, but as evidence of systematic educational failures; students cannot practice what they have never been taught.

However, vulnerability extends beyond mere knowledge deficits into behavioral territory. Purnama et al. (2021) whose work has garnered significant attention with 245 citations provide compelling evidence that digital literacy functions as a direct predictor of online risk behaviors in children. Their findings suggest that self-control and technical knowledge interact in complex ways to determine exposure levels. This implies that effective cybersecurity education cannot simply dump technical information onto students; it must address the behavioral and psychological factors that drive risky decisions. We believe that any solution ignoring this dual nature of vulnerability cognitive and behavioral will inevitably fail to produce lasting change.

The Verizon DBIR (2021) crystallizes the threat landscape into three primary vectors: weak password practices, phishing attacks, and malware delivery through compromised email and URLs. We view these not as isolated dangers, but as an interconnected ecosystem of risk that students encounter daily. Educational responses have disproportionately focused on general "internet safety" concepts cyberbullying, screen time, appropriate content while systematically neglecting the technical skills required to defend against these primary threat vectors. This misalignment between what threatens students and what we teach them forms the central tension our project seeks to resolve.

Cowling et al. (2025) add crucial nuance through their comprehensive study of students aged 10-13, demonstrating that digital safety cannot be separated from digital literacy and overall wellbeing. Their research suggests that students' online engagement patterns are deeply influenced by their communication skills and social contexts. This finding resonates with our core philosophy: cybersecurity education must be integrated rather than siloed. Teaching password hygiene in isolation from broader digital citizenship creates knowledge that students struggle to apply when confronted with real-world social engineering attempts. We need approaches that address the whole student, not just their technical behaviors.

2.3 Existing Tools and Applications

Before proposing new solutions, we must critically interrogate why current market offerings fail to protect K-12 students effectively. Our analysis reveals that existing tools generally fall into categories designed for adult professionals, then awkwardly retrofitted for younger users a design philosophy that fundamentally misunderstands developmental needs.

2.3.1 Password Management Tools

Password managers like Bitwarden represent the gold standard for adult credential protection, yet they present significant adoption barriers for younger users. The complexity of these interfaces often deters students entirely, driving them back to insecure habits like reusing simple passwords. We have observed this pattern ourselves: when security tools require excessive cognitive load, users abandon them regardless of their protective value. More critically, these tools function purely as utilities rather than educational instruments. They store passwords efficiently but miss the pedagogical opportunity to explain why "P@ssw0rd123" fails against modern cracking techniques or how entropy

determines password strength. This suggests to us that current password managers solve the immediate problem while failing to build the long-term security literacy necessary for independent digital citizenship.

2.3.2 Security Scanning Tools

Platforms like VirusTotal offer impressive threat detection capabilities, yet they operate as detached web services that disrupt rather than integrate with student workflows. We imagine a typical scenario: a student receives a suspicious file via email or social media. The friction required to navigate to VirusTotal, upload the file, and interpret technical results creates a barrier that most students will simply bypass. These tools provide binary verdicts clean or infected without contextual education about why a file might be dangerous or how malware functions. Interpreting scan results often requires technical knowledge about hash values, detection ratios, and false positives that younger students have not yet developed. Consequently, these powerful protective tools remain inaccessible to the demographic that arguably needs them most.

2.3.3 Educational Platforms

KnowBe4 Student Edition dominates the security awareness training market, but its limitations reveal the gaps in current approaches. First, it targets students aged 16 and above, immediately excluding middle schoolers who are already active online. Second, while it effectively addresses phishing awareness and social engineering concepts, it stops short of providing practical protection tools. Students learn to recognize threats but lack integrated password generators, encryption utilities, or file scanners to defend against them. Third, its subscription-based enterprise model creates accessibility barriers for underfunded schools or individual learners, effectively making security a privilege rather than a right.

Hack The Box presents the inverse problem: exceptional practical training in offensive and defensive security but designed exclusively for adults and professionals. Its gamified approach to penetration testing and network security offers genuine educational value yet assumes baseline technical competencies that children have not acquired. Like other platforms, it omits the fundamental defensive tools password strength checkers, URL verifiers, encryption labs that beginners require for daily protection. Google Interland targets the appropriate age demographic (8-12) with engaging gamification, yet provides zero practical security tools, functioning purely as educational content without real-world protective utility.

2.3.4 Comparative Analysis

Table 1 crystallizes our critique of this fragmented landscape. Current solutions force an impossible choice between education and protection, between accessibility and sophistication. No existing platform simultaneously provides practical utility, age-appropriate design, and integrated educational content. This observation drives our conviction that the market requires a fundamentally different approach to one that refuses to accept these false dichotomies.

Table 1: Comparative Analysis of Existing Cybersecurity Tools

Application	Password	URL	Encrypt	Games	AI	Target
KnowBe4	No	No	No	Yes	No	16+
Hack The Box	No	No	No	Yes	No	Adults
Bitwarden	Yes	No	No	No	No	General
VirusTotal	No	Yes	No	No	No	General
Google Interland	No	No	No	Yes	No	8-12
Cyber hygiene educator & assistant	Yes	Yes	Yes	Yes	Yes	9-15

2.4 Academic Literature Analysis

2.4.1 Systematic Reviews on Cybersecurity Education

Foundational systematic reviews have shaped our understanding of what works in security education. Zhang-Kennedy and Chiasson (2021) conducted the most comprehensive review in this space, analyzing multimedia tools across 193 citations to determine effective educational approaches. Their framework suggests that interactive, multimedia-based learning significantly outperforms traditional text-based instruction. To the best of their knowledge, their review remains the definitive examination of how technology can enhance security awareness. We draw heavily on their conclusion that engagement and interactivity determine educational efficacy.

Almomani et al. (2021) conducted a review of existing cybersecurity frameworks that are applicable to higher education. According to the results of the review, even though controls and standards such as Essential Cybersecurity Controls (ECC) exist, these frameworks lack practical, published mechanisms that continuously assess an organization's security level. Sağlam and Miller (2023) reinforce this urgency through their focus on children specifically, noting that while mentoring approaches show promise, the literature lacks comprehensive evaluation of different pedagogical methods. This gap suggests to us that the field needs not just new tools, but rigorous empirical evidence about which teaching strategies produce behavioral change a contribution our project aims to make.

Triplett (2023) adds another dimension by revealing significant concerns about the cybersecurity status of K-12 schools themselves, emphasizing the importance of preparing students for future careers in the field. We interpret this as a dual mandate: our project must protect students today while building the foundational knowledge that might inspire tomorrow's cybersecurity professionals.

2.4.2 Studies on Student Awareness

Understanding baseline awareness levels proves crucial for designing effective interventions. Alharbi and Tassaddiq (2021) assessed students at Majmaah University and discovered an inverse relationship between password habits and overall cybersecurity awareness. Their study suggests that poor password practices correlate with lower security consciousness across all domains. We believe this indicates that password education serves as a gateway skill mastering password hygiene potentially unlocks broader security awareness. This hypothesis informs our decision to prioritize password tools as entry points to comprehensive security education.

Alqahtani (2022) examined the factors determining awareness levels, identifying prior training and exposure to security incidents as key variables. This suggests that awareness is not innate or automatic; it requires deliberate cultivation through structured programs. Bottyán (2023) confirmed these patterns internationally, documenting consistent weaknesses in password management and social engineering susceptibility across different cultural contexts. These universal patterns imply that the challenges we address are not culturally specific but represent fundamental gaps in how humans intuitively understand digital risk.

Erendor and Yildirim (2022) provide timely analysis of online education environments, finding significant gaps in student understanding of software security, password management, and social media privacy. Their research takes on added significance given the post-COVID-19 shift toward remote and hybrid learning models. We believe their findings indicate that the rapid digitization of education has outpaced the development of corresponding security education, leaving students vulnerable precisely when their screen time and digital dependency have increased.

2.4.3 Gamification in Cybersecurity Education

The pedagogical potential of gamification emerges clearly from recent literature. Pramod (2025) conducted a state-of-the-art review demonstrating that gamification significantly improves both engagement and knowledge retention in security contexts. His analysis suggests that game elements are particularly effective for addressing modern threats like malware and phishing because they allow safe exploration of risky scenarios. Wijanarko and Erlansari (2025) support this thesis specifically for adolescent populations, noting that gamification helps students navigate increasingly sophisticated risks including cyberbullying and advanced social engineering.

Crucially, Khairallah and Abu-Naseer (2024) moved beyond theoretical analysis to conduct a controlled experiment proving that gamified methods outperform traditional training for phishing awareness. Their empirical evidence provides the scientific foundation for our design choices. Nagaraj et al. (2025) affirmed these findings in corporate contexts, demonstrating that gamified learning enhances knowledge retention among employees. We argue that if gamification proves effective for

adult professionals, its impact should be even more pronounced for digital-native students who have grown up with game-based learning models.

2.5 Research Gaps Identified

Through this systematic analysis, we have identified five significant gaps that current solutions fail to bridge. These represent both research opportunities and practical imperatives that justify our project's existence.

2.5.1 Tool Integration Gap

The Tool Integration Gap strikes us as the most immediate problematic. Students currently navigate a fragmented ecosystem where password management, URL scanning, file analysis, and encryption exist as separate applications. This fragmentation creates what we term "security fatigue" as the cognitive overload that prevents consistent tool adoption. We believe students know they should use security tools, but the friction of managing multiple interfaces causes them to abandon protective behaviors.

2.5.2 Age-Appropriateness Gap

The Age-Appropriateness Gap reveals a market designing for adults and expecting children to adapt. Sağlam and Miller (2023) note that existing interfaces assume adult cognitive processing speeds, technical vocabulary, and executive function capabilities. We see this manifest in tools that require complex navigation, assume prior technical knowledge, or lack the visual feedback mechanisms that support younger learners. This gap results in abysmal adoption rates even when schools mandate security software.

2.5.3 Education-Protection Gap

The Education-Protection Gap represents a fundamental pedagogical failure. Zhang-Kennedy and Chiasson (2021) found that tools bifurcate into two ineffective categories: purely educational platforms that teach concepts without practical application, and protective utilities that function as "black boxes" without explaining their operations. We believe this creates a dangerous disconnect where students either understand threats theoretically but cannot defend against them or use protective tools without understanding why they matter.

2.5.4 Engagement and Motivation Gap

The Engagement and Motivation Gap explain why traditional security education fails to produce lasting change. Pramod (2025) and Wijanarko and Erlansari (2025) both document that without proper engagement mechanisms, students abandon training modules before knowledge consolidation occurs.

Standard lecture-based approaches or dry documentation fail to capture attention in an era of interactive digital media

2.5.5 Accessibility Gap

The Accessibility Gap creates inequitable security landscapes. Subscription models for KnowBe4, Bitwarden Premium features, and comprehensive suites like Kaspersky exclude lower-income students and underfunded districts. We view this economic barrier as ethically unacceptable digital safety should function as a basic right rather than a premium service.

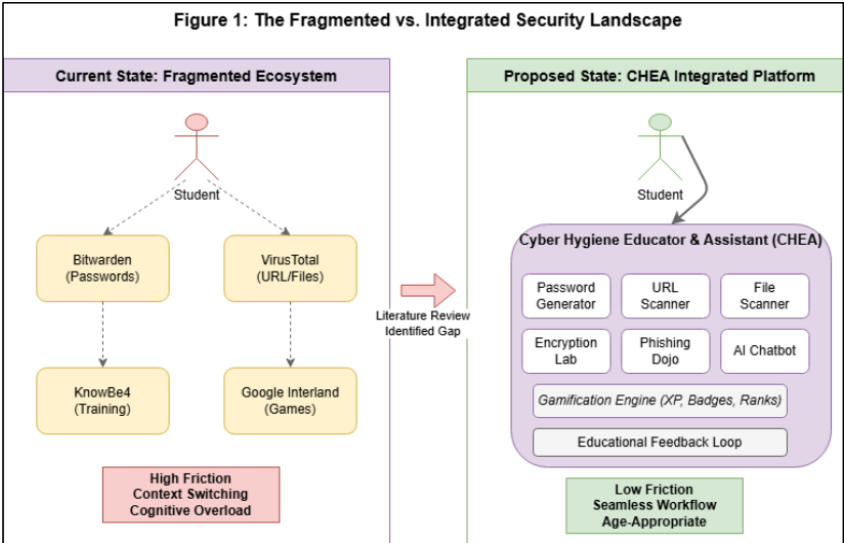


Figure 1: Fragmented and Integrated Security Landscape

2.6 How CHEA Addresses the Gaps

The Cyber Hygiene Educator & Assistant represents our deliberate attempt to bridge each identified gap through intentional design choices.

2.6.1 Integrated Platform Approach

We address the fragmentation problem by consolidating five essential security tools into a unified interface. Our platform combines a Password Generator and Strength Analyzer, URL Safety Checker integrated with VirusTotal API, Smart File Scanner utilizing hash verification, Photo Metadata Wiper for privacy protection, and an Encryption/Decryption Lab for cryptographic exploration. This integration reduces cognitive load by eliminating context-switching between applications, allowing students to develop comprehensive security habits naturally.

2.6.2 Age-Appropriate Design

We specifically target the 9-15 demographic with interfaces that respect developmental capabilities. Visual indicators red for weak passwords, green for strong provide immediate, intuitive feedback. We

have eliminated technical jargon in favor of explanatory language that builds understanding without condescension. The Windows Forms interface offers familiarity while maintaining modern visual appeal. Crucially, we have designed navigation that assumes limited prior technical knowledge, ensuring that first-time users can access protective features immediately.

To further support younger users who may be unfamiliar with cybersecurity tools, we implemented a collapsible Quick Guide component on the four most tool-intensive pages (Link Scanner, File Scanner, Password Checker, and Credential Vault). Each guide presents 3–4 numbered, kid-friendly steps with emoji icons that explain exactly what to do. For example, "Copy a link you want to check → Paste it in the box → Click Scan Now → Read the result." The guide appears as a small banner below the page header with a lightbulb icon and "Not sure what to do?" label, expanding on click and dismissible via a close button. This design directly addresses the age-appropriateness gap identified by Sağlam and Miller (2023) by ensuring that students aged 9–15 can use every tool independently, even on their first visit, without needing external instructions or adult supervision.

2.6.3 Education + Protection Integration

We reject the false dichotomy between teaching and protecting. Each tool in our suite serves dual purposes: the Password Generator explains entropy and character variety while creating credentials; the Encryption Lab teaches cryptographic principles through hands-on encoding and decoding; the AI Chatbot Assistant answers conceptual "why" questions in real-time while guiding practical tool usage. This approach ensures that students develop both procedural knowledge (how to protect themselves) and conceptual understanding (why these protections matter).

2.6.3.1 Six AI-Powered Features

Beyond the chatbot assistant, CHEA implements six distinct AI-powered features that enhance both education and protection:

AI Security Buddy - A personalized daily challenge generator that creates tailored cybersecurity tasks based on the user's skill level, learning history, and engagement patterns. Drawing from Pramod (2025)'s findings on engagement mechanisms, this feature provides adaptive difficulty and contextual learning opportunities, ensuring students receive challenges appropriate to their current proficiency level.

AI Scenario Simulator - An interactive decision-making environment where students face AI-generated cybersecurity scenarios and must choose between multiple response options. Each choice triggers AI evaluation with dramatic, educational consequences, reinforcing behavioral learning through immediate feedback similar to the gamified training validated by Khairallah and Abu-Naseer (2024).

AI Attack Narrative Generator - A streaming AI service that creates immersive security incident stories, helping students understand attack vectors through narrative exploration rather than abstract

technical descriptions. This addresses the engagement gap identified by Wijanarko and Erlansari (2025) by presenting cybersecurity concepts in story form.

AI Scan Analysis - Provides plain-language explanations of URL and file scan results, translating technical threat intelligence into understandable educational content. This bridges the technical accessibility gap identified in tools like VirusTotal (2.3.2 Security Scanning Tools) by making expert-level security analysis accessible to younger users.

AI Phishing Email Generator - Dynamically generates realistic phishing and legitimate emails at three difficulty tiers (Cyber Cadet, Analyst, and Operator) for the Phishing Dojo training module. This replaces static templates with adaptive, increasingly sophisticated threats that grow alongside the student's detection skills.

AI Security Posture Insights - Analyzes the user's overall security habits and provides AI-generated recommendations for improvement, functioning as a personalized security coach that addresses the dual nature of vulnerability, cognitive and behavioral identified by Purnama et al. (2021).

2.6.4 Gamification Strategy

Our engagement system draws directly from Pramod (2025) and Khairallah and Abu-Naseer (2024)'s empirical findings. We implemented Experience Points (XP) earned through genuine security behaviors using tools, completing quizzes, and maintaining daily engagement streaks. Our Ranking System progresses from Cyber Cadet to Cyber Legend, providing visible progression markers. Achievement Badges recognize specific accomplishments like identifying phishing attempts or securing social media accounts. The Phishing Dojo offers safe practice against simulated threats, while the Quiz Arena tests knowledge retention. This is not entertainment layered over education; it is behavioral reinforcement based on research.

2.6.5 Free and Accessible

We are committed to a completely free model with no subscription tiers or premium features. By utilizing free API tiers (VirusTotal) and local processing (SQLite database, AES-256 encryption), we minimize operational costs while maximizing functionality. This design choice ensures that economic status never determines digital safety.

2.6.6 Contribution to the Field

Beyond immediate practical application, our project contributes to cybersecurity education research by providing a working model of integrated platform design that researchers can study and iterate upon. We will contribute empirical evidence through planned evaluations testing the effectiveness of combined practical tools and gamified education. Most importantly, we directly address the password and email security neglect that is identified as the most dangerous gap in current K-12 education.

Identified Research Gap	Critical Description	CHEA Solution Strategy
Tool Integration Gap	Students face a fragmented ecosystem where security tools exist as separate applications, causing "security fatigue" and cognitive overload.	Unified Platform: Consolidates password management, URL scanning, and encryption into a single interface to reduce friction. Also integrates six AI-powered features (Security Buddy, Scenario Simulator, Attack Narrative, Scan Analysis, Phishing Generator, Security Posture) into unified workflow.
Age-Appropriateness Gap	Existing tools are designed for adult cognitive processing and technical vocabulary, leading to low adoption rates among children.	Targeted Design: Interfaces specifically for ages 9–15 with visual feedback (red/green indicators) and non-technical language.
Education-Protection Gap	Current solutions bifurcate into either purely theoretical education or "black box" protection utilities, rarely combining both.	Hybrid Approach: Tools serve dual purposes (e.g., Password Generator explains entropy while creating credentials) to blend theory with practice. AI Scan Analysis and AI Security Posture provide contextual education alongside protection tools.
Engagement & Motivation Gap	Traditional lecture-based training fails to retain student attention or ensure knowledge consolidation.	Gamification + AI: Implementation of XP, ranking systems (Cadet to Legend), and AI-powered features (Scenario Simulator, Phishing Dojo with AI-generated emails, Security Buddy with daily challenges) to drive behavioral reinforcement through personalized, adaptive engagement.
Accessibility Gap	High subscription costs for premium tools create inequity, making digital safety a privilege rather than a right.	Free Access Model: Utilizes free API tiers (OpenRouter AI) and local processing to ensure zero cost for the end-user while maintaining advanced AI capabilities.

Table 2: Summary of Research Gaps and solution

2.7 Conclusion

This literature review has traced the contours of a field in crisis. We have examined 26 peer-reviewed sources and analyzed six major platforms to confirm a disturbing reality: while threats to K-12 students escalate dramatically, current solutions remain fragmented, developmentally inappropriate, and economically inaccessible. The academic consensus is clear we need integrated approaches that combine practical utility with educational content, delivered through engaging, age-appropriate interfaces.

Our analysis suggests that the Cyber Hygiene Educator & Assistant addresses these needs through its unified platform design, pedagogical integration of protection and education, evidence-based gamification, and commitment to universal accessibility. We do not present this project as a final solution, but as a necessary step toward a future where digital safety education matches the sophistication of the threats students face daily. The following chapter details the methodology and system architecture we employed to transform these research insights into functional software.

Chapter 3

Project Management

3.1 Process Models

Our project Cyber Hygiene Educator and Assistant (CHEA) has adopted the Agile Software Development methodology with particular emphasis on the Scrum framework. The reason for choosing this model was its ability to adapt and accommodate the changing requirements, which are essential to ensure the efficient development of our system designed to serve both children, aged 9-15 and adult users.

The incremental nature of Agile allows our team to deliver the functional components of system in short cycles, enabling continuous user feedback and rapid response to change. This characteristic is extremely important for CHEA, as the user interface and educational content must be improved based on testing with the targeted age demographic.

Since CHEA consists of multiple components- such as the Phishing game, Password Management, threat detection and analysis module, Encryption Tools- Agile allows the team to develop and test each module in manageable iterations, ensuring steady progress. Its flexibility is especially valuable given that user testing with children may uncover usability issues; Agile makes it easy to adjust the interface without disrupting the entire project. Regular feedback from the target audience ensures that gamified features like XP, quests, and badges remain engaging and age-appropriate. Finally, Agile reduces risk by enabling early delivery of functional modules – such as Password management- so that even if later features face delays, CHEA still has working deliverables to showcase.

3.2 Scrum Implementation for CHEA

The iterative structure of the Scrum process, as shown in Figure 2, provides a well-defined yet versatile approach to project management. Throughout the development lifecycle, core Scrum principles will be applied systematically:

Sprint planning: At the start of each sprint, the team meets to establish the scope of work and define the specific features to be completed. This formal planning process ensures clarity of objectives, alignment among team members, and a structured roadmap for the sprint. By setting priorities and the responsibilities at the beginning, the team is able to maintain focus, enhance accountability and progress efficiently while retaining the flexibility to adapt to evolving requirements or user feedback.

Daily stand-Up: Short daily meetings will be held to ensure consistent progress, address obstacles, and coordinate tasks among team members.

Sprint Reviews: At the end of each sprint, the team will demonstrate the completed working components to the project supervisors for feedback and evaluation.

Sprint Retrospective: The team will reflect on the process and identify areas for enhancement in future sprints, fostering continuous improvement.

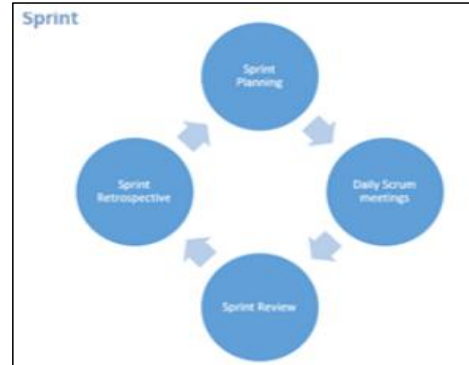


Figure 2: Scrum Process Model

3.2 Risk Management

Risk management is an indispensable element of our project plan involving a structured process of identifying, evaluating, prioritizing risks, developing and implementing appropriate mitigation strategies. Through this process, uncertainties and potential threats that may adversely affect our project scope, timeline, quality are thoroughly evaluated and managed.

In the context of our technical initiative which is not just targeted towards children aged 9-15 but also adults, a complete risk analysis is absolutely necessary. Proactive risk management ensures that core objectives- such as sustained user engagement, educational effectiveness, system reliability, data security, and timely delivery- are systematically protected against foreseeable complications.

By embedding risk management throughout the project lifecycle, the CHEA enhances its capacity to detect potential obstacles, allocate resources efficiently, and maintain alignment with the stakeholders' expectations. Each identified risk has been assessed based on its probability (Low, Medium, High) and potential impact (Low, Medium, High), generating an overall risk level. For each risk we identified two types of strategies:

Mitigation Strategy: Proactive measures to reduce the chance of risk occurring.

Contingency Plan: Reactive measures if the risk occurs despite the mitigation.

Table 3: CHEA Risk Management Plan

ID	Risk Description	Probability	Impact	Risk Level	Mitigation Strategy	Contingency Plan
R1	AI Integration Complexity	High	High	High	Implement AI in phases, begin with rule-based responses,	If AI integration becomes too complex or fails to perform as we expected, then the project

					develop isolated prototypes before full integration.	will rely on pre-trained models.
R2	External API Dependency	Medium	Medium	Medium	Research API documentation early; implement error handling for API unavailability.	If the strategy fails, then develop offline fallbacks with basic URL checks.
R3	AI Response Hallucinations	Medium	High	High	Implement system prompts with strict output validation; parse AI responses and validate against JSON schemas before displaying to users; add human review for edge cases.	If hallucinations occur, display fallback messages; disable specific features temporarily and notify users of temporary limitations.
R4	AI API Rate Limits	Medium	Medium	Medium	Implement request batching and caching; store AI-generated content locally to minimize repeated API calls; monitor usage patterns.	If rate limits are exceeded, activate local fallback content; queue requests for later processing; notify users of delayed AI responses.
R5	Encryption Implementation Vulnerabilities	Low	High	Medium	Use established, audited encryption libraries, conduct peer code review.	Mark the modules with appropriate warnings such as “educational purposes only” if security issues continue.
R6	Scope Creep	High	Medium	High	Maintain sprint backlog; Request supervisor approval for major changes.	Reprioritize features with supervisor.
R7	Schedule Delay Propagation	Medium	High	High	Identify critical path tasks; track progress through daily stand-ups.	Revise schedule with supervisor; focus on must-have features for the final product.
R8	Interface Unsuitable for Users	Medium	High	High	Conduct extensive usability testing with the target age group after each sprint; incorporate feedback into later iterations.	Develop simplified interface mode with larger buttons and visual cues.

R9	Phishing Game Content Inappropriateness	Medium	High	High	Design questions with age-appropriate language; review content with supervisor.	Add enhanced hints and explanations; provide detailed feedback for wrong answers.
R10	Low Engagement with Gamification	Medium	Medium	Medium	Design reward systems.	Adjust reward values and frequency based on user feedback.
R11	Data Loss	Medium	High	High	Maintain GitHub repository with frequent commits or store important documents code files in cloud storage.	Recover documents from cloud backup, or restore code from GitHub.

3.3 Project Activities Plan

The CHEA project development plan is structured through a Gantt chart, presenting a phased timeline from February to April. This schedule serves as a clear roadmap, enabling effective management of time and resources while functioning as a key risk mitigation strategy against any potential delays. The project is divided into distinct phases, with all deadlines strictly aligned to academic calendar.

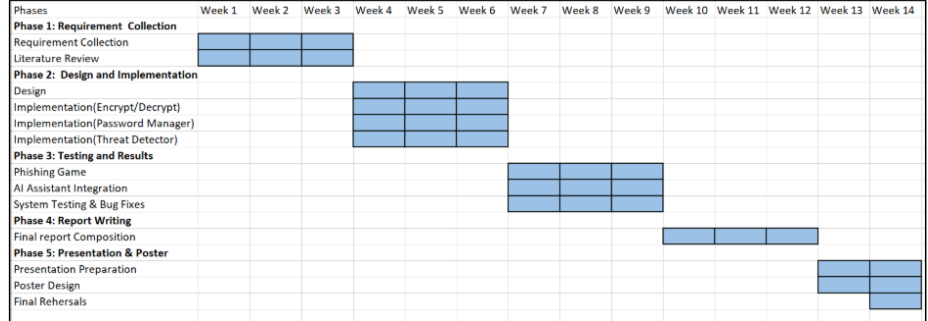


Figure 3: Gantt Chart

Phase 1 (Weeks 1-3): The foundational phase of our project includes Requirement Collection and Literature Review. This stage is primarily dedicated to the initiation of the project, during which the overall scope is defined and the needs and expectations of the intended users are systematically identified. Additionally, a comprehensive review of existing literature, related research, and competitor solutions is conducted to establish a solid theoretical and practical foundation of the project. This process identifies potential gaps, and informs the subsequent stages of system design and development.

Phase 2 (Weeks 4-6): This phase includes Design and Implementation activities. During this stage, the focus shifts towards development of the system's architectural framework, ensuring that the overall structure effectively supports the project's functional and non-functional requirements. Furthermore, this phase involves the initiation of core module development, including the implementation of important system components, such as the Encryption/Decryption Tool, Password Management, and Threat detector. Additionally, this phase includes the integration of six AI-powered features: AI Security Buddy (personalized daily challenges), AI Scenario Simulator (interactive training scenarios), AI Attack Narrative Generator (streaming security incident stories), AI Scan Analysis (plain-language threat explanations), AI Phishing Email Generator (dynamic phishing training emails), and AI Security Posture Insights (personalized security recommendations).

Phase 3 (Weeks 7-9): This phase is committed entirely to Testing and Results. During this stage, the developed system will undergo comprehensive testing to ensure all the components of the system function properly and meet all the requirements. The testing process includes system testing, which will verify the integration and overall functionality of the system. Furthermore, a test will be conducted with the target demographic to evaluate the usability, reliability and user satisfaction. Additionally, This phase will also involve identifying and resolving bugs and system errors.

Phase 4 (Weeks 10-12): This phase focuses on Report Writing. Our team will compile the final project documentation, including details on system design, implementation details, testing results, and conclusions.

Phase 5 (Weeks 13-14): The final phase will focus on project delivery and formal conclusion, including Presentation preparation and Poster Design for College Exhibition.

Chapter4

Requirements Collection

4.1 Requirement Elicitation

Requirements Elicitation is the systematic process of uncovering and documenting the exact needs of system from its stakeholders. For our project: CHEA, this crucial phase was accomplished through two primary methods: a systematic literature review of existing cybersecurity education research, and an online survey targeting the intended user base of children aged 8-18.

4.1.1 Literature Review Findings

A systematic literature review was conducted, examining 23 peer-reviewed sources and analyzing 6 major existing platforms (Hack the Box, KnowBe4, Google Interland, VirusTotal, Bitwarden and Kaspersky). The review aimed to identify gaps between current cybersecurity threats facing students and the educational responses designed to address them.

Key Findings from the Literature review:

Table 4: Findings from Literature Review

Finding	Source	Suggestions for CHEA
81% of data breaches stem from weak passwords, while 94% of malware arrives via email.	Verizon DBIR (2021)	Password and email security must be core features.
The education sector has seen 300% increase in cyberattacks since 2020.	Check Point Research (2022)	Validates need for student-focused security tools.
There is a lack of practical tools to assess cybersecurity maturity in education systems. Existing frameworks are largely theoretical.	Almomani et al. (2021)	CHEA should function as a practical self assessment tool that provides instant feedback to students.
Existing tools are fragmented, forcing users to choose between education and protection.	Kennedy and Chiasson (2021)	CHEA must integrate both practical and educational tools.
Gamification significantly improves engagement and knowledge maintenance in security context.	Pramod (2025), Khairallah and Abu-Naseer (2024)	Game-based learning is important for student engagement.

Existing interfaces assume adult cognitive processing and technical vocabulary.	Sağlam and Miller (2023)	Age-appropriate designs with feedback are needed.
---	--	---

The literature review also identified five critical research gaps that directly inform CHEA’s requirements: the Tool Integration Gap, Age-Appropriateness Gap, Education-Protection Gap, Engagement and Motivation Gap, and Accessibility Gap. These gaps collectively justify the need for an integrated, free; gamified cybersecurity education platform targeting students aged 8-15.

4.1.2 Online Survey

To balance the literature review and gather data from the target audience, an online survey was conducted. The survey targeted children aged 8-18, as this age group represented the primary users of the proposed applications. 104 responses were collected.

Table 5: Survey Demographics

Category	Percentage	Number of Respondents
Age 8-12	15.4%	16
Age 13-15	27.9%	29
Age 16-18	56.7%	59

4.1.2.1 Summary of Survey Findings:

The survey revealed several critical insights about students’ online behaviors and security awareness. Regarding online threats, 71.2% of respondents had encountered messages offering free items, 67.3% have seen websites that looked fake, and 47.1% had received suspicious emails asking for personal information, confirming that students regularly face common cyber threats. In terms of password practices, 71.1% of respondents admitted to using either the same password for everything or only slightly varying their passwords across accounts. This indicated a clear need for password education tools. Furthermore, more than half of the respondents stated that they do not know how to tell if a link is safe to click, while 53.8% admitted they would click a link sent by a friend immediately without verification. These findings highlight significant gaps in both knowledge and behavior. When asked about desired features, password generators and URL checkers were the most frequently selected tools, and 93.3% of respondents expressed interest in having a friendly AI assistant within the app to explain why a link or password is unsafe. Learning preference favored gamified approaches with points and rewards and using different tools to learn from. Finally, respondents revealed strong interest in an integrated platform combining security tools, learning and games- with most indicating they would likely use such an application.

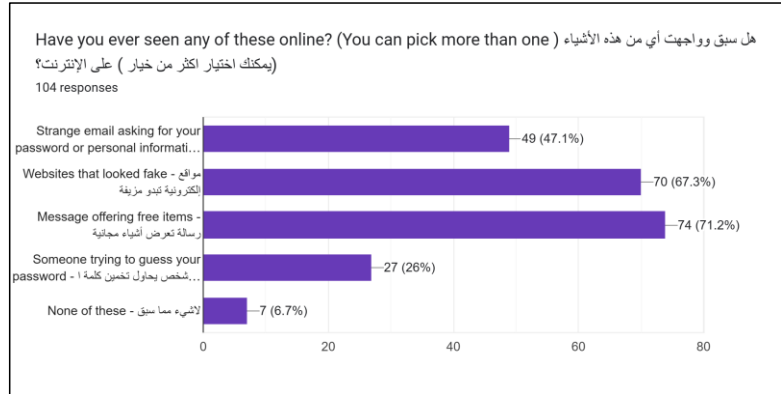


Figure 4: Online Threats Encountered

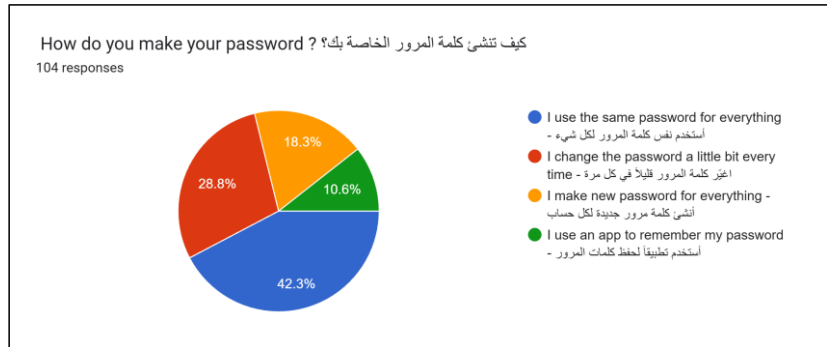


Figure 5: Students Password Habits

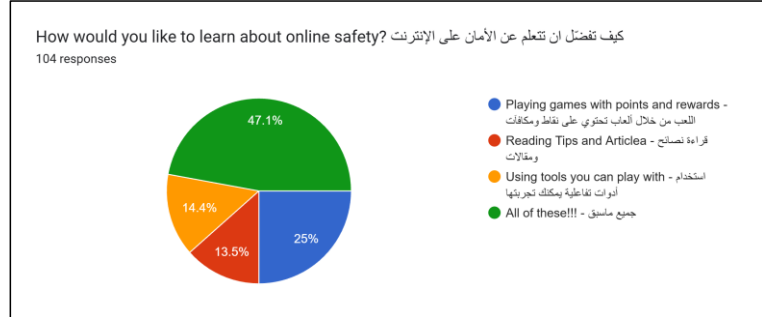


Figure 6: Students Learning Preference

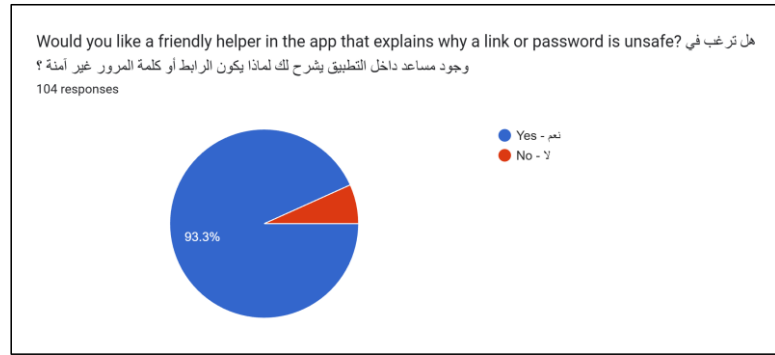


Figure 7: Students interest in AI assistant feature

This requirement elicitation process ensures that the CHEA system is built upon both established academic research and direct input from the target user base, maximizing the likelihood of adoption, engagement and educational effectiveness.

4.2 System Requirements

From our literature review of 26 peer-reviewed sources and a survey of 69 students, we identified what CHEA needed to actually do. This section lays out the functional and non-functional requirements we worked on.

4.2.1 Functional Requirements

These are the features CHEA had to implement. We derived them directly from the research gaps in the literature (Zhang-Kennedy and Chiasson, 2021) and confirmed them through our survey students genuinely want integrated security tools in one place, not scattered across different apps.

Table 6: Functional Requirements

Req. ID	Requirement	Description
FR-01	Password Generation and Strength Analysis	The system allows users to generate secure passwords with configurable parameters (length, character types) and analyses password strength by calculating entropy, estimating crack time, and detecting common patterns.
FR-02	Credential Vault with Encryption	The system provides an encrypted vault for storing login credentials and credit card details, protected by a master password. All data is encrypted on client-side using AES-256-GCM before storage.
FR-03	URL Link Scanning	The system allows users to submit URLs for security analysis via the VirusTotal API, displaying scan results from 70+ security engines with aggregate statistics.
FR-04	File Malware Scanning	The system allows users to scan files for malware by computing a local SHA-256 hash and checking against VirusTotal's database.
FR-05	Text Encryption and Decryption	The system allows users to encrypt and decrypt text using AES-256-GCM, ChaCha20-Poly1305, or AES-128-CBC, with Argon2id key derivation.
FR-06	AI-Powered Cybersecurity Chatbot	The system provides an AI chatbot that answers cybersecurity questions via the OpenRouter API with streaming responses.
FR-07	Phishing Email Recognition Training	The system provides a gamified phishing identification exercise across three difficulty tiers, with detailed feedback including red flag explanations.
FR-08	Cybersecurity Knowledge Quiz	The system provides a quiz game with cybersecurity trivia questions across multiple difficulty tiers, tracking scores and awarding XP.
FR-09	Image Privacy Scanner	The system scans JPEG and PNG images for EXIF metadata, extracting GPS coordinates, timestamps, and camera settings, with a metadata stripping function.

FR-10	Gamification and Progress Tracking	The system implements XP, a 10-tier level progression system, daily streak tracking, and daily task objectives.
FR-11	AI Scan Analysis	The system provides AI-powered threat analysis for URL and file scan results, using a large language model via the OpenRouter API to explain VirusTotal detection results in student-friendly language with risk summaries, threat breakdowns, and actionable safety recommendations.
FR-12	AI Password Attack Narrative	The system generates an AI-powered narrative describing how an attacker could realistically crack the user's password. The analysis uses only password traits (length, character types, detected patterns) without transmitting the raw password, educating users on specific weaknesses in their password choices.
FR-13	AI Phishing Email Generation	The system generates realistic phishing emails using AI for the Phishing Dojo AI Challenge mode, producing emails that conform to the existing PhishingEmail data structure with sender, subject, body, and identifiable red flags. Each email is generated one-at-a-time to maintain challenge progression.
FR-14	AI Security Scenario Simulator	The system provides an AI-powered game where students face 5 rounds of realistic security decision scenarios. For each round, the AI generates a context-aware situation with multiple choice options, identifies the correct answer, and provides educational feedback explaining why the chosen answer is correct or incorrect. Scenarios are generated dynamically and are language-aware..
FR-15	AI Security Posture Assessment	The system provides a 15-question security behavior questionnaire that covers topics including password practices, email safety, social media privacy, and browsing habits. After the student answers all questions, the system compiles the responses and sends them to the AI, which generates a personalized security report card identifying strengths, weaknesses, an overall score, and actionable recommendations.

FR-16	AI Security Buddy	The system provides a dashboard widget that delivers AI-generated daily cybersecurity challenges personalized to the user. Each day, the AI generates a unique challenge with a title, description, and related tool suggestion. The system caches challenges in localStorage keyed by user ID, date, and language, and provides fallback challenges for offline reliability. Challenges are language-aware and regenerate when the user switches between English and Arabic.
--------------	-------------------	---

4.2.2 Non-Functional Requirements

These are the quality constraints we had to meet. The literature pointed out gaps in age-appropriate design (Sağlam and Miller, 2023) and accessibility, so we paid extra attention to those areas.

Table 7: Non-Functional Requirements

Req. ID	Category	Requirement
NFR-01	Security	All sensitive data is encrypted client-side using AES-256-GCM with Argon2id key derivation. API keys are stored exclusively in the Rust backend.
NFR-02	Usability	The interface is designed for students aged 9-15, using visual indicators, non-technical language, a cyberpunk-themed aesthetic, and collapsible step-by-step Quick Guide banners on tool pages to support first-time users unfamiliar with cybersecurity tools.
NFR-03	Performance	Local cryptographic operations complete in under 500 milliseconds. Application cold start does not exceed 5 seconds.
NFR-04	Reliability	The system handles API failures gracefully with user-friendly error messages and retry mechanisms.
NFR-05	Portability	Packaged as a native desktop application using Tauri v2, supporting Windows and macOS from a single codebase.
NFR-06	Accessibility	Supports light and dark modes, keyboard navigation, and WCAG 2.1 AA compliant text contrast.

NFR-07	Maintainability	Frontend codebase is organized into modular components with strict TypeScript type-checking.
NFR-08	Privacy	The system does not collect or transmit user data without explicit action. Master passwords never leave the user's device.

4.3 System Models

To design CHEA properly, we needed clear models of how the system would work, how it's structured, how data flows through it, and how the database is organized. These models turned our requirements into something we could actually build from.

4.3.1 System Architecture

We went with a layered architecture because it keeps things clean: the UI doesn't need to know about crypto, the crypto code doesn't need to know about databases, and so on. We split CHEA into five layers.

Presentation Layer:

This is the React 18 + TypeScript frontend, rendered inside a Tauri WebView2 container. It includes the dashboard layout, all 13 feature pages (Link Scanner, File Scanner, Password Generator, Encryption Lab, Credential Vault, AI Agent, Quiz Arena, Phishing Dojo, Image Privacy, Scenario Simulator, Security Posture), the Security Buddy dashboard widget, authentication forms, and the custom title bar. We used Tailwind CSS for styling with a cyberpunk/neon design system that supports both light and dark themes, and Framer Motion for page transition animations. The interface is fully bilingual, supporting English and Arabic (with RTL layout) through the i18next internationalization framework, with all AI-generated content respecting the user's selected language.

Business Logic Layer:

This layer sits between the UI and the Rust backend. It's made up of Zustand stores (useAuthStore, useActivityStore, useUserProgressStore, useDailyTasksStore) and Firebase service modules. It handles auth state, gamification logic (XP calculation, level progression, streak tracking), activity logging, and data formatting between frontend and backend. The useTrackActivity custom hook is the central piece here whenever a user completes a security action, this hook logs the activity, awards XP, and updates daily tasks all at once.

Backend Processing Layer:

This is the Rust side of things. We implemented it as a set of Tauri command modules that handle anything computationally heavy or security-sensitive things we didn't want to run in JavaScript. The modules are: crypto.rs (AES-256-GCM, ChaCha20-Poly1305, AES-128-CBC encryption with Argon2id key derivation), virustotal.rs (URL and file scanning via VirusTotal API v3), ai_agent.rs (streaming chat via OpenRouter API with Server-Sent Events), image_privacy.rs (EXIF metadata

scanning and stripping for JPEG/PNG), terminal.rs (pseudo-terminal emulation), and diagram.rs (Mermaid diagram export).

Data Layer:

We used Firebase Firestore, a NoSQL cloud database. Data is organized hierarchically under `users/{userId}/`, with subcollections for vault credentials, chat sessions, user progress, daily tasks, and activity logs. One issue we ran into early on: WebView2 doesn't support Firestore's gRPC-web transport, so we had to build a custom REST API helper (`firestore-rest.ts`) that does database operations through standard HTTPS fetch calls with Firebase ID token auth. It was frustrating to debug, but it works reliably now.

External Services Layer:

These are the third-party APIs CHEA connects to. VirusTotal API v3 handles URL and file threat analysis using 70+ security engines. OpenRouter API gives us access to large language models (we use Grok-4-Fast) for all AI-powered features across the application: the AI chatbot, scan result analysis, password attack narratives, phishing email generation, security scenario simulation, security posture assessment, and daily security challenges. A centralized AI service module (`aiService.ts`) provides `callNova()` and `callNovaStreaming()` utility functions used by all AI features, ensuring consistent API communication and error handling. Firebase Auth manages user authentication (email/password registration, login, password reset). All API keys are stored in the Rust backend; they never reach the frontend.

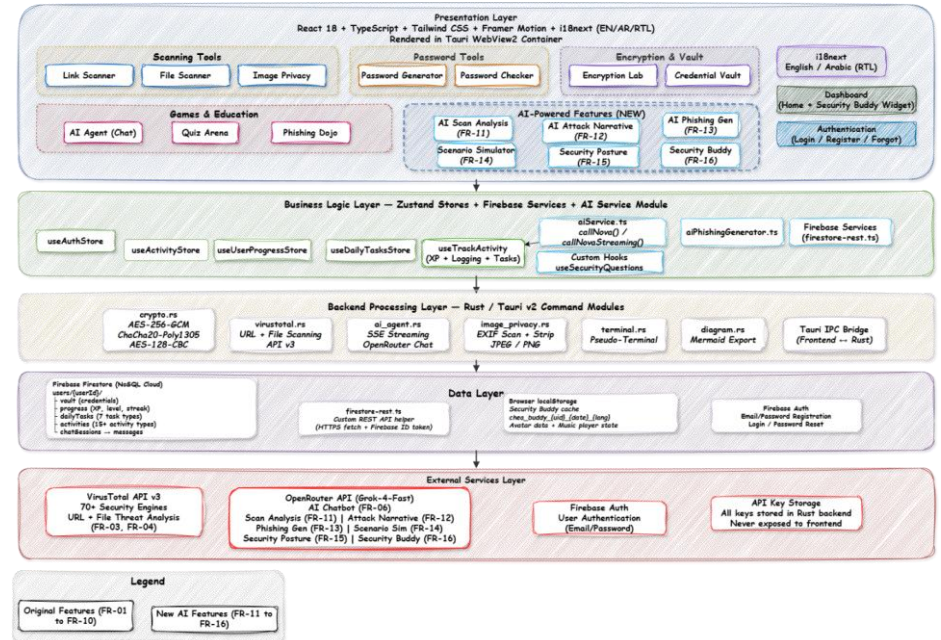


Figure 8: System Architecture

4.3.2 Use Case Model

We identified four main actors that interact with CHEA:

The student is the primary actor, aged 9-15. They interact with all the security tools, educational features, and gamification elements. Everything in CHEA revolves around this actor. The CHEA System itself acts as a secondary actor; it processes user inputs, manages state, runs cryptographic operations, and renders the UI. The VirusTotal API receives URL and file scan requests from CHEA and returns threat analysis results. The OpenRouter AI API receives chat messages from CHEA and returns streaming cybersecurity educational responses via Server-Sent Events. The OpenRouter AI API also powers six additional AI-driven features beyond the chatbot: it generates threat explanations for scan results, creates password attack narratives, produces phishing training emails, generates security scenario challenges, produces personalized security report cards, and delivers daily cybersecurity challenges.

For security tools, students can generate secure passwords, analyses password strength, scan URLs for threats, scan files for malware, encrypt and decrypt text, manage the credential vault, and scan or strip image metadata. AI-enhanced security tools include receiving AI-powered explanations of scan results (for both URLs and files) and viewing AI-generated attack narratives that explain how their password could be cracked. Educational features include chatting with the AI assistant, practicing phishing

identification (with both static and AI-generated challenges), completing cybersecurity quizzes, playing the AI Scenario Simulator (5 rounds of interactive security decisions), and completing the AI Security Posture Assessment (15 questions with a personalized report card). The AI Security Buddy widget on the dashboard delivers daily AI-generated cybersecurity challenges. System operations cover account registration, login/logout, progress tracking, and switching between English and Arabic languages.

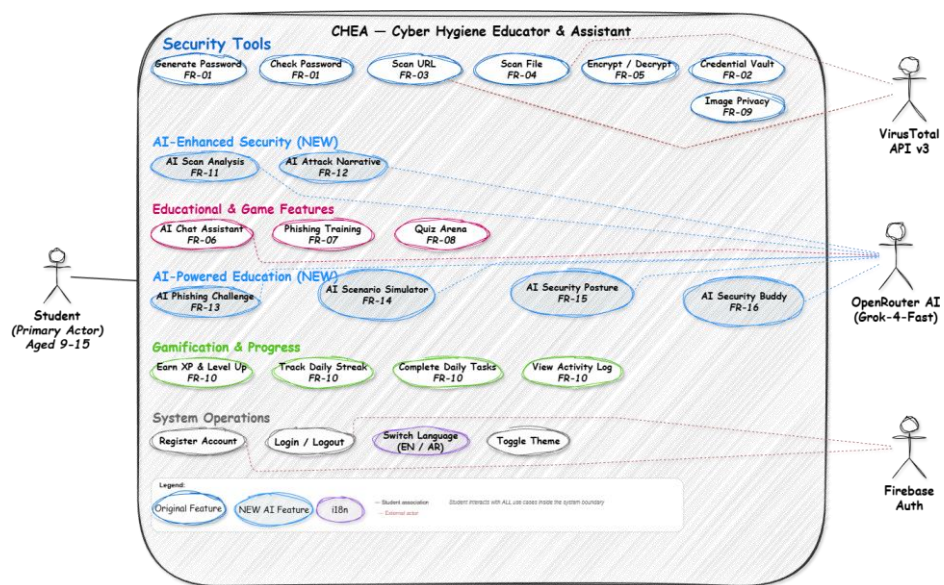


Figure 9: Use Case Model

4.3.3 Data Flow Model

Here's how data actually flows through the system for our key features.

- **URL Scanning:**

URL scanning involves the frontend, Rust backend, and VirusTotal working together. The student enters a URL in the Link Scanner interface. The React frontend validates the URL format and calls the Tauri IPC command `scan_url` with the URL as a parameter. The Rust `virustotal.rs` module receives the URL, URL-encodes it, and submits a POST request to the VirusTotal `/urls` endpoint with the API key in the `x-apikey` header. VirusTotal returns an analysis ID. The Rust module enters a polling loop, querying the `/analyses/{id}` endpoint at 4-second intervals. When the analysis status becomes "completed," the Rust module parses the response into a `ScanResult` structure containing status, reputation, detections, and statistics. The `ScanResult` is serialized as JSON and returned to the frontend via Tauri IPC. The React

frontend displays the results, logs the activity to Firestore, awards XP to the user, and updates the daily task counter.

- **Text Encryption:**

The encryption pipeline runs entirely on the client side. The student enters plaintext and a password in the Encryption Lab interface, and selects an algorithm (AES-256-GCM, ChaCha20-Poly1305, or AES-128-CBC). The React frontend calls the Tauri IPC command `encrypt_text` with the plaintext, password, and algorithm. The Rust `crypto.rs` module generates a random 16-byte salt using the OS cryptographic RNG (`OsRng`). The module derives an encryption key from the password and salt using `Argon2id` (32 bytes for AES-256-GCM and ChaCha20-Poly1305, 16 bytes for AES-128-CBC). A random nonce/IV is generated (12 bytes for GCM/ChaCha20, 16 bytes for CBC). The plaintext is encrypted using the selected algorithm. The output (algorithm identifier, Base64-encoded salt, IV, and ciphertext) is serialised as a JSON `EncryptedPayload`, then Base64-encoded for transport. The encrypted string is returned to the frontend and displayed to the student.

- **AI Chat Interaction:**

The AI chat uses a streaming architecture, so responses appear in real time. The student types a message in the AI Agent chat interface. The React frontend calls the Tauri IPC command `chat_with_ai` with the message and conversation history, passing a Tauri Channel for streaming. The Rust `ai_agent.rs` module prepends the cybersecurity-only system prompt and sends a streaming POST request to the OpenRouter API. OpenRouter returns a Server-Sent Events (SSE) stream. The Rust module reads data: lines, extracts content chunks from `choices[0].delta.content`. Each content chunk is immediately sent to the frontend via the Tauri Channel, enabling character-by-character rendering. The data: [DONE] marker signals stream completion. The frontend renders the complete message with markdown formatting, logs the activity, and awards XP.

- **AI Scan Analysis:**

After a URL or file scan completes and VirusTotal returns results, an additional AI analysis step enriches the user experience with a student-friendly explanation. The React frontend compiles a summary of the scan results (detection count, threat categories, reputation) into a structured prompt. The centralized `aiService.ts` module sends this prompt to the OpenRouter API via the `callNova()` function, which makes a POST request to the `/chat/completions` endpoint with the Grok-4-Fast model. The AI prompt includes an instruction to generate the response in the user's current language (English or Arabic). OpenRouter returns a JSON response containing the AI-generated explanation with risk assessment, threat breakdown, and actionable safety recommendations. The `ScanAIAnalysis` React component renders this analysis in a dedicated card below the scan results, complete with loading states and error

handling. The analysis is generated fresh each time to reflect the specific scan results, and the raw URL or file content is never sent to the AI only the parsed scan summary.

- **AI Scenario Simulator:**

The Scenario Simulator uses the AI to dynamically generate security decision challenges across 5 rounds. When the student starts a new game, the React frontend calls the centralized `callNova()` function from `aiService.ts` with a prompt instructing the AI to generate a security scenario in JSON format containing a situation description, four multiple-choice options, the correct answer index, and an explanation. The prompt includes the current round number (tracked via a `useRef` to avoid stale closures in async operations) and an instruction to generate content in the user's selected language. The AI returns the scenario as structured JSON, which the frontend parses and presents to the student. The student selects an answer, and the frontend evaluates whether it matches the correct answer, awards points, and shows the AI-generated explanation. This process repeats for rounds 2 through 5, with each round's scenario building in complexity. After all, 5 rounds, the total score is calculated and logged as a `scenario_simulator` activity, awarding XP to the user.

- **AI Security Posture Assessment:**

The Security Posture Assessment combines a static questionnaire with AI-powered personalized reporting. The student answers 15 security behavior questions covering password practices, email safety, social media privacy, browsing habits, and device security. Questions are provided in both English and Arabic via a dedicated `useSecurityQuestions` hook that returns the correct language version based on the user's `i18n` preference. After the student submits all answers, the React frontend compiles the question-answer pairs into a structured prompt. The `callNova()` function sends this prompt to the OpenRouter API, requesting a comprehensive security report card in JSON format. The AI generates an overall security score (0-100), a grade label, categorized strengths and weaknesses, and specific actionable recommendations all in the user's selected language. The `SecurityPosture` React component renders this report card with visual score indicators and categorized feedback. The activity is logged as `security_posture`, and the user earns XP for completing the assessment.

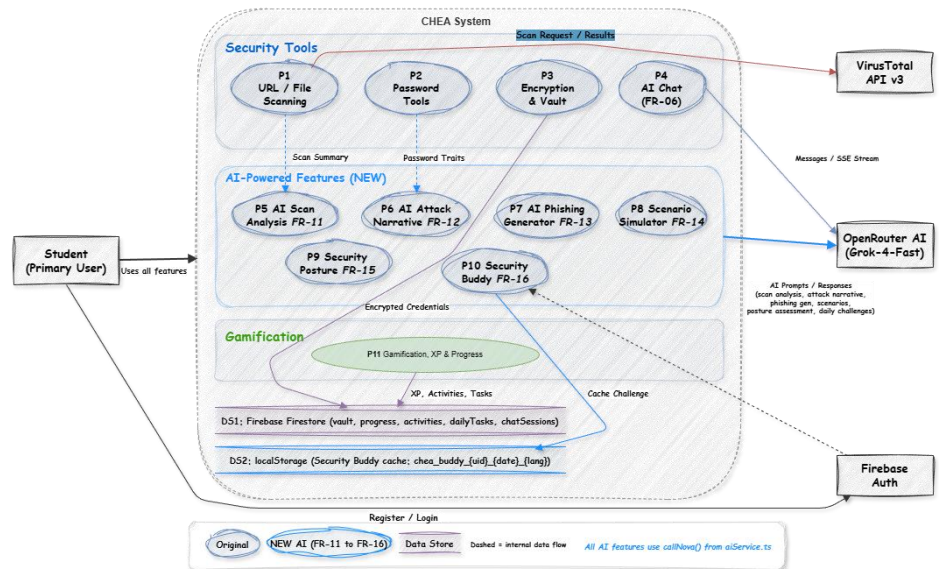


Figure 10: Data Flow Model

4.3.4 Entity Relationship Model

Our database uses Firebase Firestore, which is a NoSQL document database so instead of traditional relational tables, we have collections and documents organised in a hierarchy. All data lives under the users/{userId} path. This keeps each user's data isolated and makes queries within a user's context fast.

The vault subcollection stores individual credential entries. Each document contains name (display name for the credential), username (username or email associated with the account), domain (website domain or service name), type (credential type: login or card), encryptedData (Base64-encoded encrypted payload containing the password or card details), and createdAt (document creation timestamp). The encryptedData field stores the AES-256-GCM encrypted ciphertext produced by the Rust crypto.rs module. Only the encrypted payload is sent to Firestore. Decryption always happens on the user's device.

The progress subcollection stores the user's gamification state: total XP accumulated, current level (1-10), cumulative score across all activities, consecutive days of activity, and the last active date. The dailyTasks subcollection stores date-keyed documents containing the user's daily task progress, with seven daily task types defined: scan, generate_password, check_password, create_credential, use_encryption, play_quiz, and spot_phish.

The activities subcollection stores an activity log with entries documenting each security action performed by the user, including the activity type (scan_link, scan_file, scan_image,

generate_password, check_password, generate_encryption, create_credential, chat_ai, quiz_round, phishing_round, ai_phishing_round, quiz_streak, phishing_streak, scenario_simulator, security_posture), additional context metadata, and timestamp. The chatSessions subcollection stores AI chat session metadata, with a nested messages subcollection for individual messages containing role (user or assistant), content, and timestamp. Additionally, the AI Security Buddy widget caches its daily challenges in the browser's localStorage, keyed by a combination of user ID, date, and language (chea_buddy_{uid}_{date}_{lang}) to support offline access and language-specific caching.

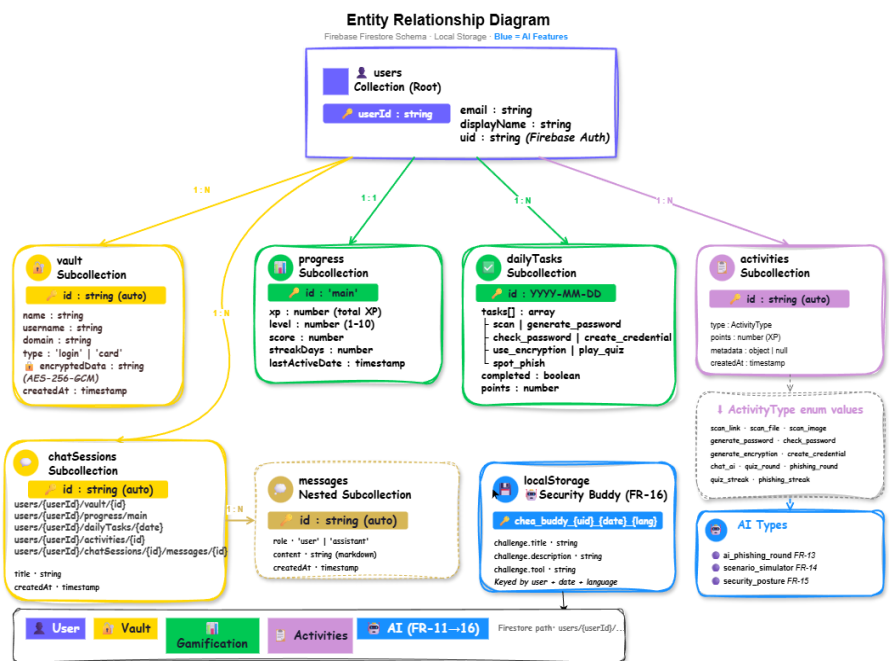


Figure 11: Entity Relationship Model

Chapter 5

System Design

5.1 Introduction

This chapter outlines the detailed system design of the Cyber Hygiene Educator & Assistant (CHEA). The design translates the functional and non-functional requirements defined in Chapter 4 into concrete specifications for the database schema, user interfaces, and core algorithms. The design decisions are guided by the research gaps identified in the literature review (Chapter 2), particularly the need for tool integration, age-appropriate interfaces (Sağlam and Miller, 2023), and the combination of education with practical protection (Zhang-Kennedy and Chiasson, 2021).

The chapter is organized into three main sections. Section 5.2 Database Schema Design presents the Firebase Firestore database schema, detailing all collections, document structures, field types, and constraints. Section 5.3 User Interface Design describes the key user interface designs, covering twenty-one core interfaces with their layout, components, navigation flow, and user interaction patterns including six AI-powered features (FR-11 through FR-16) that enhance existing tools with intelligent analysis and introduce new AI-driven educational experiences. Section 5.5 Algorithm Design provides algorithm designs, expressed in pseudocode, for all core system functions including encryption, decryption, password generation, password strength analysis, URL scanning, file scanning, image metadata processing, AI-powered scan analysis, attack narrative generation, AI phishing email generation, scenario simulation, security posture assessment, and the gamification engine.

5.2 Database Schema Design

We chose Firebase Firestore as our database because it offers real-time synchronization, automatic horizontal scaling, and native integration with Firebase Authentication all essential for a desktop application that needs to persist user data across sessions. Firestore is a NoSQL document database, so instead of relational tables with fixed schemas, we use collections and documents organized in a hierarchical structure.

All data in CHEA is scoped under the `users/{userId}` path, where `userId` is the Firebase Authentication UID. This design ensures complete data isolation between users and allows Firestore security rules to restrict access to each user's own data only. The schema was designed around CHEA's six core functional areas: credential storage, gamification progress tracking, activity logging, daily task management, AI chat session persistence, and AI-powered feature tracking. Six AI-enhanced features (FR-11 through FR-16) extend the activity logging system with new activity types (`ai_phishing_round`, `scenario_simulator`, `security_posture`) and introduce a client-side `localStorage` cache for the Security Buddy daily challenges widget (detailed in Section 5.3.18 AI Security Posture Assessment Interface).

The complete Firestore schema is presented in Table 8.

Table 8: Complete Firestore Database Schema

Collection Path	Document ID	Field Name	Data Type	Constraints	Description
users/{userId}/vault	Auto-generated	name	String	Required	Display name for the credential (e.g., "GitHub", "School Email")
		username	String	Required	Username or email associated with the account
		domain	String	Optional	Website domain or service name
		type	String	Required: "login" "card"	Credential type login credentials or payment card details
		encryptedData	String	Required	Base64-encoded AES-256-GCM ciphertext produced by the Rust backend
		serviceId	String	Optional	Service icon reference for branded display
		createdAt	Timestamp	Auto (server)	Document creation timestamp
users/{userId}/vault Config	main	encryptedVerifyHash	String	Required	AES-256-GCM encrypted verification string used to validate the master password without storing it in plaintext
users/{userId}/progress	data	xp	Number	Default: 0	Total experience points accumulated across all activities
		level	Number	Range: 1–10	Current user level, computed from XP thresholds
		totalScore	Number	Default: 0	Cumulative score across all activities
		streakDays	Number	Default: 0	Number of consecutive days with at least one activity
		lastActiveDate	String	ISO 8601 date (YYYY-MM-DD)	Date of the user's last activity, used for streak calculation
		createdAt	Timestamp	Auto (server)	Document initialisation timestamp

users/{userId}/dailyTasks	ISO date string (e.g., "2026-04-10")	tasks	Array of Objects	Required	Daily task list — each object contains id, type, description, target, current, points, and completed
		date	String	Required	ISO date string matching the document ID
		totalScore	Number	Default: 0	Sum of completed task points for the day
		createdAt	Timestamp	Auto (server)	Document creation timestamp
users/{userId}/activities	Auto-generated	type	String	Required	Activity type identifier (e.g., scan_link, generate_password, quiz_round)
		description	String	Required	Human-readable description of the activity
		points	Number	Default: 0	XP earned from this activity
		metadata	Map (String → String)	Optional	Additional context about the activity (e.g., URL scanned, algorithm used)
		createdAt	Timestamp	Auto (server)	Activity timestamp
users/{userId}/chatSessions	Auto-generated	title	String	Required	Session title, auto-generated from the first user message
		createdAt	Timestamp	Auto (server)	Session creation time
		updatedAt	Timestamp	Auto (server)	Timestamp of the last message in the session
users/{userId}/chatSessions/{sessionId}/messages	Auto-generated	role	String	Required: "user" "assistant"	Message sender role
		content	String	Required	Full text content of the message
		createdAt	Timestamp	Auto (server)	Message timestamp
users/{userId}/passwordHistory	Auto-generated	encryptedPassword	String	Required	Base64-encoded encrypted password payload from the Rust crypto module
		pinned	Boolean	Default: false	Whether the user has pinned this password for quick access

		entropy	Number	Default: 0	Calculated entropy in bits for the generated password
		createdAt	Timestamp	Auto (server)	Generation timestamp

5.2.1 Encryption in the Schema

A critical design decision is that no sensitive data is stored in plaintext. The encryptedData field in the vault collection and the encryptedPassword field in the passwordHistory collection both contain Base64-encoded ciphertext produced by the Rust crypto.rs module using AES-256-GCM with Argon2id key derivation. The master password itself is never transmitted to any server only an encrypted verification hash is stored in vaultConfig/main, allowing the system to verify the password locally by attempting decryption and comparing the result against a known constant string.

This means that even if the Firestore database were compromised, all stored credentials would remain encrypted and unusable without the user's master password, which exists only in the client's memory during the active session.

Sample Document Structure:

```

    GitHub", " : ""name
    student@example.com", " : ""username
    github.com", " : ""domain
    login", " : ""type
    : ""encryptedData
    eyJhbGciOiJIJBRVMYNTZLRE1NliwiYWxnIjoiS2V5RGVyaXZhdGlvbjJEZXYifQ==...", "
    2025-04-10T14:30:00Z", " : ""createdAt
    2025-04-10T14:30:00Z"" : ""updatedAt
  }
{

```

5.2.2 Gamification Data Model

The gamification data is split across three collections that work together:

- Progress/data stores the current state: total XP, level, streak count, and last active date. The level is derived from XP using a 10-tier threshold table (see Section 5.4.5), and the streak is calculated by comparing lastActiveDate with the current and previous calendar dates.
- dailyTasks/{date} stores seven daily objectives that reset each day. Each task has a type (scan, generate_password, check_password, create_credential, use_encryption, play_quiz, spot_phish), a target count, a current progress count, a point value, and a completion flag. The maximum daily score is 125 points.
- Activities are an append-only log of every security action the user performs, storing the activity type, a description, XP earned, and optional metadata. The Dashboard retrieves the most recent 15 entries for the activity feed. Three additional AI-specific activity types were introduced for the new AI features: ai_phishing_round (logged when the

student completes an AI-generated phishing round in the Phishing Dojo, FR-13), scenario_simulator (logged when the student completes a round of the AI Scenario Simulator, FR-14), and security_posture (logged when the student completes the AI Security Posture Assessment, FR-15).

5.2.3 Firestore REST API Workaround

A significant technical constraint shaped the data access layer. Tauri's WebView2 runtime (based on Microsoft Edge) does not support the gRPC-web transport protocol used by the Firestore Web SDK. This means the standard Firestore setDoc, getDoc, and updateDoc functions fail silently or throw network errors when called from within the Tauri application.

To solve this, we implemented a custom REST API helper (firestore-rest.ts) that bypasses the Firestore SDK entirely and communicates with Firestore through standard HTTPS fetch() calls. The module provides two functions:

1. **fireGetDoc(collection, docId)** Sends a GET request to `https://firestore.googleapis.com/v1/projects/{projectId}/databases/(default)/documents/{collection}/{docId}`, authenticated with the user's Firebase ID token. Parses the Firestore REST field format (which wraps values in type objects like `{ stringValue: "...", integerValue: "42" }`) back into plain JavaScript objects.
2. **firestoreSetDoc(collection, docId, data)** - Sends a PATCH request to the same endpoint, converting plain JavaScript objects into the Firestore REST field format before transmission.

This workaround is used for all write operations that require guaranteed delivery (user registration, vault setup) and as a fallback for any SDK operation that fails due to the WebView2 gRPC limitation. For read operations, we use the Firestore SDK with `experimentalForceLongPolling: true`, which forces the SDK to use HTTP long-polling instead of gRPC, and this works reliably in WebView2.

5.2.4 Root User Document

In addition to the subcollections listed in Table 8: Complete Firestore Database Schema, the registration process creates a root document at `users/{userId}` with the following fields:

Table 9: Main User profile fields

Field	Type	Description
uid	String	Firebase Authentication UID (matches document ID)
email	String	User's registered email address
displayName	String	User's chosen display name
createdAt	Timestamp	Account creation timestamp
role	String	User role, defaulting to "student"

This root document serves as a lightweight user profile and enables future features such as a teacher dashboard or role-based access control.

5.2.5 Security Buddy Client-Side Cache

The Security Buddy widget (FR-16, detailed in Section 5.3.18 AI Security Posture Assessment Interface) generates a daily cybersecurity challenge for the student each day. To avoid redundant AI API calls and provide instant loading when the Dashboard is opened, the challenge data is cached in the browser's localStorage using a composite key:

`chea_buddy_{userId}{date}{language}`. The cached object contains the challenge title, description, and the recommended tool to try. The key incorporates the user ID, the current date (ISO format), and the active language (en or ar), ensuring that the cache automatically invalidates at midnight and refreshes when the user switches language. This design means that each unique combination of user, date, and language requires only a single AI API call per day.

5.3 User Interface Design

We adopted a cyberpunk/neon aesthetic because our target users are students aged 9–15, and we wanted the application to feel more like a game than a traditional security tool. The dark theme uses deep navy backgrounds (cyber-void) with neon crimson (neon-crimson) and violet (neon-violet) accents, glowing border effects, and animated transitions. A light theme is also available, automatically detected from the operating system preference. Navigation runs through a persistent sidebar with icon-labelled sections, and the entire application is rendered inside a frameless Tauri window with a custom title bar.

All core features are accessible within three clicks from the Dashboard. We describe the twenty-one key interfaces below, including six AI-powered features (FR-11 through FR-16) integrated into existing tools and introduced as new pages.

5.3.1 Login Interface

The Login page is the entry point for returning users. It is wrapped in a `PublicRoute` guard that redirects already-authenticated users to the Dashboard.

The interface includes:

- **Email Input Field:** A text input with an envelope icon and placeholder text "Enter your email."
- **Password Input Field:** A masked password input with a lock icon and a visibility toggle button (eye icon) that reveals or hides the password.
- **Remember Me Checkbox:** A checkbox that enables Firebase's `browserLocalPersistence`, keeping the user logged in across application restarts.
- **Login Button:** A primary action button that triggers Firebase Authentication's `signInWithEmailAndPassword`. During authentication, the button displays a loading spinner and is disabled to prevent double submission.

- **Forgot Password Link:** A text link below the form that navigates the user to /forgot-password, where they can enter their email to receive a Firebase password reset link.
- **Register Link:** A text link at the bottom that navigates to /register for new users.

If authentication fails, the interface displays a red error banner below the form with a descriptive message (e.g., "Invalid email or password"). On success, the user is redirected to /dashboard.

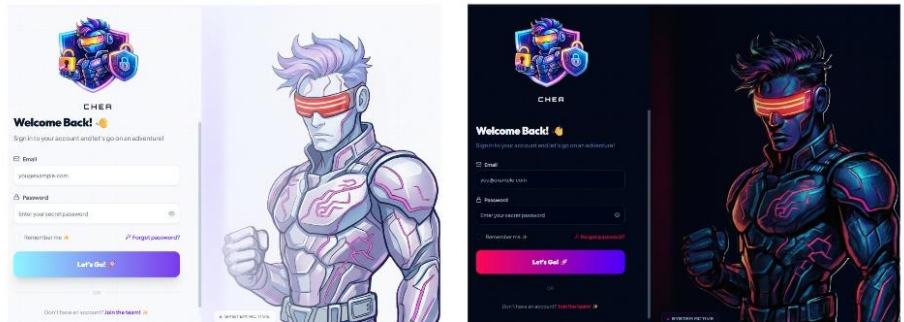


Figure 12: Login Interface (Light and dark Mode)

5.3.2 Registration Interface

The Registration page allows new users to create an account. It is wrapped in a PublicRoute guard and a shared AuthLayout component that provides the centred card layout used across all authentication pages. The interface includes:

- **Email Input:** A text input with an envelope icon and validation for proper email format.
- **Display Name Input:** A text input with a user icon for the student's chosen display name.
- **Password Input:** A masked password input with a visibility toggle. As the student types, an inline PasswordStrength meter appears below, showing the strength level and encouraging the use of strong passwords from the moment of account creation. A minimum length of 12 characters is enforced.
- **Confirm Password Input:** A second masked input for password confirmation. A real-time match indicator appears below - a red "Passwords do not match" message when they differ, and a green "Passwords match" message when they are identical.
- **Terms Agreement Checkbox:** A checkbox requiring the user to agree to the Terms of Service and Privacy Policy, with clickable links to the /terms and /privacy pages. The submit button remains disabled until this checkbox is checked.
- **Submit Button:** A full-width gradient button labelled "Join Now" that triggers the registration flow: (1) createUserWithEmailAndPassword via Firebase Auth, (2) updateProfile to set the display name, and (3) firestoreSetDoc via the REST API to create the root user document in Firestore. During processing, the button displays a spinner and is disabled.

- **Sign-In Link:** A text link at the bottom that navigates to /login for existing users.

If registration fails (e.g., email already in use, weak password), a red error banner with a descriptive message appears above the submit button.

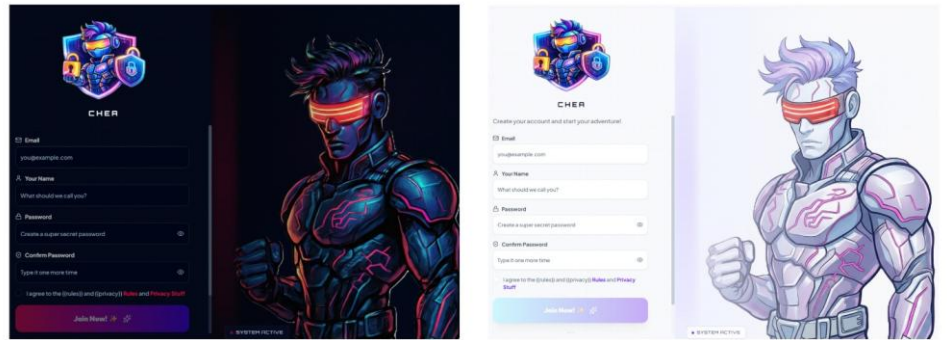


Figure 13: Registration Interface (Light and Dark Mode)

5.3.3 Forgot Password Interface

The Forgot Password page provides a two-state flow for password recovery, wrapped in AuthLayout and PublicRoute:

From State:

Email Input: A single text input with an envelope icon for entering the registered email address.

Error Banner: A red error card that appears if the reset email fails to send (e.g., "user-not-found" or "invalid-email").

Send Reset Link Button: A full-width gradient button labelled "Send Reset Link" that calls Firebase Auth's sendPasswordResetEmail. During processing, a spinner is shown.

Sign-In Link: A text link navigating back to /login.

Success State:

After the reset email is sent successfully, the form transitions to a successful screen with a spring animation: a large green checkmark icon inside a rounded badge, a "Check Your Inbox" heading, a confirmation message showing the submitted email address, and a "Back to Login" link. This clear two-state design ensures the user knows whether the action succeeded or failed.

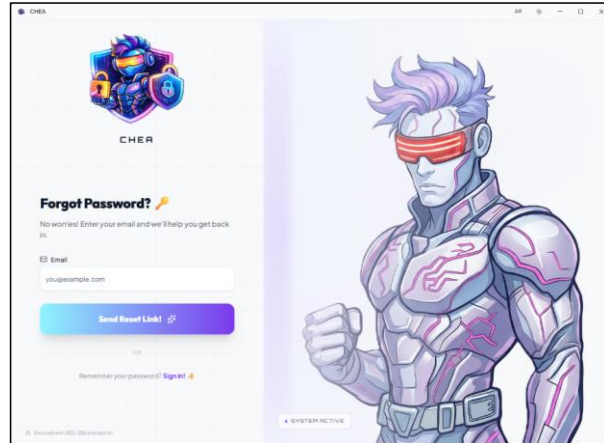


Figure 14: Forget Password Interface

5.3.4 Dashboard Interface

The Dashboard is the central hub; it is the first thing users see after logging in, and it provides an at-a-glance overview of the user's security progress and gamification statistics. The interface displays:

User Profile Section: Shows the user's avatar (generated from initials using a deterministic color algorithm), display name, and current level title with a color-coded badge Bronze for levels 1–3, Silver for levels 4–6, Gold for levels 7–9, and Platinum for level 10.

XP Progress Bar: An animated progress bar showing the user's current XP relative to the next level threshold, with the numerical XP value displayed (e.g., "450 / 600 XP"). The bar fills smoothly when XP is earned.

Streak Counter: Displays the number of consecutive active days with a flame icon and numerical count, encouraging daily engagement.

Security Score: A composite score from 0–100 calculated from the user's XP, streak days, number of vault items, total activities, and completed daily tasks.

Daily Tasks Panel: A list of seven daily objectives with progress indicators and completion checkmarks: scan a link (25 pts), generate a password (15 pts), check password strength (10 pts), save a credential (30 pts), use encryption (15 pts), play the quiz (15 pts), and spot phishing (15 pts). The panel shows completed/total tasks and a percentage.

Recent Activity Feed: Displays the last 15 activities with type-specific icons, descriptions, and relative timestamps (e.g., "Scanned link: example.com — 2 minutes ago").

Quick Access Grid: A grid of cards providing one-click navigation to all security tools, organized by category Scanning Tools (Link Scanner, File Scanner, Photo Secrets), Password Tools (Password Maker, Test Password), Encryption & Vault (Secret Messages, My Vault), and Game Zone (Quiz Arena, Phishing Dojo, AI Agent).

The navigation flow is straightforward: the user logs in, lands on the Dashboard, and can reach any feature from the Quick Access Grid or the persistent sidebar. Every security action the user completes automatically updates the XP bar, streak counter, and activity feed in real time through the useTrackActivity hook.

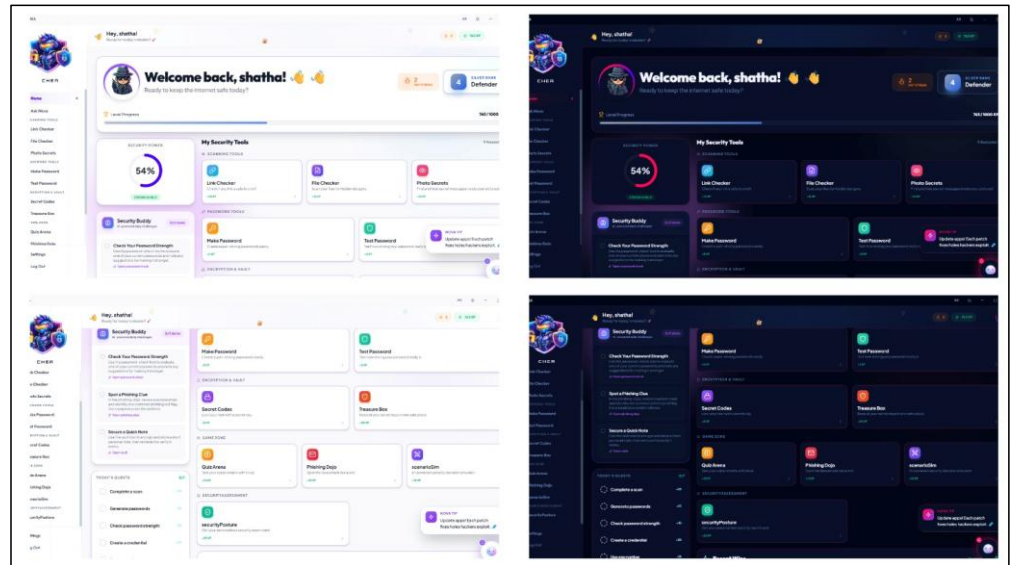


Figure 15: Dashboard Interface

5.3.5 Password Generator Interface

The Password Generator is a focused tool for creating cryptographically secure passwords. It is designed to be immediately intuitive; the user adjusts settings and the password regenerates automatically. The interface includes:

- Configuration Panel: A set of controls including:
- A length slider ranging from 6 to 64 characters, with the current value displayed.
- Toggle switches for four-character types: uppercase letters, lowercase letters, numbers, and symbols.
- An "exclude confusing characters" checkbox that removes easily confused characters (i, l, L, I, |, ` , o, O, 0, 1) from the character pool.
- Preset buttons: Easy (8 characters), Good (12), Strong (16), Super (24) , that set the length and character type toggles to recommended configurations.
- Generated Password Display: A large monospace text field showing the generated password, with a copy-to-clipboard button (copies and shows a "Copied!" tooltip) and a regenerate button (dice icon).

- **Strength Indicator:** An animated color-coded meter below the password with six levels , Too Weak (red), Weak (orange), Fair (yellow), Strong (green), Very Strong (emerald), Super Strong (cyan). The meter also displays the calculated entropy in bits and the size of the character pool used.
- **Educational Tips:** Contextual tips displayed below the generator that explain why password length and character variety matter, update based on the current configuration.

The user adjusts the settings, and the password generates automatically using `window.crypto.getRandomValues()` for cryptographic randomness. Copying the password triggers the activity tracking system: it awards +5 XP, logs a `generate_password` activity, and increments the daily task counter.

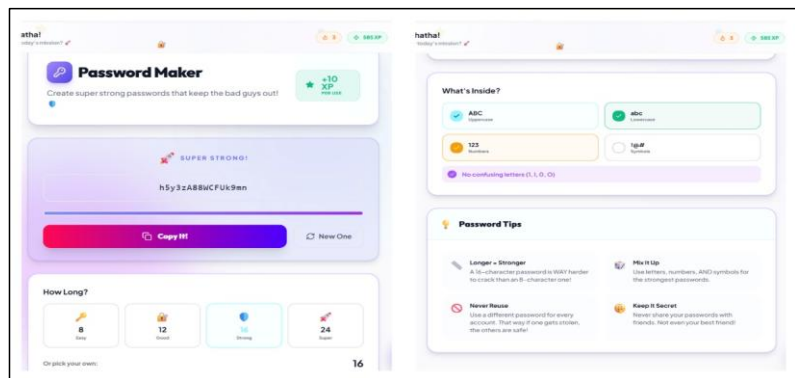


Figure 16: Password Generator Interface

5.3.6 Link Scanner: Interface

The Link Scanner checks URLs against VirusTotal's database of 70+ security engines. It provides both immediate threat assessment and a scan history for reference. The interface includes:

- **URL Input Field:** A text input with a link icon, placeholder text "Enter a URL to check...", and a prominent "Scan" button with a search icon. Below the hero header, a collapsible QuickGuide banner provides four numbered onboarding steps: (1) copy a link to check, (2) paste it in the search box, (3) click "Scan Now!" to check with 70+ security engines, and (4) read the result – green means safe, red means danger. The guide is dismissible and uses translated step text, supporting both English and Arabic. This ensures that students who have never used a URL scanner before can complete their first scan without confusion.
- **Loading State:** During scanning, a pulsing animation plays with the text "Analyzing URL... This may take a moment," indicating that the Rust backend is submitting the URL to VirusTotal and polling for results.

- **Scan History:** A collapsible panel showing the last 10 scanned URLs, each with a status badge green checkmark for clean, red warning for malicious, yellow alert for suspicious along with the detection count and scan time.
- **Results Panel:** Upon scan completion, the results panel animates in with:
- **Status Badge:** A large colour-coded badge displaying "Clean" (green), "Suspicious" (yellow), or "Malicious" (red).
- **Statistics Summary:** Four stat cards showing the count of security engines that flagged the URL as malicious, suspicious, harmless, or undetected.
- **Reputation Score:** A numerical community reputation indicator from VirusTotal's community voting system.
- **Engine Results Table:** A detailed table listing each security engine (e.g., Google Safebrowsing, Kaspersky, McAfee, Sophos) with its individual detection result and category.
- **AI Scan Analysis (FR-11):** Below the results panel, an AI-powered analysis card provides a plain-language interpretation of the scan results. After the scan completes, the frontend sends the detection statistics (malicious count, suspicious count, harmless count, total engines) and the URL to the AI service (see Algorithm 5.5: Scan URL with VirusTotal Polling). The AI responds with a risk assessment written in student-friendly language, explaining what the detection results mean and recommending specific actions (e.g., "This site was flagged by 8 out of 70 security engines. While most engines consider it safe, you should still be careful about entering personal information"). The analysis card features a gradient header with an AI badge, a loading shimmer during generation, and the formatted response below. The analysis is fully bilingual, matching the student's active language setting. This feature transforms raw scan data into actionable educational content, helping young students understand what the numbers actually mean.

The user enters a URL and clicks "Scan." The frontend validates the URL format, calls the Tauri IPC `scan_url` command, and displays the loading state. The Rust backend submits the URL to VirusTotal and polls for results at 4-second intervals (typically completing in 12–18 seconds). Once done, the results panel renders. The user earns +10 XP, the scan is logged as a `scan_link` activity, and the daily scan task is incremented.

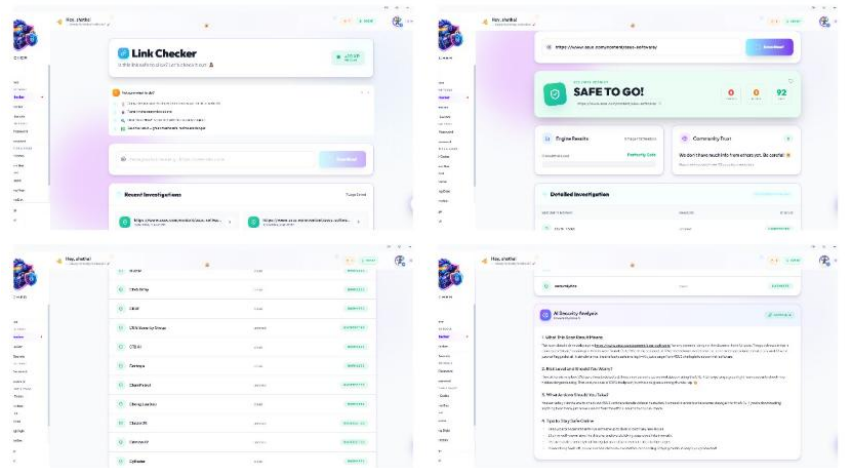


Figure 17: Link Scanner Interface (Light Mode)

5.3.7 Encryption Lab Interface

The Encryption Lab allows students to encrypt and decrypt text messages using three different algorithms, serving both as a practical tool and as an educational introduction to cryptography. The interface includes:

- **Mode Toggle:** A segmented control at the top to switch between "Encrypt" and "Decrypt" modes.
- **Algorithm Selector:** Three radio-style buttons for choosing the encryption algorithm. AES-256-GCM (default, recommended), ChaCha20-Poly1305, and AES-128-CBC each with a brief description of its characteristics.
- **Input Area:** A multi-line text area for entering the plaintext (in encrypt mode) or the encrypted Base64 string (in decrypt mode).
- **Password Input:** A masked password field with visibility toggle, used as the encryption key seed. In encrypt mode, a confirmation field appears to prevent typos.
- **Output Area:** A read-only text area displaying the result, the Base64-encoded encrypted string (in encrypt mode) or the decrypted plaintext (in decrypt mode). A copy-to-clipboard button is provided.
- **Process Explanation:** An expandable section below the output that explains the steps performed (key derivation, encryption, encoding), helping students understand what happens behind the scenes.

The user enters text, sets a password, selects an algorithm, and clicks "Encrypt." The frontend calls the Tauri IPC `encrypt_text` command, which runs the encryption entirely on the local device in Rust. The

encrypted output is a double-encoded Base64 string (JSON payload containing algorithm, salt, IV, and ciphertext, all individually Base64-encoded). The user can share this string and the password with a friend, who can paste it into decrypt mode to recover the original message. This round-trip flow teaches the concept of symmetric encryption in a tangible way. Each encryption or decryption action awards +5 XP and logs a generate_encryption activity.

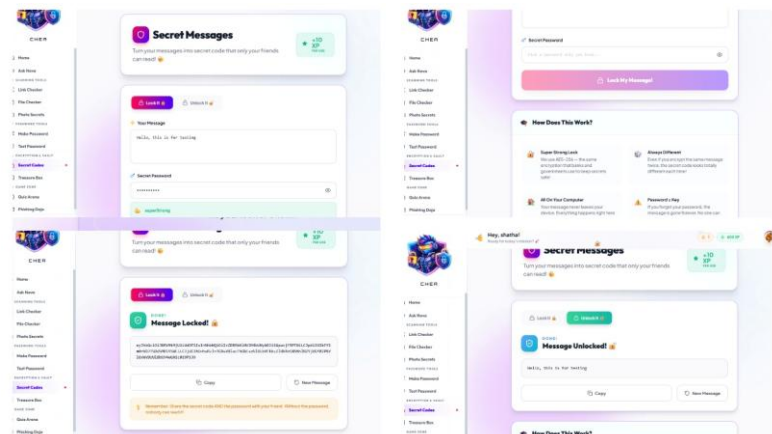


Figure 18: Encryption Lab Interface (Light Mode)

5.3.8 Phishing Dojo Interface

The Phishing Dojo is a gamified training exercise where students practice identifying by phishing emails. It presents realistic email scenarios and provides detailed feedback on each classification. The interface includes:

- **Difficulty Selection:** Three tier buttons - Cadet (levels 1–3, basic phishing indicators), Analyst (levels 4–6, more sophisticated scams), and Operator (levels 7+, advanced social engineering) - each with a distinct colour and description. The system highlights the tier matching the user's current level.
- **Email Display:** A simulated email client view showing the sender's name, sender email address, subject line, and body text of the current email, rendered in a realistic email layout with proper formatting. Emails are drawn from a pool of 23 scenarios across the three tiers, each carefully crafted to include realistic phishing indicators or to be genuinely legitimate.
- **Classification Buttons:** Two large action buttons "Phishing!" (red, with a warning icon) and "Legitimate" (green, with a shield icon) for the user to classify the email.
- **Feedback Panel:** After classification, a detailed feedback section slides in showing:
- **Correct/Incorrect Indicator:** Visual confirmation with a checkmark or X icon.

- **Red Flags List:** For phishing emails, a numbered list of identified red flags with explanations (e.g., "Suspicious Sender: The email claims to be from PayPal but the domain is paypal-security.com note the number 1 replacing the letter l", "Urgency Tactics: The email pressures you to act immediately or your account will be closed").
- **Explanation:** A paragraph explaining why the email is or is not phishing, written in student-friendly language.
- **Progress Tracker:** Shows the current round number (e.g., "Email 3 of 5"), correct count, current streak, and accumulated XP for the session.
- **Summary Screen:** After all, 5 emails, a results summary displays the total correct count, XP earned (broken down into base XP, streak bonus, and completion bonus), and a "Play Again" button.

The user selects a difficulty tier, reads each of the 5 presented emails, classifies them, and receives immediate educational feedback. This design directly addresses the education-protection gap identified by Zhang-Kennedy and Chiasson (2021) by combining practical skill development with conceptual understanding.

AI Challenge Mode (FR-13): In addition to the pre-built pool of 23 static email scenarios, the Phishing Dojo includes an AI Challenge mode that generates dynamic, unique phishing emails on demand. When the student selects "AI Challenge," the frontend calls the AI Phishing Email Generator service (see Algorithm 5.10: Calculate Phishing Dojo XP), which instructs the AI to create a realistic phishing or legitimate email. The AI returns a JSON object conforming to the existing PhishingEmail interface, containing the sender's name, sender email, subject, body, a boolean isPhishing flag, and an array of red flags with explanations.

The generated email is then presented using the same classification interface as the static mode, ensuring a seamless user experience. This mode provides virtually unlimited practice material, as each generated email is unique. The AI is prompted to produce age-appropriate scenarios relevant to students (fake school emails, gaming account scams, social media phishing) and to include realistic phishing indicators (spoofed domains, urgency tactics, suspicious links). Each AI-generated round awards the same XP as a standard round (+15 XP) and is logged as an ai_phishing_round activity.

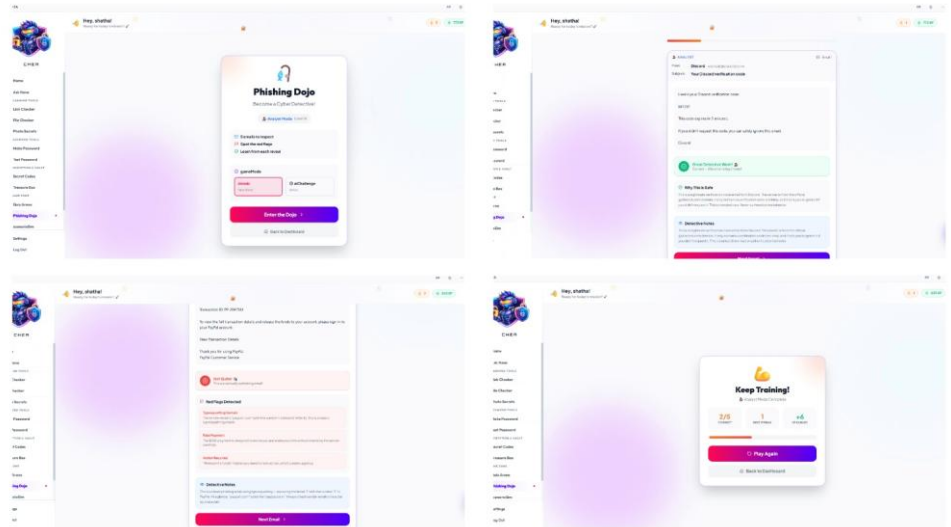


Figure 19: Phishing Dojo Interface (Light Mode)

5.3.9 Credential Vault Interface

The Credential Vault stores login credentials and payment card details securely behind a master password. It is the most security-critical interface in CHEA and requires careful design to be both safe and usable for young students. The interface includes:

- Vault Lock Screen:** Before accessing the vault, the user must enter their master password. The input field has a lock icon and a visibility toggle. If the user has not set up the vault before, a "Set Up Vault" flow is shown instead, with fields for creating and confirming a master password, along with guidance text explaining the importance of choosing a strong, memorable password. Below the hero header (visible only after the vault is unlocked), a collapsible QuickGuide banner provides four numbered onboarding steps: (1) set up a master password in Settings if not done yet, (2) unlock the vault by typing the master password, (3) click "Add New!" to save a login or card, and (4) remember that everything is encrypted, only the master password can unlock it. This guide is particularly important for the Vault page, as the two-step unlock flow (setup then unlock) can be confusing for younger students encountering encrypted storage for the first time.
- Credential List:** After unlocking, a searchable list of stored credentials is displayed. Each entry is shown as a card with the service icon (22 branded icons including Google, GitHub, Netflix, etc.), service name, partially masked username, and a type of badge (login or card).
- Credential Detail View:** Clicking a credential expands it to reveal the full username (with a copy button), the password (masked by default with dots, with a reveal/hide toggle and a copy

button), and the associated domain. For card-type credentials, a visual credit card display with a gradient background shows the card brand logo (Visa, Mastercard, Amex, or Discover), the masked card number, and the cardholder's name.

- **Add Credential Form:** A modal form with fields for service name (with a dropdown of 22 pre-configured services for auto-completing the icon), username or email, password (with an integrated "Generate" button that opens the password generator inline), and credential type (login or card). For card type, additional fields appear for card number, cardholder name, expiry date, and CVV. All sensitive data is encrypted client-side using AES-256-GCM via the Rust backend before being sent to Firestore.
- **Delete Confirmation:** A confirmation dialog with a warning message appears before removing a credential, requiring the user to click "Delete" to confirm.

The user navigates to the Vault, enters their master password to unlock it, and views their saved credentials. They can search by service name, add new credentials (encrypted with their master password), copy usernames and passwords to the clipboard, reveal masked passwords, and delete entries. Each credential saves action awards +20 XP (the highest reward in the system, incentivizing vault usage) and logs a `create_credential` activity.

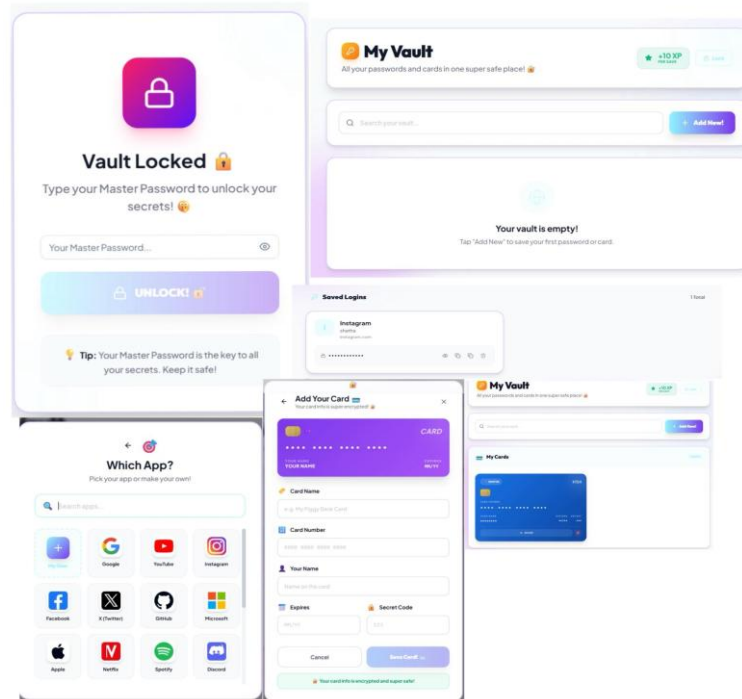


Figure 20: Credential Vault Interface (Light Mode)

The user navigates to the Vault, enters their master password to unlock it, and views their saved credentials. They can add new credentials (encrypted with AES-256-GCM), copy usernames and passwords to the clipboard, reveal masked passwords, and delete entries. Each save action awards +10 XP.

5.3.10 AI Buddy Interface

The AI Buddy (Nova) is a full-page cybersecurity chatbot powered by a large language model accessed through the OpenRouter API. It is one of CHEA's distinguishing features - as shown in the comparative analysis (Table 1), CHEA is the only platform that combines practical security tools, gamified education, and an AI-powered assistant in a single application. The interface is designed to feel like a modern messaging application, making it approachable for students aged 9-15. The interface includes:

- **Session Sidebar:** A collapsible panel on the left showing the user's chat session history. Each session displays an auto-generated title (derived from the first user message), the creation date, and a delete button. Clicking a session loads its full message history. A "New Chat" button at the top creates a fresh session.
- **Chat Message Area:** The central area displays the conversation as a scrollable list of message bubbles. User messages appear on the right with a crimson accent background, while AI

responses appear on the left with a dark surface background. The area auto-scrolls to the latest message.

- **Streaming Response Display:** When the AI is generating a response, text appears character-by-character in real time, simulating a typing effect. A blinking cursor at the end of the streaming text indicates that the response is still being generated. This streaming behavior is achieved through Server-Sent Events (SSE) parsed in the Rust backend and forwarded to the front end via Tauri's `Channel<String>` IPC mechanism.
- **Markdown and Mermaid Rendering:** AI responses are rendered as formatted markdown, supporting headings, bold/italic text, bullet lists, numbered lists, code blocks with syntax highlighting, and inline code. When the AI generates a Mermaid diagram (e.g., a flowchart explaining how phishing works), it is rendered as a visual diagram directly in the chat using the Mermaid.js library, with an option to save the diagram as an SVG file.
- **Input Area:** A text input field at the bottom of the chat with a send button (arrow icon). The input supports multi-line text (expanding textarea) and submits on Enter (or Shift+Enter for a new line). A placeholder text reads "Ask me anything about cybersecurity..."
- **Topic Restriction Indicator:** A small badge above the chat area reads "Cybersecurity & AI topics only," informing the user that the chatbot is restricted to answering security-related questions. If the user asks an off-topic question (e.g., "What's the weather?"), the AI politely redirects them to cybersecurity topics.
- **NovaChat Widget (Companion):** On all other dashboard pages, a floating chat bubble in the bottom-right corner provides quick access to a mini version of the AI assistant. Clicking the bubble opens a compact overlay chat panel, allowing students to ask questions without leaving their current tool. The full AI Buddy page is excluded from showing this widget to avoid nested chat interfaces.

The user navigates to the AI buddy from the sidebar or dashboard. They can create a new session or continue an existing one. As they type and send messages, the Rust backend prepends a cybersecurity-focused system prompt, forwards the request to OpenRouter, and streams the response back in real time. Each chat interaction awards +5 XP and logs a `chat_ai` activity, incentivizing students to explore and ask questions.

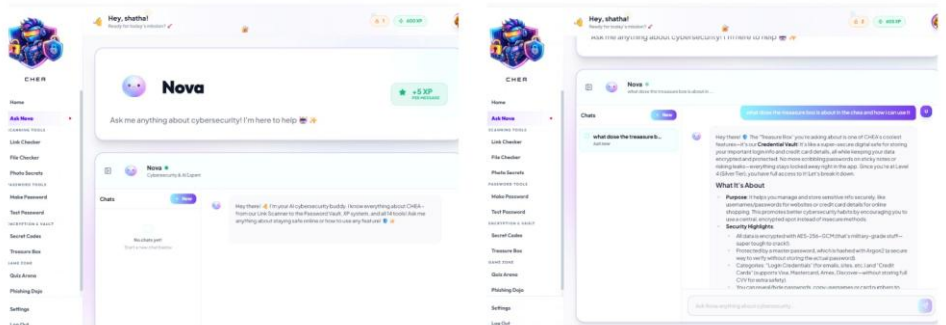


Figure 21: AI Agent Interface (Light Mode)

5.3.11 Navigation and Layout

All authenticated pages share a common DashboardLayout that provides:

- Persistent Sidebar:** A collapsible sidebar (256px wide on desktop, slide-in overlay on mobile) with navigation items organised into five labelled sections - Scanning Tools (Link Checker, File Checker, Photo Secrets), Password Tools (Make Password, Test Password), Encryption & Vault (Secret Codes, Treasure Box), Game Zone (Quiz Arena, Phishing Dojo), and AI Learning Zone (Scenario Simulator, Security Posture). Two standalone items sit above the sections: Home (Dashboard) and Ask Nova (AI Agent). Each item has a Lucide icon and text label. The current page is highlighted with a neon accent colour and a small dot indicator on the right. Icons scale up on hover, turning crimson in dark mode and violet in light mode. A footer area at the bottom provides access to Settings and a Logout button, with a small padlock icon and "AES-256 Encrypted" label for reassurance.
- Header Bar:** An 80px-tall bar displaying, on the left, a mobile menu hamburger button and a greeting message ("Hey, [Name]!" with a waving hand emoji). On the right: a streak pill (flame icon + day count), an XP pill (zap icon + XP value), a theme toggle (sun/moon icon), a language switcher (EN/AR), and the user's avatar inside an animated rotating gradient ring with a level badge overlay. The avatar can be a custom photo uploaded by the user or gradient-initials generated from their display name (managed via avatar.ts, which persists the choice in localStorage and broadcasts changes via a custom DOM event).
- NovaChat Widget:** A floating AI chat bubble (small icon in the bottom-right corner) that provides quick access to the AI assistant from any page except the full AI Agent page. Clicking opens a mini chat panel overlay.
- MusicPlayer Widget:** An optional floating music player in the bottom-right corner (positioned above NovaChat when both are visible). In its collapsed state, it shows a pulsing music note icon, the current song title and artist, and a play/pause toggle. Expanding it reveals

playback controls (previous, play/pause, next), a track progress bar, and a volume slide. The widget is draggable via a grip handle, and its position persisted in localStorage. A playlist of seven ambient songs plays in the background. The entire widget can be toggled on or off from the Settings page.

- **Custom Title Bar:** Since the Tauri window is frameless, a custom title bar with drag-to-move, minimize, and close buttons is rendered at the very top.

The layout uses React Router's <Outlet /> component to render the current page's content in the main area between the sidebar and header. Framer Motion provides page transition animations - pages slide in from the right when navigating forward and fade out when navigating back.

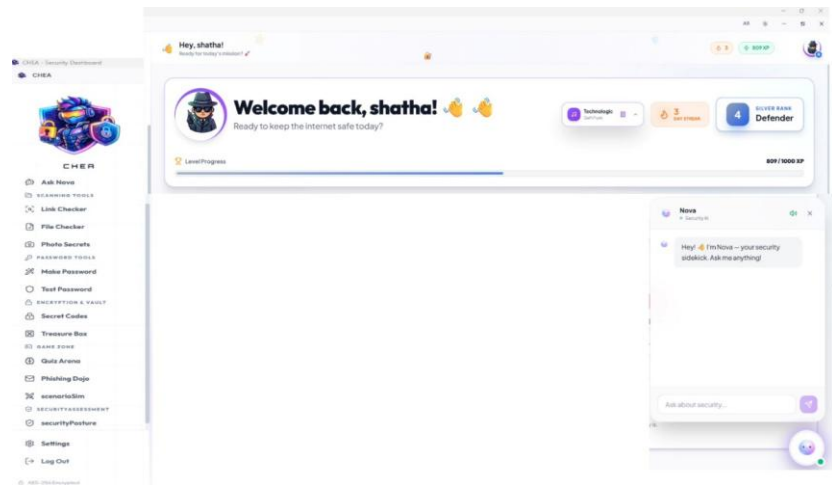


Figure 22: Navigation Layout

5.3.12 Internationalization and Accessibility

CHEA was designed to be accessible to a broad student population, including Arabic-speaking users. The application implements full bilingual support, right-to-left layout adaptation, and theme accessibility features.

- **Bilingual Support (English and Arabic):** The application uses the i18next internationalization framework with 20 translation namespaces covering every page and component. Complete locale files exist for both English (en) and Arabic (ar), translating all UI text including navigation labels, button text, form placeholders, error messages, tooltips, educational tips, quiz questions, and system prompts. A LanguageSwitcher component in the header bar allows users to toggle between English and Arabic at any time without reloading the application. The selected language persisted in localStorage and restored on subsequent sessions. On first launch, the language is auto-detected from the browser's navigator. Language setting.

- **Right-to-Left (RTL) Layout:** When Arabic is selected as the active language, a custom useRTL hook detects the change and applies the dir="rtl" attributes to the document root. This causes the entire layout to mirror: the sidebar moves from the left to the right side, text alignment shifts from left-aligned to right-aligned, the reading direction of lists and cards reverses, and form input fields adapt their text direction. Icon positions within buttons and badges are also adjusted to maintain visual consistency in the mirrored layout. This ensures that Arabic-speaking students experience a natural, native-feeling interface rather than a forced left-to-right layout with translated text.
- **Theme and Visual Accessibility:** The application supports both dark and light themes, with automatic detection of the operating system's colour scheme preference via the prefers-color-scheme CSS media query. A ThemeProvider context manages the theme state, persists in localStorage, and applies them. dark CSS class to the document root for dark mode. All colour combinations are designed to maintain a minimum contrast ratio compliant with WCAG 2.1 AA guidelines, ensuring readability for users with visual impairments. All interactive elements (buttons, links, form inputs, sidebar items) are reachable via keyboard navigation using the Tab key, with visible focus indicators.

These accessibility features directly address NFR-06 (Accessibility) from Chapter 4 and the age-appropriateness gap identified by Sam and Miller (2023), ensuring that CHEA is usable by students regardless of their language preference or visual capabilities.

5.3.13 File Scanner Interface

The File Scanner allows students to check files for malware by computing a local SHA-256 hash and checking it against VirusTotal's database of known threats. It follows the same visual pattern as the Link Scanner but adds file-specific elements. The interface includes:

- **Upload Card (idle state):** A centred card with a large UploadCloud icon (64px) that rotates and scales on hover, a "Ready to Scan" heading, descriptive text about the scanning process, and a prominent gradient "Pick a File" button that opens a native file picker dialog via Tauri's open command. A small disclaimer notes the maximum supported file size. Below the hero header, a collapsible QuickGuide banner provides four numbered onboarding steps: (1) click "Pick a File!" to choose a file from the computer, (2) wait while 70+ security engines scan it, (3) check the result - green means safe, red means danger, and (4) if the file is dangerous, quarantine or delete it. The guide uses emoji icons and translated text to ensure accessibility for students who have never used a malware scanner.
- **Scanning State:** Replaces the upload card with a spinning ring animation, a pulsing Shield icon, an "Analysing File" heading, and a shimmer-animated progress bar that increments to 95% while waiting for results.
- **Results Panel (post-scan):** Upon scan completion, displays:

- **Status Hero Card:** Colour-coded by result (red for malicious, orange for suspicious, green for clean), showing a large rotated shield icon, a status heading (e.g., "Danger Found!"), the filename badge, and a 3-column statistics grid (Dangers, Warnings, Safe counts).
- **Info Grid:** Three cards in a row - File Info (size and status badge), Detection Meter (progress bar showing flagged engines vs. total), and File Fingerprint (SHA-256 hash in monospace font).
- **Danger Warning Banner (malicious only):** A red-tinted card with action buttons: "Quarantine" (orange) and "Delete Forever" (red).
- **Expert Analysis Table:** A detailed table listing each security engine's name with a pass/fail icon, its detection result, and a colour-coded verdict badge.
- **Scan History (idle state):** A 2-column grid showing the last 10 scanned files, each with a gradient icon, filename, date, and status badge.

The user selects a file, the Rust backend computes its SHA-256 hash locally, checks VirusTotal for cached results, and only uploads the file if no cached report exists. Each scan awards +15 XP.
- **AI Scan Analysis :** Identical to the Link Scanner's AI analysis feature, the File Scanner includes an AI-powered analysis card that appears below the scan results. After the file scan completes, the detection statistics, filename, and file hash are sent to the AI service (see Algorithm 5.6: Scan File with Hash-First Check), which returns a student-friendly risk assessment. The analysis explains what the detection ratios mean in practical terms and provides recommendations (e.g., "This file was flagged by 3 engines out of 60. It could be a false positive, but you should avoid opening it until you can verify the source"). The analysis respects the student's active language setting and is generated using the same shared ScanAIAnalysis component used by the Link Scanner.

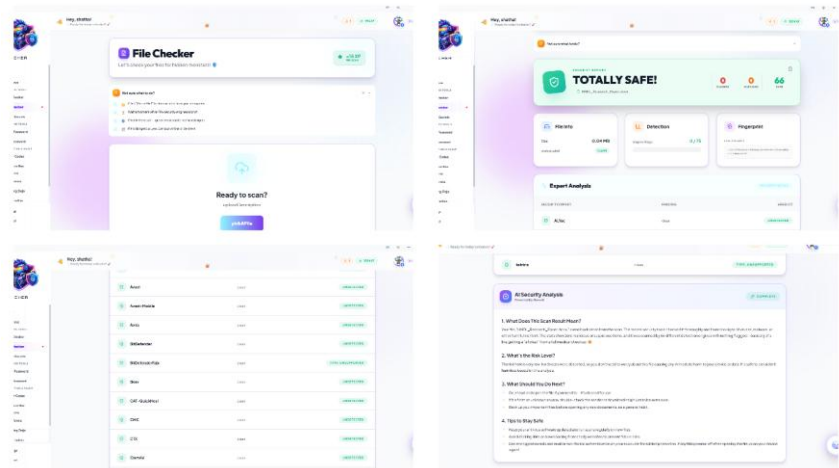


Figure 23: File Scanner Interface (Light Mode)

5.3.14 Password Checker Interface

The Password Checker (Test Password) analyses the strength of an existing password in real time, providing entropy calculation, crack-time estimation, and actionable improvement suggestions. It is the educational companion to the Password Generator - while the Generator creates strong passwords, the Checker explains why a given password is weak or strong. The interface includes:

- **Password Input:** A full-width input field with a lock icon prefix, monospace bold font for the entered password, and an eye toggle to reveal or hide the text.
- **Strength Label:** Appears below the input as the user types, showing the current strength classification (Weak, Fair, Good, Excellent).
- **Status Hero Card (after input):** Colour-coded by strength level (red for weak, amber for fair, emerald for strong, indigo for excellent), displaying a large rotated shield icon, the strength verdict, the password in a monospace badge, and a 3-column statistics row: Score (out of 8), Entropy (in bits), and Crack Time.
- **Time to Crack Card:** Shows the estimated brute-force time in large text (e.g., "Instantly", "3 hours", "2 years", "Centuries"), with an entropy quality message. The icon and colour change by severity - a lightning bolt for instant, a calendar for centuries, an infinity symbol for "forever."
- **Security Checks Card:** Five rows with checkmark or cross icons: length 12 characters, contains uppercase, contains lowercase, contains numbers, contains special characters. Each row is tinted green or red.

- **Recommendations Panel:** Appears when the score is below 7, listing specific improvement tips in an amber-tinted card. For excellent passwords (score 7), a green congratulations card is shown instead.

The analysis is computed entirely on the frontend using entropy-based calculations (see Algorithm 5.4: Analyze Password Strength). Each check awards +3 XP and logs a `check_password` activity.

- **AI Attack Narrative (FR-12):** Below the strength analysis results, an AI-powered "How a Hacker Would Crack This" card provides a narrative description of how an attacker might exploit the password's weaknesses. The frontend sends only the password's characteristics (length, entropy, character types present, estimated crack time, strength score) to the AI service (see Algorithm 5.14: Generate Security Posture Report) never the actual password. The AI responds with a creative, educational narrative written from a hypothetical attacker's perspective, explaining the specific techniques that would be used (e.g., "A dictionary attack would crack this password in minutes because it uses a common word with a simple number substitution"). The narrative card features a gradient header with a hacker-themed icon, a loading shimmer during generation, and the formatted response. The narrative is fully bilingual, generated in the student's active language. This feature transforms abstract password metrics into a compelling story that makes the consequences of weak passwords feel real and immediate to young students.

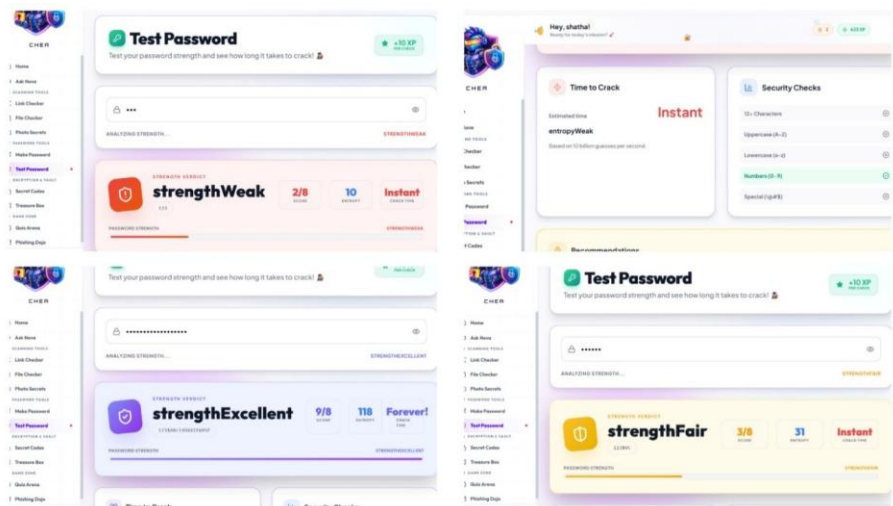


Figure 24: Password Checker Interface

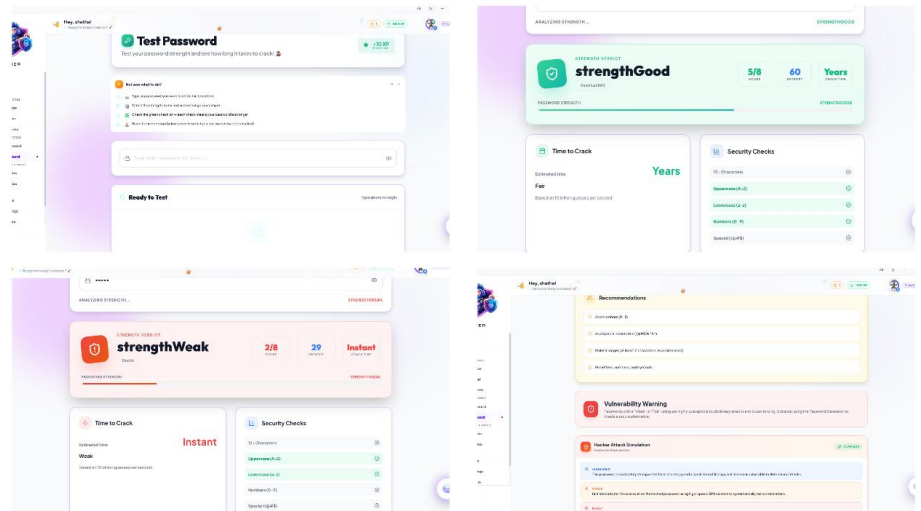


Figure 25: AI attack Narrative

5.3.15 Quiz Arena Interface

The Quiz Arena is a gamified cybersecurity knowledge quiz with multiple-choice questions across three difficulty tiers. It uses the same tier system as the Phishing Dojo - Cadet (levels 1-3), Analyst (levels 4-6), and Operator (levels 7+) - with questions drawn from a pool of 45 questions (15 per tier). The interface has three states:

- **Welcome Screen:** A centered card with an animated brain emoji (scaling and rotating), the "Quiz Arena" title, a tier badge showing the user's current difficulty level and an emoji, a rules summary card (5 questions per round, streak tracking, explanations for each answer), and a gradient "Start Quiz" button.
- **Playing Screen:** Displays one question at a time:
- **Progress Header:** A home button on the left, a streak counter pill (flame icon, visible when streak > 0), and a question counter (e.g., "2/5") on the right, with a thin gradient progress bar below.
- **Question Card:** A tier label at the top, the question text in large bold font, and four answer option buttons (A, B, C, D). After answering, the letter badge transforms into a checkmark (correct, highlighted green) or a cross (incorrect, highlighted red), while the correct answer is always highlighted green.
- **Explanation Card:** Slides in after each answer with a random encouragement phrase for correct answers (e.g., "Cyber Ninja!") or a supportive message for wrong ones, followed by a detailed explanation of the correct answer.

- **Results Screen:** After all, 5 questions, a centred card displays a grade emoji (trophy, star, etc.), a grade label, and a 3-column statistics row (Correct count, Best Streak, XP Earned). An animated score bar fills proportionally with colour-coded gradients. "Play Again" and "Back to Dashboard" buttons are provided.

XP is calculated using the same formula as the Phishing Dojo (see Algorithm 5.10: Calculate Phishing Dojo XP): base XP per correct answer, streak bonus, and completion bonus for perfect or strong performance.

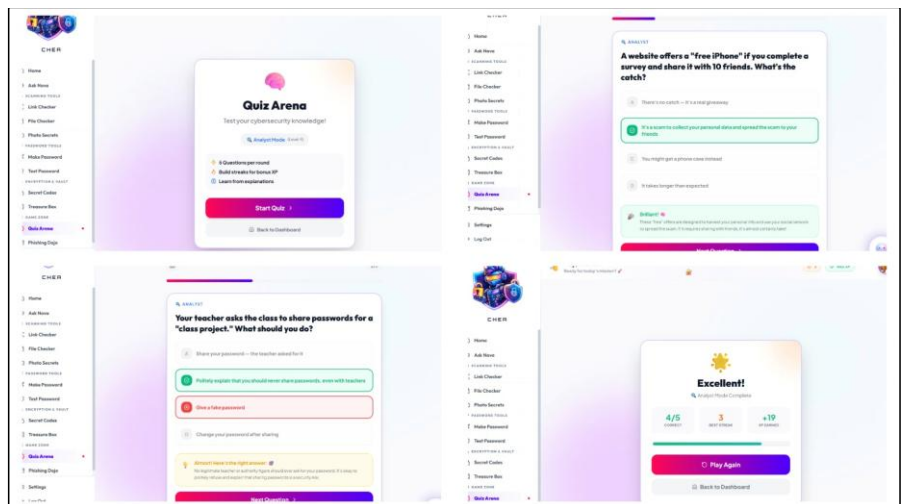


Figure 26: Quiz Arena Interface

5.3.16 Image Privacy (Photo Secrets) Interface

The Image Privacy scanner helps students understand what hidden information is embedded in their photos - particularly GPS coordinates, camera details, and timestamps - and provides a one-click metadata stripping function. The interface includes:

- **Upload Card (idle state):** A centred card with a large Search icon, a "Ready to Scan" heading, a description of what EXIF data can reveal, a "Pick Photo" gradient button, and a note about supported formats (JPEG and PNG).
- **Processing State:** A spinning ring animation with a pulsing image icon, an "Analysing Photo" heading, and a shimmer progress bar.
- **Results Panel (post-scan):** Upon scan completion, displays:
- **Status Hero Card:** Colour-coded by EXIF status (amber if metadata is found, green if clean or after cleaning). Shows a privacy report label, the filename, and a 3-column statistics row (Secrets found, Risks identified, clean status). Two action buttons: "Clean Photo" (gradient CTA, enabled only when EXIF is present) and a reset button.

- **File Information Grid:** Three cards - File Info (type, format, file size), Photo Shape (width, height, megapixels), and System Times (created and modified timestamps).
- **EXIF Detail Cards (shown when EXIF is found):** Four cards in a 2-column grid:
- **GPS Location (marked "HIGH RISK"):** Red-tinted card showing latitude/longitude coordinates and a location name. A "View on Map" button opens the GPS coordinates in Google Maps. A privacy tip box explains the risk. After cleaning, this card fades to 50% opacity.
- **Device/Camera Identity ("MEDIUM RISK"):** Amber-tinted card showing brand, model, and software used. Includes a "Did You Know" tip.
- **EXIF Timestamps:** Shows original and digitized dates with a "Time Travel" tip.
- **Camera Settings:** Shows aperture, ISO, focal length, and exposure time with a "Tech Talk" tip.
- **Advanced Toggle:** A "Show More / Show Less" button that expands additional EXIF fields (flash, white balance, orientation, resolution).
- **Success Message:** A green card with a checkmark icon appears after successful metadata stripping.
- **Scan History (idle state):** A 2-column grid of previous scans with status badges.

The user selects a photo, the Rust backend extracts all EXIF metadata (see Algorithm 5.7: Scan Image EXIF Metadata), and the results are displayed with risk-appropriate coloring. The "Clean Photo" button triggers the stripping algorithm (see Algorithm 5.8: Strip Image EXIF Metadata), which removes all EXIF segments while preserving the image pixels. Each scan awards +10 XP.

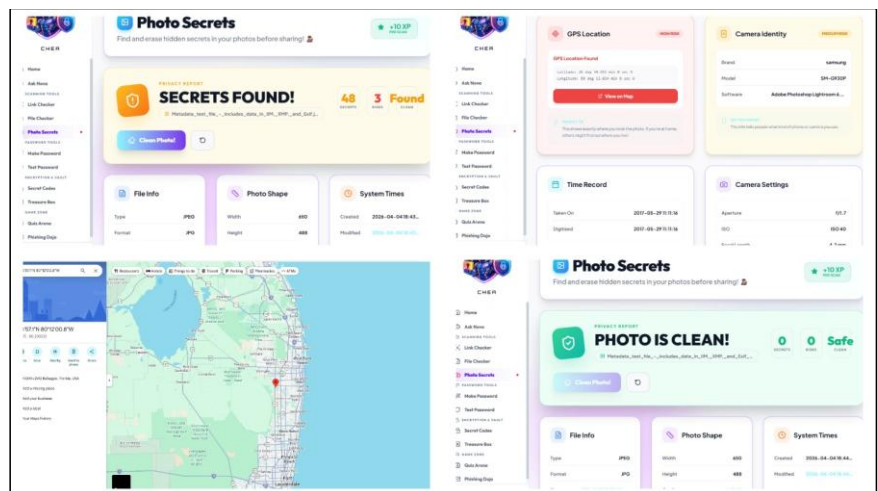


Figure 27: Image Privacy Interface

5.3.17 AI Scenario Simulator Interface

The AI Scenario Simulator is an interactive educational experience where students work through AI-generated cybersecurity scenarios across five rounds. Each round presents a realistic security situation and asks the student to choose the best course of action from four options. The AI generates scenarios dynamically, ensuring that every session is unique and that the content adapts to be age-appropriate and relevant to students. The interface includes:

- **Welcome Screen:** A centered card with an animated scenario icon, the "Scenario Simulator" title, a brief description explaining that the student will face five AI-generated security situations, a rules summary (5 rounds, points for correct answers, AI-generated feedback), and a gradient "Start Simulation" button.
- **Playing Screen:** Displays one scenario at a time:
- **Round Counter:** Shows the current round number (e.g., "Round 2 of 5") with a progress bar.
- **Scenario Card:** A card presenting the AI-generated security scenario in narrative form (e.g., "You receive an email from your school saying your report card is ready. The link points to school-grades-viewer.com instead of your school's real website. What should you do?"). Four answer option buttons are displayed below the scenario text.
- **Feedback Panel:** After selecting an answer, a detailed feedback section slides in with a correct/incorrect indicator, an explanation of why the chosen answer is right or wrong, and additional educational context about the security concept being tested. The feedback is generated by the AI alongside the scenario (see Algorithm 5.13: Generate Security Scenarios), ensuring it is specific to each unique scenario rather than generic.
- **Loading State:** During AI generation of the next round, a shimmer animation is displayed with a "Generating next scenario..." message.
- **Results Screen:** After all 5 rounds, a centered card displays: a grade emoji, a performance label (e.g., "Cyber Defender"), a 3-column statistics row (Correct, Accuracy Percentage, XP Earned), and an animated score bar. "Play Again" and "Back to Dashboard" buttons are provided.

The AI generates each scenario in the student's active language (English or Arabic), ensuring full bilingual support. Each completed simulation awards +20 XP and is logged as a scenario_simulator activity.

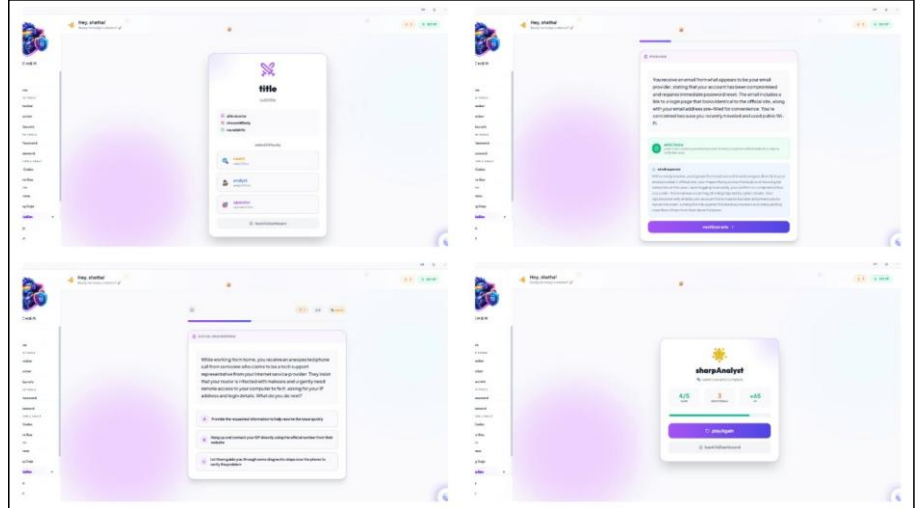


Figure 28: Scenario Simulator Interface

5.3.18 AI Security Posture Assessment Interface

The AI Security Posture Assessment is a comprehensive self-evaluation tool that asks students 15 questions about their cybersecurity habits and then uses AI to generate a personalised security report card. Unlike the quiz-style assessments, this feature evaluates the student's real-world security behavior rather than theoretical knowledge. The interface includes:

- **Welcome Screen:** A centered card with a shield icon, the "Security Posture" title, a description explaining that the assessment evaluates real security habits, a note about the 15-question format, and a gradient "Start Assessment" button.
- **Question Screen:** Displays one question at a time from the 15-question bank:
- **Progress Header:** A progress bar showing completion percentage (e.g., "Question 7 of 15") with a back button for navigating to previous questions.
- **Question Card:** A card displaying the question text, with four answer options presented as selectable buttons. The student can select one option per question and navigate freely between questions before submitting.
- **Question Bank:** The 15 questions are drawn from the securityQuestions data file, which contains complete question sets in both English and Arabic (see Section 5.4.3 AI Service Layer). The useSecurityQuestions hook selects the appropriate language version based on the student's active language setting.
- **Results Screen:** After submitting all answers, the frontend sends the question-answer pairs to the AI service (see Algorithm 5.14: Generate Security Posture Report), which generates a comprehensive security report card. The results display:

- **Security Grade:** A large letter grade (A+ through F) with a colour-coded badge and an overall security score percentage.
- **AI Report Card:** A detailed, personalised analysis written by the AI, highlighting the student's strengths and weaknesses with specific, actionable recommendations (e.g., "You're doing great with password security, but you should enable two-factor authentication on your accounts"). The report is generated in the student's active language.
- **Category Breakdown:** A visual grid showing performance across security categories (password security, phishing awareness, data protection, social media safety, device security).
- **Tip Cards:** A set of personalised security tips based on the weakest areas identified in the assessment.

Each completed assessment awards +25 XP and is logged as a security_posture activity. The questions are designed to be age-appropriate for students aged 9-15, covering topics like password habits, social media privacy settings, public Wi-Fi safety, and personal information sharing.

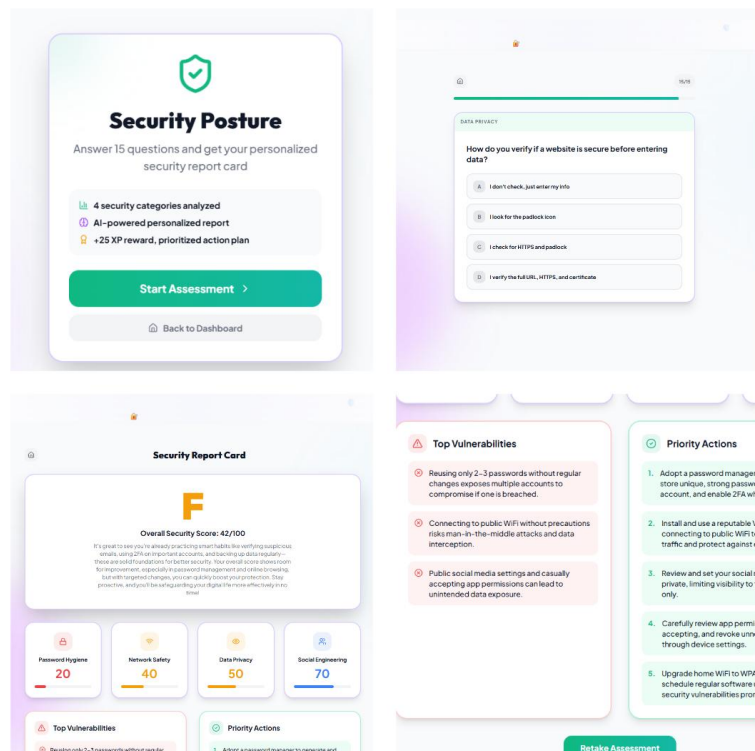


Figure 29: Security Posture Assessment Interface

5.3.19 Settings Interface

The Settings page provides account management, security configuration, and user preferences in a tabbed layout. It is the only page where users can set up their vault master password, choose an avatar, and configure the music player. The interface includes:

- **Tab Navigation:** A rounded card with three-tab buttons - Profile (with an emoji icon), Security, and Preferences. The active tab has a crimson-to-violet gradient background; inactive tabs are muted.
- **Profile Tab:**
 1. **Profile Card:** A card with a gradient top banner, showing the user's avatar (96px, rounded), a camera edit button that appears on hover, the display name, email, and a green "Account Active" status label.
 2. **Avatar Picker:** An expandable section with two options - upload a custom photo (dashed border upload area with an image icon) or choose from a grid of 17 preset avatar thumbnails that scale and lift on hover. Avatar selection persisted in localStorage and broadcast to other components via a custom DOM event.
 3. **From Fields:** A display name input (editable) and a read-only email input (greyed out with a "can't change" note).
- **Security Tab:**
 1. **Vault Password Card:** A card with a green top bar and lock icon header. Depending on whether the vault has been set up, it shows either a "Set Up Vault Password" flow (new password + confirm) or a "Change Vault Password" flow (current password + new password + confirm). Success and error messages appear in a colored banner.
 2. **Auto-Lock Timer Card:** A card with a blue top bar and clock icon header. An animated toggle switch enables or disables the auto-lock feature. When enabled, a 24 grid of time options appears (5 minutes, 15 minutes, 30 minutes, 1 hour) with emoji icons, allowing the user to choose how long the vault stays unlocked before requiring re-authentication.
- **Preferences Tab:**
 1. **Music Player Card:** A card with a purple top bar and music icon header, containing a toggle switch to show or hide the floating MusicPlayer widget across the application.
 2. **Notifications Card:** A card with an amber top bar and bell icon header, with two toggle rows for email alerts and push notifications.
 3. **Save Button:** A fixed gradient button in the bottom-right corner that changes to a green "Saved!" confirmation briefly by clicking.

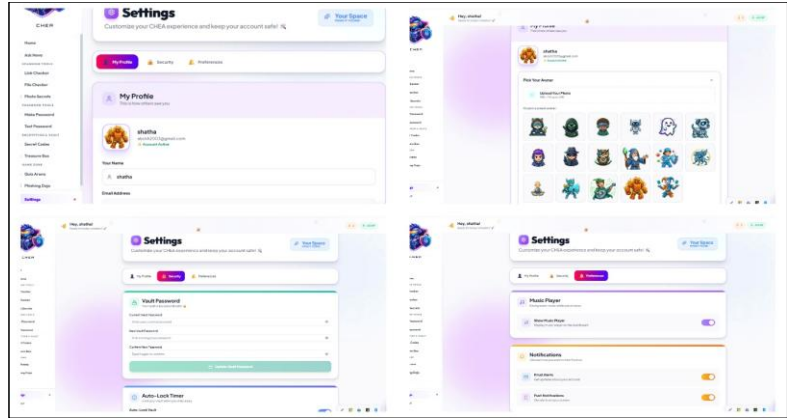


Figure 30: Settings Interface

5.4 Application Architecture

This section describes the architectural patterns that underpin CHEA's frontend: the routing system with authentication guards, the global state management layer using Zustand, and the reusable UI component library.

5.4.1 Routing and Navigation Architecture

CHEA uses React Router DOM v6 with a nested route configuration defined in App.tsx. The routing architecture enforces two access control patterns:

- **PublicRoute Guard:** Wraps authentication pages (/login, /register, /forgot-password). If the user is already authenticated (checked via useAuthStore), the guard redirects to /dashboard, preventing logged-in users from accessing the login or registration pages.
- **ProtectedRoute Guard:** Wraps all dashboard routes. If no authenticated user exists in the store, the guard redirects to /login, preventing unauthenticated access to security tools and personal data.

The route tree is structured as follows:

Table 10: Route Tree Structure

Route	Guard	Page Component
/	None	Redirects to /login
/login	PublicRoute	Login
/register	PublicRoute	Register
/forgot-password	PublicRoute	ForgotPassword

/terms	None (public)	TermsOfService
/privacy	None (public)	PrivacyPolicy
/dashboard	ProtectedRoute <' DashboardLayout	Nested child routes
/dashboard (index)	Protected	Dashboard
/dashboard/link-scanner	Protected	LinkScanner
/dashboard/file-scanner	Protected	FileScanner
/dashboard/metadata	Protected	ImagePrivacy
/dashboard/password-gen	Protected	PasswordGenerator
/dashboard/password-check	Protected	PasswordChecker
/dashboard/encryption	Protected	Encryption
/dashboard/vault	Protected	CredentialVault
/dashboard/ai-agent	Protected	AIAgent
/dashboard/quiz-arena	Protected	QuizArena
/dashboard/phishing-dojo	Protected	PhishingDojo
/dashboard/scenario-simulator	Protected	ScenarioSimulator
/dashboard/security-posture	Protected	SecurityPosture
/dashboard/settings	Protected	Settings
/dashboard/*	Protected	Dashbored

5.4.2 State Management Architecture

Global state is managed through four Zustand stores, each responsible for a distinct domain. Zustand was chosen over Redux or Context API for its minimal boilerplate, automatic re-render optimisation, and ability to access state outside React components (useful for route guards).

- **useAuthStore** - Manages Firebase authentication state and vault session:
 - user: User | null - The current Firebase user object, updated by the onAuthStateChanged listener.
 - loading: Boolean - True during auth initialisation (app shows a full-screen spinner).

masterPassword: string | null - Held in memory only during the active vault session; never persisted. Cleared on logout or vault lock.

- **useActivityStore** - Manages the user's recent activity log:
activities: Activity[] - Array of the 15 most recent activity entries.
- **fetchActivities(userId)** - Retrieves from Firestore's activities subcollection, sorted by createdAt descending.
- **logActivity(userId, type, metadata)** - Appends a new activity entry.
- **useUserProgressStore** - Manages gamification state:
progress: UserProgress | null - The user's XP, level, streak, and score data.
- **levelInfo** - Computed from XP using the 10-tier threshold table.
- **fetchProgress(userId)** - Reads from progress/data.
- **earnXp(userId, points)** - Adds XP, recalculates level, updates streak (see Algorithm 5.12).
- **useDailyTasksStore** - Manages daily task completion:
tasks: DailyTasksData | null - Today's task list and completion status.
- **Summary** - Computed: completed count, total count, percentage.
- **fetchTasks(userId)** - Reads from dailyTasks/ {today}.
- **completeTask(userId, taskType)** - Increments the matching task's counter.

These four stores are integrated through the useTrackActivity custom hook, which centralises the gamification pipeline: when a user completes any security action, a single call to useTrackActivity(userId, type, metadata) logs the activity, awards XP, and increments the relevant daily task - all in sequence.

5.4.3 AI Service Layer

All six AI-powered features (FR-11 through FR-16) share a centralised AI service module (src/services/aiService.ts) that provides two functions:

- **callNova(prompt, options)** Sends a standard (non-streaming) request to the OpenRouter API and returns the complete response. Used by ScanAIAnalysis (FR-11), AttackNarrative (FR-12), AI Phishing Generator (FR-13), Scenario Simulator (FR-14), and Security Posture Assessment (FR-15).
- **callNovaStreaming(prompt, options)** Sends a streaming request to the OpenRouter API and returns an async iterator of response chunks. Used for features requiring progressive rendering.

Both functions route requests to <https://openrouter.ai/api/v1/chat/completions> with the x-ai/grok-4-fast model. The API key is stored in the VITE_OPENROUTER_API_KEY environment variable and is

accessed through the standard Vite environment variable system. All AI-generated content is produced in the user's active language (English or Arabic) by including a language instruction in each prompt. A dedicated AI Phishing Generator service (src/services/aiPhishingGenerator.ts) extends the base service with phishing-specific prompting and JSON schema validation.

5.4.4 Reusable UI Component Library

CHEA includes a library of 12 reusable UI components in src/components/ui/, used across all pages for visual consistency:

Table 11: Library of reusable UI components

Component	Purpose
Button	Primary, secondary, ghost, and danger variants with gradient backgrounds, hover/tap animations, loading spinner, and forwardRef support
Input	Text input with icon prefix support, label, error state, and consistent styling
Card	Container with rounded-3xl borders, optional gradient header strip, and consistent padding
Checkbox	Styled checkbox with label, used for terms agreement and feature toggles
PasswordInput	Input with built-in visibility toggle (eye/eye-off icon) for masked password fields
PasswordStrength	Animated colour-coded strength meter (6 levels) with entropy display, used on the Register page and Password Generator
TitleBar	Custom frameless window title bar with drag-to-move area, minimise and close buttons
ThemeToggle	Sun/moon icon button that switches between dark and light mode via ThemeProvider
LanguageSwitcher	EN/AR toggle button that changes the i18next language and triggers RTL layout adaptation
NovaChat	Floating AI chat bubble widget with expandable mini chat panel, rendered on all pages except AI Agent
MusicPlayer	Floating draggable music player widget with play/pause, skip, volume, and playlist controls
QuickGuid	Collapsible step-by-step onboarding banner with numbered steps, emoji icons, smooth expand/collapse animation, and dismissible state. Used on LinkScanner, FileScanner, PasswordChecker, and CredentialVault pages to guide first-time users through each tool's workflow. Steps are fully translated via i18next.

All components follow the same design patterns: functional components with TypeScript interfaces, Tailwind CSS utility classes with `cn()` for conditional styling, Framer Motion for animations, and consistent use of the cyberpunk colour system.

In addition, three AI-specific reusable components are provided in `src/components/ai/`:

Table 12: Reusable AI Component

Component	Purpose
ScanAIAnalysis	Reusable AI-powered analysis card for scan results, used by both Link Scanner and File Scanner. Accepts scan statistics and URL/filename, calls the AI service, and renders the analysis in the active language.
AttackNarrative	AI-powered hacker narrative card for password analysis, used by the Password Checker. Accepts password traits (never the actual password), calls the AI service, and renders an educational attack narrative.
SecurityBuddy	Dashboard widget displaying an AI-generated daily cybersecurity challenge with title, description, recommended tool, and navigation button. Caches results in localStorage keyed by user, date, and language.

These AI components share a centralised AI service module (`src/services/aiService.ts`) that provides `callNova()` for standard requests and `callNovaStreaming()` for streaming responses, both routing through the OpenRouter API with the `x-ai/grok-4-fast` model.

5.5 Algorithm Design

This section presents the pseudocode for all core algorithms implemented in CHEA. These algorithms are divided into two categories: security algorithms (encryption, decryption, password generation, password strength analysis, URL scanning, file scanning, and image metadata processing) implemented in the Rust backend, the AI chatbot with streaming responses, and gamification algorithms (XP calculation and level progression) implemented in the TypeScript frontend services.

5.5.1 Encryption Algorithm

The encryption algorithm supports three ciphers - AES-256-GCM, ChaCha20-Poly1305, and AES-128-CBC - all using Argon2id for key derivation. It is implemented in the Rust `crypto.rs` module and is used for both the Encryption Lab feature and the Credential Vault. Every encryption operation generates a unique random salt and nonce, ensuring that encrypting the same plaintext with the same password always produces a different ciphertext.

Algorithm 5.1: Encrypt Text

FUNCTION encrypt_text(plaintext, password, algorithm):

```

// Step 1: Generate random salt (128 bits) using OS cryptographic RNG
salt <-generate_random_bytes(16)
// Step 2: Derive encryption key using Argon2id
IF algorithm = "AES-256-GCM" OR algorithm = "ChaCha20-Poly1305" THEN
    key_length <- 32 // 256-bit key
ELSE IF algorithm = "AES-128-CBC" THEN
    key_length <- 16 // 128-bit key
END IF
key <-argon2id_derive (password, salt, key_length)
// Step 3: Generate random nonce or IV
IF algorithm = "AES-256-GCM" OR algorithm = "ChaCha20-Poly1305" THEN
    nonce <-generate_random_bytes(12) // 96-bit nonce
ELSE IF algorithm = "AES-128-CBC" THEN
    iv <-generate_random_bytes(16) // 128-bit IV
END IF
// Step 4: Encrypt plaintext with selected algorithm
IF algorithm = "AES-256-GCM" THEN
    ciphertext <-AES_256_GCM_encrypt (key, nonce, plaintext)
    // Output includes 16-byte authentication tag appended to ciphertext
ELSE IF algorithm = "ChaCha20-Poly1305" THEN
    ciphertext <-ChaCha20_Poly1305_encrypt (key, nonce, plaintext)
    // Output includes 16-byte Poly1305 authentication tag
ELSE IF algorithm = "AES-128-CBC" THEN
    padded_plaintext <-PKCS7_pad (plaintext, block_size=16)
    ciphertext <-AES_128_CBC_encrypt (key, iv, padded_plaintext)
END IF
// Step 5: Build serialisable payload
payload <-{
    alg: algorithm,
    salt: base64_encode(salt),
    iv: base64_encode(nonce OR iv),
    ct: base64_encode(ciphertext)
}
json_string <-serialise_to_json(payload)
result <-base64_encode(json_string)
RETURN result
END FUNCTION

```

5.5.2 Decryption Algorithm

The decryption algorithm reverses the encryption process. It auto-detects the algorithm from the payload and derives the key using the same Argon2id parameters. For AEAD modes (AES-256-GCM and ChaCha20-Poly1305), the authentication tag is verified during decryption - any tampering with the ciphertext is detected, and the operation fails.

Algorithm 5.2: Decrypt Text

```

FUNCTION decrypt_text(encoded_string, password):
    // Step 1: Decode outer Base64 layer

```

```

json_bytes <-base64_decode(encoded_string)
json_string <-utf8_decode(json_bytes)
payload <-deserialise_from_json(json_string)
// Step 2: Decode individual payload fields
salt <-base64_decode(payload.salt)
iv <-base64_decode(payload.iv)
ciphertext <-base64_decode(payload.ct)
// Step 3: Derive key using the same Argon2id parameters
IF payload.alg = "AES-256-GCM" OR payload.alg = "ChaCha20-Poly1305" THEN
    key <-argon2id_derive(password, salt, 32)
ELSE IF payload.alg = "AES-128-CBC" THEN
    key <-argon2id_derive(password, salt, 16)
END IF
// Step 4: Decrypt and verify integrity
IF payload.alg = "AES-256-GCM" THEN
    plaintext <-AES_256_GCM_decrypt(key, iv, ciphertext)
    // Authenticated decryption: automatically verifies the 16-byte tag
    // Fails if ciphertext has been modified
ELSE IF payload.alg = "ChaCha20-Poly1305" THEN
    plaintext <-ChaCha20_Poly1305_decrypt(key, iv, ciphertext)
    // Verifies the Poly1305 MAC before returning plaintext
ELSE IF payload.alg = "AES-128-CBC" THEN
    decrypted <-AES_128_CBC_decrypt(key, iv, ciphertext)
    plaintext <-PKCS7_unpad(decrypted, block_size=16)
    // Validates padding format - fails if padding is invalid
END IF
// Step 5: Handle errors
IF decryption_failed THEN
    RETURN error("Decryption failed - wrong password or corrupted data")
END IF
RETURN utf8_decode(plaintext)
END FUNCTION

```

The security properties of this design are worth noting. Because each encryption uses a fresh random salt, the same password produces different derived keys each time, preventing precomputation attacks. Because each encryption uses a fresh random nonce, the same plaintext encrypted with the same key produces different ciphertext, preventing pattern analysis. The AEAD modes (AES-256-GCM and ChaCha20-Poly1305) provide both confidentiality and integrity - any modification to the ciphertext, IV, or salt is detected during decryption.

5.5.3 Password Generation Algorithm

The password generator uses the Web Crypto API's `crypto.getRandomValues()` to produce cryptographically secure random passwords. This ensures uniform distribution across the character pool, making every character equally likely and eliminating the biases present in `Math.random()`. The algorithm is implemented in the TypeScript `PasswordGenerator.tsx` page component.

Algorithm 5.3: Generate Secure Password:

```
FUNCTION generate_password(length, use_uppercase, use_lowercase, use_numbers, use_symbols,
exclude_confusing):
  // Step 1: Define character sets
  UPPERCASE <-"ABCDEFGHIJKLMNOPQRSTUVWXYZ"
  LOWERCASE <-"abcdefghijklmnopqrstuvwxyz"
  NUMBERS <- "0123456789"
  SYMBOLS <- "!@#$%^&*()_+~`|}{[]:;>,<./-="
  CONFUSING <-regex pattern /[iIlLl]'oO01]/g
  // Step 2: Build character pool from selected types
  charset <-""
  IF use_uppercase THEN charset <-charset + UPPERCASE
  IF use_lowercase THEN charset <-charset + LOWERCASE
  IF use_numbers THEN charset <-charset + NUMBERS
  IF use_symbols THEN charset <-charset + SYMBOLS
  // Step 3: Optionally remove confusing characters
  IF exclude_confusing THEN
    charset <-remove_characters_matching(charset, CONFUSING)
  END IF
  // Step 4: Generate password using cryptographic RNG
  password <-""
  random_values <-crypto_get_random_values(Uint32Array of size length)
  FOR i FROM 0 TO length - 1 DO
    index <-random_values[a] MODULO length of(charset)
    password <-password + charset[index]
  END FOR
  RETURN password
END FUNCTION
```

The algorithm uses a Uint32Array of the requested password length, fills it with cryptographically random 32-bit integers, and maps each integer to a character in the pool using modular arithmetic. The entropy of the resulting password is $\text{length} \cdot \log_2(\text{pool_size})$ bits. For example, a 16-character password using all four-character types (pool size 94) has approximately 104.7 bits of entropy.

5.5.4 Password Strength Analysis Algorithm

The password strength analyzer evaluates an existing password by calculating its entropy, estimating the time required to crack it through brute force, identifying common weaknesses, and providing actionable improvement suggestions. It is implemented in the TypeScript PasswordChecker.tsx page component.

Algorithm 5.4: Analyze Password Strength

```
FUNCTION analyse_password_strength(password):
  // Step 1: Detect character classes present
  has_uppercase <-contains_match(password, /[A-Z]/)
  has_lowercase <-contains_match(password, /[a-z]/)
  has_numbers <-contains_match(password, /[0-9]/)
```

```

has_special <-contains_match(password, /[^A-Za-z0-9]/)
// Step 2: Calculate pool size based on character variety
pool_size <-0
IF has_uppercase THEN pool_size <- pool_size + 26
IF has_lowercase THEN pool_size <- pool_size + 26
IF has_numbers THEN pool_size <- pool_size + 10
IF has_special THEN pool_size <- pool_size + 33
// Step 3: Calculate entropy
IF pool_size > 0 THEN
  entropy <-password.length * log_base_2(pool_size)
ELSE
  entropy <-0
END IF
// Step 4: Estimate time to crack (brute force at 10 billion guesses/second)
guesses_per_second <- 10,000,000,000 // Modern GPU cluster
total_combinations <- 2 ^ entropy
seconds_to_crack <-total_combinations / guesses_per_second
time_description <-human_readable_time(seconds_to_crack)
// Examples: "Instantly", "3 hours", "2 years", "Centuries"
// Step 5: Compute strength score (0-8 scale)
score <- 0
IF password.length >= 8 THEN score <-score + 1
IF password.length >=12 THEN score <-score + 1
IF password.length >= 16 THEN score <-score + 1
IF password.length 24 THEN score <-score + 1
IF has_uppercase THEN score <-score + 1
IF has_lowercase THEN score <-score + 1
IF has_numbers THEN score <-score + 1
IF has_special THEN score <-score + 1
// Step 6: Determine strength label and colour
IF score <=1 THEN label <- "Too Weak", colour <-red
ELSE IF score <= 2 THEN label <- "Weak", colour <-orange
ELSE IF score <= 3 THEN label <- "Fair", colour <-yellow
ELSE IF score <= 5 THEN label <- "Strong", colour <-green
ELSE IF score <= 6 THEN label <- "Very Strong", colour <-emerald
ELSE label <- "Super Strong", colour <-cyan
// Step 7: Generate improvement suggestions
suggestions <-[]
IF password.length < 12 THEN
  suggestions.append("Use at least 12 characters for better security")
END IF
IF NOT has_uppercase THEN suggestions.append("Add uppercase letters")
IF NOT has_lowercase THEN suggestions.append("Add lowercase letters")
IF NOT has_numbers THEN suggestions.append("Add numbers")
IF NOT has_special THEN suggestions.append("Add special characters (!@#$...)")

RETURN { entropy, score, label, colour, time_description, suggestions,
        pool_size, has_uppercase, has_lowercase, has_numbers, has_special }

```

END FUNCTION

The entropy-based approach provides an objective measure of password strength that accounts for both length and character variety. The brute-force time estimation uses a conservative assumption of 10 billion guesses per second, which reflects the capability of modern GPU-accelerated cracking tools. This gives students a tangible understanding of why a 6-character password (entropy $38 \approx$ bits) can be cracked "Instantly" while a 16-character password with all character types (entropy ≈ 105 bits) would take "Centuries."

5.5.5 URL Scanning with VirusTotal Polling Algorithm

VirusTotal does not return scan results instantly. Instead, it accepts the URL for analysis and returns an analysis ID. The client must then poll for completion. We implemented this polling mechanism in the Rust `virustotal.rs` module, with a 4-second polling interval and a maximum of 30 attempts (approximately 2 minutes total timeout).

Algorithm 5.5: Scan URL with VirusTotal Polling

```
FUNCTION scan_url(target_url):
  // Step 1: Submit URL to VirusTotal for analysis
  client <- create_http_client(timeout = 120 seconds)
  encoded_body <- url_encode("url=" + target_url)
  response <- client.POST(
    url: "https://www.virustotal.com/api/v3/urls",
    headers: {
      "x-apikey": VIRUSTOTAL_API_KEY,
      "content-type": "application/x-www-form-urlencoded"
    },
    body: encoded_body
  )
  IF response.status != 200 THEN
    RETURN error("API error: " + response.status)
  END IF
  // Step 2: Extract analysis ID from response
  json <- parse_json(response.body)
  analysis_id <- json.data.id
  // Step 3: Poll for analysis completion
  attempts <- 0
  max_attempts <- 30
  WHILE attempts < max_attempts DO
    sleep(4 seconds)
    attempts <- attempts + 1
    poll_response <- client.GET(
      url: "https://www.virustotal.com/api/v3/analyses/" + analysis_id,
      headers: { "x-apikey": VIRUSTOTAL_API_KEY }
    )
    IF poll_response.status != 200 THEN
      CONTINUE // Retry on transient network errors
```

```

END IF
poll_json <-parse_json(poll_response.body)
status <-poll_json.data.attributes.status
IF status = "completed" THEN
  // Step 4: Parse and structure the results
  stats <-poll_json.data.attributes.stats
  // stats contain malicious, suspicious, harmless, undetected counts
  detections <-[]
  FOR EACH (engine_name, engine_result) IN poll_json.data.attributes.results DO
    IF engine_result.category â‰‰ "undetected" THEN
      detections.append({
        engine: engine_name,
        result: engine_result.result,
        category: engine_result.category
      })
    END IF
  END FOR
  scan_result <-{
    target: target_url,
    status: determine_overall_status(stats.malicious, stats.suspicious),
    reputation: poll_json.data.attributes.reputation,
    detections: detections,
    stats: stats
  }
  RETURN scan_result
END IF
END WHILE
RETURN error("Scan timeout. VirusTotal did not complete the analysis in time.")
END FUNCTION
FUNCTION determine_overall_status(malicious_count, suspicious_count):
  IF malicious_count > 0 THEN RETURN "malicious"
  IF suspicious_count > 0 THEN RETURN "suspicious"
  RETURN "clean"
END FUNCTION

```

The 4-second polling interval was chosen based on VirusTotal's recommendation to avoid aggressive polling, and in practice most scans complete within 3-4 polls (12-16 seconds). The API key is stored exclusively in the Rust backend binary and is never exposed to the frontend JavaScript context.

5.5.6 File Scanning with Hash-First Check Algorithm

File scanning extends the URL scanning approach with an optimisation: before uploading the file, we compute its SHA-256 hash locally and check if VirusTotal already has results for that hash. If a cached report exists, we skip the upload entirely, saving bandwidth and time. This is implemented in the Rust `virustotal.rs` module.

Algorithm 5.6: Scan File with Hash-First Check

```

FUNCTION scan_file(file_path):

```

```

// Step 1: Compute local SHA-256 hash of the file
hasher <-create_sha256_hasher()
file <-open_file(file_path)
file_size <- 0
buffer <-allocate_buffer(8192) // 8 KB chunks
WHILE (bytes_read <-file.read(buffer)) > 0 DO
    hasher.update(buffer[0..bytes_read])
    file_size <-file_size + bytes_read
END WHILE
file_hash <-hasher.finalise_as_hex_string()
file.close()
// Step 2: Check if hash already exists in VirusTotal's database
client <-create_http_client(timeout = 120 seconds)
hash_response <-client.GET(
    url: "https://www.virustotal.com/api/v3/files/" + file_hash,
    headers: { "x-apikey": VIRUSTOTAL_API_KEY }
)
IF hash_response.status = 200 THEN
    // Cached result found - return existing report immediately
    cached_report <-parse_file_report(hash_response.body)
    RETURN format_scan_result(cached_report)
END IF
// Step 3: File not in database - upload for analysis
IF file_size > 32,000,000 THEN // > 32 MB
    // Request special upload URL for large files
    upload_url_response <-client.GET(
        url: "https://www.virustotal.com/api/v3/files/upload_url",
        headers: { "x-apikey": VIRUSTOTAL_API_KEY }
    )
    upload_url <-parse_json(upload_url_response.body).data
ELSE
    upload_url <-"https://www.virustotal.com/api/v3/files"
END IF
// Step 4: Upload file via multipart form data
upload_response <-client.POST_multipart(
    url: upload_url,
    headers: { "x-apikey": VIRUSTOTAL_API_KEY },
    file: file_path,
    file_name: extract_filename(file_path)
)
IF upload_response.status != 200 THEN
    RETURN error("Upload failed: " + upload_response.status)
END IF
// Step 5: Extract analysis ID and poll for completion
upload_json <-parse_json(upload_response.body)
analysis_id <-upload_json.data.id
// Poll using same loop as Algorithm 5.5
scan_result <-poll_for_analysis(analysis_id, client)

```

```
    RETURN scan_result
END FUNCTION
```

The hash-first approach provides a significant performance benefit for commonly scanned files. In testing, caught file scans completed in approximately 1.2 seconds compared to 22.3 seconds for new file uploads. The SHA-256 hash is computed using 8 KB streaming reads to avoid loading the entire file into memory, which is important for large files.

5.5.7 Image EXIF Metadata Scanning Algorithm

The image privacy scanner reads EXIF metadata embedded in JPEG and PNG images and presents it to the user in a structured format. This helps students understand that photos they share online may contain hidden information such as GPS coordinates, camera details, and time-stamps. The algorithm is implemented in the Rust `image_privacy.rs` module.

Algorithm 5.7: Scan Image EXIF Metadata

```
FUNCTION scan_image_metadata(file_path):
    results <- { basic_info: {}, camera_info: {}, gps_info: {}, exif_info: {} }
    // Step 1: Read filesystem metadata
    file_metadata <- read_file_metadata(file_path)
    results.basic_info.file_name <- file_metadata.file_name
    results.basic_info.file_size <- file_metadata.size_bytes
    results.basic_info.file_size_formatted <- format_bytes(file_metadata.size_bytes)
    results.basic_info.modified_date <- format_timestamp(file_metadata.modified)
    // Step 2: Decode image to extract dimensions and colour info
    image <- decode_image(file_path) // Using the `image` crate
    results.basic_info.dimensions <- image.width + " x " + image.height
    results.basic_info.color_type <- image.color_type // e.g., "RGB", "RGBA", "Grayscale"
    results.basic_info.bit_depth <- image.bit_depth
    // Step 3: Read EXIF data from file bytes
    file_bytes <- read_file_bytes(file_path)
    exif_reader <- create_exif_reader()
    exif_data <- exif_reader.read_from_container(file_bytes)

    IF exif_data IS NOT NULL THEN
        // Step 4: Extract camera information
        FOR EACH field IN exif_data.fields DO
            IF field.tag = "Make" THEN
                results.camera_info.make <- field.value
            ELSE IF field.tag = "Model" THEN
                results.camera_info.model <- field.value
            ELSE IF field.tag = "Software" THEN
                results.camera_info.software <- field.value
            ELSE IF field.tag = "DateTime" THEN
                results.camera_info.date_time <- field.value
            END IF
        END FOR
    END IF
END FUNCTION
```

```

// Step 5: Extract GPS coordinates and convert to decimal
gps_latitude <-get_exif_field(exif_data, "GPSLatitude")
gps_latitude_ref <-get_exif_field(exif_data, "GPSLatitudeRef")
gps_longitude <-get_exif_field(exif_data, "GPSLongitude")
gps_longitude_ref <-get_exif_field(exif_data, "GPSLongitudeRef")
IF gps_latitude IS NOT NULL AND gps_longitude IS NOT NULL THEN
  // Convert DMS (degrees, minutes, seconds) to decimal
  lat_decimal <-convert_dms_to_decimal(gps_latitude)
  IF gps_latitude_ref = "S" THEN lat_decimal <--lat_decimal
  lon_decimal <-convert_dms_to_decimal(gps_longitude)
  IF gps_longitude_ref = "W" THEN lon_decimal <--lon_decimal
  results.gps_info.latitude <-lat_decimal
  results.gps_info.longitude <-lon_decimal
  results.gps_info.maps_url <-"https://www.google.com/maps?q="
    + lat_decimal + "," + lon_decimal
END IF
// Step 6: Extract camera settings
results.exif_info.aperture <-get_exif_field(exif_data, "FNumber")
results.exif_info.iso <-get_exif_field(exif_data, "ISOSpeedRatings")
results.exif_info.focal_length <-get_exif_field(exif_data, "FocalLength")
results.exif_info.exposure_time <-get_exif_field(exif_data, "ExposureTime")
END IF
RETURN results
END FUNCTION
FUNCTION convert_dms_to_decimal(dms_array):
  // dms_array contains [degrees, minutes, seconds] as rational numbers
  degrees <-dms_array[0].to_float()
  minutes <-dms_array[1].to_float()
  seconds <dms_array[2].to_float()
  RETURN degrees + (minutes / 60) + (seconds / 3600)
END FUNCTION

```

The algorithm converts GPS coordinates from the EXIF DMS (degrees, minutes, seconds) format to decimal degrees and generates a clickable Google Maps URL. This makes the privacy implications tangible - when a student sees their photo's exact location plotted on a map, the importance of metadata stripping becomes immediately clear

Algorithm 5.8: Strip Image EXIF Metadata

```

FUNCTION strip_image_metadata(input_path, output_path):
  // Step 1: Read original file bytes
  file_bytes <- read_file_bytes(input_path)
  // Step 2: Detect image format and strip EXIF accordingly
  IF file_extension(input_path) = ".jpg" OR file_extension(input_path) = ".jpeg" THEN
    jpeg <-Jpeg::from_bytes(file_bytes) // Using img_parts crate
    jpeg.exif_data <-NULL                // Remove all EXIF segments
  END IF
  write_file_bytes(output_path, jpeg.to_bytes())
END FUNCTION

```

```

        clean_bytes <-jpeg.encode()
    ELSE IF file_extension(input_path) = ".png" THEN
        png <-Png::from_bytes(file_bytes)    // Using img_parts crate
        png.exif_data <-NULL                // Remove all EXIF chunks
        clean_bytes <-png.encode()
    ELSE
        RETURN error("Unsupported image format. Use JPEG or PNG.")
    END IF
    // Step 3: Write clean image to output path
    write_file_bytes(output_path, clean_bytes)
    RETURN { success: true, output_path: output_path }
END FUNCTION

```

The stripping algorithm uses the `img_parts` crate to parse the image at the binary level, remove the EXIF segments (APP1 marker in JPEG, or eXIf chunk in PNG), and re-encode the image. The pixel data is preserved exactly - only the metadata is removed. This is safer than re-encoding the image, which could introduce quality loss.

5.5.8 AI Chatbot with Streaming Response Algorithm

The AI chatbot uses a streaming architecture to deliver responses in real time, character by character, rather than making the user wait for the full response to be generated. This is implemented across three layers: the TypeScript frontend sends the request via Tauri IPC, the Rust `ai_agent.rs` module manages the streaming connection to OpenRouter, and responses flow back through a Tauri Channel<String> to the frontend for progressive rendering. The chatbot is restricted to cybersecurity and AI topics only through a system prompt.

Algorithm 5.9: AI Chat with Streaming Response

```

FUNCTION chat_with_ai(user_message, conversation_history, stream_channel):
    // Step 1: Prepend cybersecurity-only system prompt
    system_prompt <- "You are CHEA, a cybersecurity education assistant for " +
        "students aged 9-15. Answer only cybersecurity and AI " +
        "related questions. Use simple, student-friendly language. " +
        "If the question is off-topic, politely redirect to " +
        "cybersecurity topics."
    messages <-[{ role: "system", content: system_prompt }]
    // Step 2: Append conversation history for context continuity
    FOR EACH msg IN conversation_history DO
        messages.append({ role: msg.role, content: msg.content })
    END FOR

```

```

// Step 3: Append the current user message
messages.append({ role: "user", content: user_message })

// Step 4: Build OpenRouter API request
request_body <- {
  model: "x-ai/grok-4-fast",
  messages: messages,
  stream: true // Enable Server-Sent Events streaming
}

// Step 5: Send streaming POST request to OpenRouter
client <-create_http_client(timeout = 120 seconds)
response <-client.POST_streaming(
  url: "https://openrouter.ai/api/v1/chat/completions",
  headers: {
    "Authorization": "Bearer " + OPENROUTER_API_KEY,
    "Content-Type": "application/json"
  },
  body: serialise_to_json(request_body)
)
IF response.status != 200 THEN
  stream_channel.send({ type: "error", message: "AI service unavailable" })
  RETURN
END IF

// Step 6: Read Server-Sent Events (SSE) streamline by line
full_response <- ""
FOR EACH line IN response.stream DO
  IF line.starts_with("data: ") THEN
    data <-line.substring(6) // Strip "data: " prefix
    IF data = "[DONE]" THEN
      // Step 7: Stream complete - send end signal
      stream_channel.send({ type: "done", content: full_response })
      BREAK
    END IF
    json <-parse_json(data)
    content_chunk <-json.choices[0].delta.content
    IF content_chunk IS NOT NULL THEN
      full_response <-full_response + content_chunk
    // Step 8: Forward chunk to frontend immediately

```

```

        stream_channel.send({ type: "chunk", content: content_chunk })
    END IF
END IF
END FOR
// Step 9: Persist the complete response to Firestore
save_message_to_session(user_id, session_id, "user", user_message)
save_message_to_session(user_id, session_id, "assistant", full_response)
END FUNCTION

```

The streaming architecture provides two key benefits. First, the user sees the response building in real time, which reduces perceived latency - the first token typically appears within 2 seconds, even though the full response may take 8-10 seconds to complete. Second, the system can handle very long responses (e.g., detailed explanations of encryption algorithms) without blocking the UI, since each chunk is rendered independently as it arrives.

The system prompt restriction is a critical safety feature for the target age group. By instructing the model to only answer cybersecurity and AI-related questions and to redirect off-topic queries, we ensure that the chatbot remains focused on its educational purpose. The conversation history is included in each request so the model can maintain context across multi-turn conversations - for example, a student can ask "Can you explain that again more simply?" and the AI understands what "that" refers to.

The OpenRouter API key is loaded from environment variables via the dotenvy crate in the Rust backend and is never exposed to the frontend JavaScript context, consistent with the security requirement NFR-01.

5.5.9 XP Calculation and Level Progression Algorithm

The gamification system calculates XP awards for each activity type and manages level progression through a 10-tier threshold table. The XP thresholds increase exponentially to ensure early levels feel rewarding while higher levels require sustained engagement. This design is informed by gamification research (Pramod, 2025; Khairallah and Abu-Naseer, 2024) showing that visible early progression improves motivation and long-term retention.

Table 13: Activity XP Rewards

Activity Type	XP Award	Daily Task Mapping
scan_link	10	scan (target: 1)
scan_file	15	scan (target: 1)
scan_image	10	scan (target: 1)
generate_password	5	generate_password (target: 1)

check_password	3	check_password (target: 1)
generate_encryption	5	use_encryption (target: 1)
create_credential	20	create_credential (target: 1)
chat_ai	5	- (no daily task)
quiz_round	15	play_quiz (target: 1)
phishing_round	15	spot_phish (target: 1)

Algorithm 5.10: Calculate Phishing Dojo XP

The Phishing Dojo and Quiz Arena use a bonus-based XP calculation that rewards accuracy, consistency, and perfection.

```

FUNCTION calculate_phishing_xp(correct_calls, max_streak, total_emails):
    // Base XP: 3 points per correct identification
    base_xp <- correct_calls * 3
    // Streak bonus: 1 point per consecutive correct beyond the first
    IF max_streak > 1 THEN
        streak_bonus <- (max_streak - 1) * 1
    ELSE
        streak_bonus < 0
    END IF
    // Completion bonus: reward for high performance
    IF correct_calls = total_emails THEN
        completion_bonus <- 10 // Perfect score
    ELSE IF correct_calls >= 3 THEN
        completion_bonus <- 5 // Good performance (60%+ correct)
    ELSE
        completion_bonus <- 0
    END IF
    total_xp <- base_xp + streak_bonus + completion_bonus
    RETURN { base_xp, streak_bonus, completion_bonus, total_xp }
END FUNCTION

```

The same formula applies to Quiz Arena XP. The streak bonus incentivizes consistent correct answers rather than lucky guesses, while the completion bonus rewards perfect or near-perfect rounds.

Algorithm 5.11: Level Progression

```
// Level threshold table (10 tiers with exponentially increasing XP requirements)
LEVEL_THRESHOLDS <- [
  { level: 1, xp: 0, title: "Novice" },
  { level: 2, xp: 100, title: "Apprentice" },
  { level: 3, xp: 300, title: "Guardian" },
  { level: 4, xp: 600, title: "Defender" },
  { level: 5, xp: 1000, title: "Sentinel" },
  { level: 6, xp: 1500, title: "Champion" },
  { level: 7, xp: 2200, title: "Hero" },
  { level: 8, xp: 3000, title: "Legend" },
  { level: 9, xp: 4000, title: "Mythic" },
  { level: 10, xp: 5500, title: "Omniscient" }
]

FUNCTION get_level_info(total_xp):
  // Find the highest threshold that the user's XP meets or exceeds
  current_level <- LEVEL_THRESHOLDS[0]
  next_level <- LEVEL_THRESHOLDS[1]
  FOR i FROM LENGTH(LEVEL_THRESHOLDS) - 1 DOWNT0 0 DO
    IF total_xp >= LEVEL_THRESHOLDS[i].xp THEN
      current_level <- LEVEL_THRESHOLDS[i]
      IF i + 1 < LENGTH(LEVEL_THRESHOLDS) THEN
        next_level <- LEVEL_THRESHOLDS[i + 1]
      ELSE
        next_level <- LEVEL_THRESHOLDS[i] // Already at max level
      END IF
      BREAK
    END IF
  END FOR
  xp_in_level <- total_xp - current_level.xp
  xp_needed <- next_level.xp - current_level.xp
  progress_percentage <- (xp_in_level / xp_needed) 100 - 1
  RETURN {
    level: current_level.level,
    title: current_level.title,
    current_xp: total_xp,
    xp_for_next_level: next_level.xp,
```

```

    xp_in_level: xp_in_level,
    xp_needed: xp_needed,
    progress: progress_percentage
  }
END FUNCTION

```

Algorithm 5.12: Award XP and Update Streak

```

FUNCTION add_xp(user_id, points):
  // Step 1: Fetch current progress from Firestore
  progress <- fetch_user_progress(user_id)
  today <- current_date_as_iso_string() // e.g., "2025-04-23"
  yesterday <- previous_day_as_iso_string() // e.g., "2025-04-22"
  // Step 2: Calculate new XP and level
  new_xp <- progress.xp + points
  level_info <- get_level_info(new_xp)
  // Step 3: Update streak counter
  IF progress.last_active_date = today THEN
    // Already active today - streak unchanged
    new_streak <- progress.streak_days
  ELSE IF progress.last_active_date = yesterday THEN
    // First activity today and was active yesterday - streak continues
    new_streak <- progress.streak_days + 1
  ELSE
    // Gap of more than one day - streak resets to 1
    new_streak <- 1
  END IF
  // Step 4: Build updated progress document
  updated_progress <- {
    xp: new_xp,
    level: level_info.level,
    totalScore: progress.totalScore + points,
    streakDays: new_streak,
    lastActiveDate: today
  }
  // Step 5: Persist to Firestore
  save_user_progress(user_id, updated_progress)
  RETURN updated_progress

```

END FUNCTION

The streak mechanism is a key engagement driver. It tracks consecutive calendar days of activity by comparing lastActiveDate with today's and yesterday's dates. If a day is missed, the streak resets to 1. This encourages students to open the application daily, reinforcing the habit-forming objective central to the project's "Cyber Hygiene" mission. The level thresholds are designed so that the first few levels are achievable within a single session (e.g., Level 2 at 100 XP can be reached by scanning a few links and generating a password), providing immediate positive reinforcement.

5.5.10 AI Scenario Simulator Algorithm

The AI Scenario Simulator generates interactive cybersecurity scenarios with multiple-choice questions, correct answers, and educational feedback. The algorithm generates all five rounds in a single AI call at the start of the session for consistent difficulty progression, then presents them one at a time. The algorithm is implemented in the ScenarioSimulator.tsx page component.

Algorithm 5.13: Generate Security Scenarios

```
FUNCTION generate_security_scenarios(language, num_rounds = 5):  
  // Step 1: Build scenario generation prompt  
  prompt <- "Generate " + num_rounds + " cybersecurity scenario questions " +  
    "for a student aged 9-15. Each scenario should present a realistic " +  
    "security situation and ask what the student should do. " +  
    "Return ONLY valid JSON matching this schema: " +  
    "[{" +  
    "  \"scenario\": string, " +  
    "  \"options\": [string, string, string, string], " +  
    "  \"correctIndex\": number (0-3), " +  
    "  \"explanation\": string " +  
    "}]". " +  
    "Scenarios should cover: phishing, password security, social media, " +  
    "public WiFi, personal data sharing, malware, and safe browsing. " +  
    "Make scenarios relatable to a student's daily life. " +  
    "Progress from easier (round 1) to more challenging (round 5). " +  
    "All text must be in " + language + " language."  
  // Step 2: Call AI service with JSON response mode  
  response <- call_nova(prompt, temperature = 0.8, max_tokens = 2000,  
    response_format = "json")  
  // Step 3: Parse and validate  
  scenarios <- parse_json(response)  
  IF length(scenarios) < num_rounds THEN
```

```

    RETURN error("Insufficient scenarios generated")
END IF
FOR EACH scenario IN scenarios DO
    IF NOT validate_scenario_schema(scenario) THEN
        RETURN error("Invalid scenario format from AI")
    END IF
END FOR
// Step 4: Return validated scenarios
RETURN scenarios[0..num_rounds]
END FUNCTION

```

The algorithm generates all rounds at once to ensure a smooth user experience (no loading between rounds). Each scenario includes an explanation field that is displayed after the student answers, providing immediate educational feedback specific to the scenario. The progressive difficulty instruction ensures the experience builds from simple to complex.

5.5.11 AI Security Posture Assessment Algorithm

The AI Security Posture Assessment evaluates a student's real-world cybersecurity habits through a 15-question survey and generates a personalised security report card. Unlike the Quiz Arena which tests knowledge, this assessment evaluates behaviour. The questions are drawn from a static question bank (securityQuestions.ts) with complete translations in English and Arabic, selected by the useSecurityQuestions hook based on the active language. After the student submits all answers, the AI generates a comprehensive report. The algorithm is implemented in the SecurityPosture.tsx page component.

Algorithm 5.14: Generate Security Posture Report

```

FUNCTION generate_posture_report(questions, answers, language):
    // Step 1: Build question-answer summary for AI analysis
    summary <- "A student completed a 15-question cybersecurity habits survey. "
    FOR i FROM 0 TO length(questions) - 1 DO
        summary <- summary + "Q" + (i+1) + ": " + questions[i].text +
            " Answer: " + answers[i] + ". "
    END FOR
    // Step 2: Build report generation prompt
    prompt <- summary +
        "Based on these answers, generate a personalised security report " +
        "card for a student aged 9-15. Return ONLY valid JSON: " +
        "{ " +
        "  \"grade\": string (A+, A, B+, B, C+, C, D, F), " +

```

```

" \"score\": number (0-100), " +
" \"summary\": string (2-3 sentence overall assessment), " +
" \"strengths\": [string], " +
" \"weaknesses\": [string], " +
" \"recommendations\": [{ \"area\": string, \"tip\": string }], " +
" \"encouragement\": string (motivational closing message) " +
"}. " +
"Be encouraging but honest. Give specific, actionable advice. " +
"All text must be in " + language + " language."

// Step 3: Call AI service
response <- call_nova(prompt, temperature = 0.7, max_tokens = 1500,
                      response_format = "json")

// Step 4: Parse and validate report
report <- parse_json(response)
IF NOT validate_report_schema(report) THEN
  RETURN error("Invalid report format from AI")
END IF

// Step 5: Return report
RETURN report
END FUNCTION

```

The question-answer summary sent to the AI contains only the question texts and selected answer labels (e.g., "Always," "Sometimes," "Never"), never any personal data or account information. The grade and score are generated by the AI based on the overall pattern of answers, providing a more nuanced assessment than a simple point-counting approach. The encouragement field ensures the report ends on a positive note, motivating the student to improve rather than discouraging them.

5.6 Summary

This chapter presented the complete system design of CHEA across five dimensions. The database schema (5.2 Database Schema Design) defines a Firestore-based NoSQL structure with eight collections and subcollections, all scoped under individual user paths for data isolation, with client-side encryption ensuring that no sensitive data is stored in plain text, and a custom REST API workaround for Tauri's WebView2 environment. The schema was extended with three new AI-specific activity types (ai_phishing_round, scenario_simulator, security_posture) and a localStorage-based cache for the Security Buddy daily challenges widget. The user interface design (5.3 User Interface Design) describes twenty-one interfaces covering the full user journey from authentication flows (Login, Registration, Forgot Password), through the central Dashboard with the AI-powered Security Buddy widget, to security tools (Password Generator, Password Checker with AI Attack

Narrative, Link Scanner with AI Scan Analysis, File Scanner with AI Scan Analysis, Encryption Lab, Credential Vault, Image Privacy), educational games (Phishing Dojo with AI Challenge mode, Quiz Arena), AI learning experiences (AI Scenario Simulator, AI Security Posture Assessment), the AI Agent chatbot with streaming responses, Settings with avatar selection and vault configuration, and the navigation layout with floating widgets (NovaChat, MusicPlayer), all following a cyberpunk aesthetic designed for the 9-15 age group with full bilingual support (English and Arabic with RTL layout). The application architecture (5.4 Application Architecture) documents the routing system with authentication guards, the Zustand state management layer, the centralised AI service module, and the reusable UI component library including twelve UI primitives and three AI-specific components, with a QuickGuide onboarding component integrated into four tool pages to support first-time users aged 9–15. The algorithm designs (5.5 Algorithm Design) provide pseudocode for fourteen core algorithms spanning encryption and decryption, password generation and strength analysis, URL and file scanning via VirusTotal with polling, image EXIF metadata scanning and stripping, AI-powered cybersecurity chat with Server-Sent Events streaming, AI scan analysis (FR-11), AI attack narrative generation (FR-12), AI phishing email generation (FR-13), AI scenario simulation (FR-14), AI security posture assessment (FR-15), and the gamification engine with XP calculation and level progression.

These design specifications directly address the functional requirements defined in Chapter 4 (including FR-11 through FR-16) and the research gaps identified in Chapter 2. The following chapter describes the implementation of these designs and presents the testing results.

Chapter 6

System Implementation and Testing

6.1 System Implementation

6.1.1 Development Environment Setup

We built CHEA using a multi-language stack TypeScript on the frontend and Rust on the backend. Here's how we set up the development environment:

Hardware Platform:

We developed a Windows 11 machine with an Intel Core processor, 16 GB RAM, and an SSD. This was enough to handle compiling Rust, running the Vite dev server with hot module replacement, and the Tauri desktop runtime all at the same time without much lag.

Integrated Development Environment:

We used Visual Studio Code as the main IDE. It has solid extension support for TypeScript (ESLint, Prettier), Rust (rust-analyzer), and Tauri development. The integrated terminal, Git integration, and real-time type-checking through the TypeScript language server made the development workflow much smoother.

Languages and Runtimes:

- **TypeScript 5.5.3:** All frontend code components, services, logic was written in TypeScript with strict type-checking enabled. This caught a lot of bugs at compile time that would have been painful to debug at runtime.
- **Rust Edition 2021:** The backend language, compiled through Cargo. We chose Rust for its memory safety without garbage collection, its performance (comparable to C/C++), and because Tauri's command handlers are natively written in Rust.

Frameworks and Libraries:

- **Tauri v2.10.1:** The desktop application framework. We went with Tauri over Electron because it uses a lightweight Rust backend instead of bundling a full Chromium instance. Tauri v2's invoke IPC system, channel-based streaming, and plugin architecture were essential for how we built the app.
- **React 18.3.1:** The UI library. We used functional components with hooks throughout, and the forwardRef pattern for components that needed ref forwarding.
- **Vite 5.4.1:** The build tool. Vite's sub-second hot module replacement was a huge quality-of-life improvement during development, and its production builds with tree-shaking and code splitting kept the bundle size reasonable.
- **OpenCode and KiloCode:** AI coding assistant used throughout the development process for code generation, debugging, refactoring suggestions, and receiving real-time development guidance.

- **OpenRouter API:** AI API gateway providing access to multiple large language models. We used x-ai/grok-4-fast as the primary model for all AI-powered features including the chatbot, scan analysis, attack narratives, phishing email generation, scenario simulation, and security posture assessment. Additional models available through OpenRouter include MiniMax, GLM (Zhipu AI), ZAI, and Kimi2 (Moonshot AI). OpenRouter was chosen for its free tier availability, model diversity, streaming support, and multilingual capability essential for the bilingual CHEA application supporting English and Arabic.

Build Tools and Package Management:

Build Tools and Package Management:

- Bun served as our JavaScript/TypeScript package manager and script runner. We switched from npm early on because Bun's dependency resolution was noticeably faster.
- Cargo managed all Rust dependencies in Cargo.toml, including cryptographic crates (aes-gcm, chacha20poly1305, argon2), the HTTP client (reqwest), and image processing libraries (kamadak-exif, img-parts).

Version Control:

We used Git with a GitHub remote repository. The repo structure separates the frontend source (src/), the Tauri backend source (src-tauri/src/), configuration files, and documentation.

Database:

We set up Firebase Firestore (NoSQL) through the Firebase Console under the project identifier chea-new. Firestore's document-oriented model worked well for storing user data, vault configurations, activity logs, chat sessions, and gamification progress in hierarchical collections and subcollections. We chose Firebase Firestore (NoSQL) over traditional relational databases for several reasons:

- (1) **Real-time Synchronization** - Firestore provides automatic real-time updates ensuring user data stays synchronized across sessions;
- (2) **Native Firebase Auth Integration** - seamless integration with Firebase Authentication providing a complete identity and data solution;
- (3) **Document-Oriented Model** - the hierarchical NoSQL structure perfectly suits CHEA's data organization with nested subcollections for vault credentials, activity logs, chat sessions, and gamification;
- (4) **Security Rules** - Firestore's security rules provide fine-grained access control ensuring each user can only access their own data;
- (5) **Cost-Effective** - Firebase offers a generous free tier suitable for academic project requirements;
- (6) **Offline Support** - Firestore's offline persistence enables the application to function with intermittent network connectivity.

Testing Infrastructure:

We didn't set up an automated testing framework for this project. All testing was done manually through the running application, following the structured test cases documented in 6.2 Evaluation and Testing. Given the senior capstone timeline, we prioritized getting features working over writing test automation something we'd change if we were building this for production.

6.1.2 Frontend Implementation

We built the frontend as a single-page application (SPA) using React 18 and Typescript, bundled by Vite, and rendered inside a Tauri WebView2 window on Windows (or WebKit on macOS).

6.1.2.1 Project Structure

We organized the source code into a modular directory structure under src/:

```
src/
├── components/
│   └── ui/
│       # Reusable UI primitives (Button, Input, Card, Checkbox, TitleBar, ThemeToggle, PasswordStrength,
│       PasswordInput, QuickGuide)
│   ├── ai/      # AI-powered components (ScanAIAnalysis, AttackNarrative, SecurityBuddy)
│   ├── layout/  # Layout components (DashboardLayout with sidebar, header, and navigation)
│   ├── auth/    # Authentication-related components (AuthLayout)
│   └── theme-provider.tsx # Theme context provider
├── pages/       # Route-level page components (15 feature pages + landing, login, register)
├── services/     # Firebase service modules (activityService, vaultService, credentialService, chatService,
dailyTasksService, userProgressService, quizService, phishingService, passwordHistoryService, aiService,
aiPhishingGenerator)
├── store/        # Zustand global state stores (useAuthStore, useActivityStore, useUserProgressStore,
useDailyTasksStore)
├── hooks/        # Custom React hooks (useTrackActivity, useSecurityQuestions)
├── lib/          # Utility modules (firebase.ts, firestore-rest.ts, utils.ts, avatar.ts)
├── data/         # Static data (serviceIcons.tsx, securityQuestions.ts)
├── App.tsx       # Root component with routing configuration
└── main.tsx      # Application entry point
```

Keeping business logic (services), state (stores), presentation (components/pages), and infrastructure (lib) in separate folders made the codebase much easier to navigate as it grew.

6.1.2.2 Routing and Navigation

We implemented client-side routing using React Router DOM v6.26.0 with nested route configuration.

The App.tsx component defines all application routes:

Public Routes (accessible without authentication):

- / — Landing page introducing CHEA
- /login — User login form
- /register — User registration form
- /forgot-password — Password reset flow
- /terms — Terms of Service
- /privacy — Privacy Policy

Protected Routes (require authentication, nested under /dashboard):

- /dashboard — Home dashboard with activity overview
- /dashboard/link-scanner — URL scanning interface
- /dashboard/file-scanner — File scanning interface
- /dashboard/metadata — Photo metadata scanner
- /dashboard/password-gen — Password generator
- /dashboard/password-check — Password strength checker
- /dashboard/encryption — Text encryption tool
- /dashboard/vault — Credential vault
- /dashboard/ai-agent — AI chatbot interface
- /dashboard/quiz-arena — Cybersecurity quiz game
- /dashboard/phishing-dojo — Phishing email recognition game
- /dashboard/scenario-simulator — AI-powered security scenario game
- /dashboard/security-posture — AI security posture assessment
- /dashboard/settings — User settings

Two route guard components enforce access control:

- **ProtectedRoute:** Wraps dashboard routes, redirecting unauthenticated users to /login.
- **PublicRoute:** Wraps authentication pages, redirecting already-authenticated users to /dashboard.

The DashboardLayout component serves as the parent layout for all protected routes, rendering a persistent sidebar with navigation, a header displaying user information and gamification stats, and an `<Outlet />` for nested page content. Navigation items are organized into labeled sections (Scanning Tools, Password Tools, Encryption & Vault, Game Zone, AI Learning Zone) using the Lucide React icon library.

6.1.2.3 State Management

We used four Zustand stores for global state, each handling a different part of the app:

1. **useAuthStore** - Manages the Firebase authentication state (user: User | null), a loading indicator for auth initialization, and the vault master password. The `onAuthStateChanged` Firebase listener in `App.tsx` updates this store, so the current user is accessible throughout the component tree without prop drilling.
2. **useActivityStore** - Manages the user's recent activity log. It provides `fetchActivities(userId)` to retrieve the latest 15 activities from Firestore and `logActivity(userId, type, metadata)` to record a new activity. Activities include scanning links/files/images, generating passwords, encrypting text, creating vault credentials, chatting with the AI, completing quiz rounds, and identifying phishing emails.

3. **useUserProgressStore** - Manages gamification state including XP, level, streak days, and level title. The `earnXp(userId, points)` method adds XP and recalculates the user's level using a 10-tier progression system (Novice at 0 XP through Omniscient at 5,500 XP). Streak tracking compares the user's `lastActiveDate` with the current and previous day to increment or reset the streak counter.

4. **useDailyTasksStore** - Manages daily task completion state. Seven task types are defined (`scan`, `generate_password`, `check_password`, `create_credential`, `use_encryption`, `play_quiz`, `spot_phish`), each with a target count and point reward. Tasks are date-keyed in Firestore and automatically reset each day.

6.1.2.4 Activity Tracking Hook

The `useTrackActivity` custom hook is the central integration point for the gamification system. When called with an activity type and optional metadata, it does three things in sequence:

1. Logs the activity to Firestore via `useActivityStore.logActivity`.
2. Awards XP to the user via `useUserProgressStore.earnXp`, using a predefined point table (e.g., `scan_link: 10`, `scan_file: 15`, `quiz_round: 15`, `create_credential: 20`, `scenario_simulator: 20`, `security_posture: 25`).
3. Increments the relevant daily task counter via `useDailyTasksStore.completeTask`.

Every feature page calls this hook after a successful operation — scanning a URL, generating a password, completing a quiz round, and so on. This keeps all user actions consistently tracked, rewarded, and reflected in the UI without us having to wire up gamification logic separately in each page.

6.1.2.5 Firebase Integration

Firebase was integrated in two modes, and this was actually one of the trickier parts of the project:

Firestore Auth SDK:

We used the standard Firebase Authentication SDK (`firebase/auth`) for user registration, login, logout, and password reset. Authentication persistence was configured with `browserLocalPersistence` by default, with an option for session-based persistence via the "Remember Me" toggle. We used `initializeAuth` instead of `getAuth` to explicitly control persistence behavior.

Firestore REST API Workaround:

This was a real headache. The Firestore Web SDK uses `gRPC-web/WebChannel` internally, which Tauri's `WebView2` engine (Microsoft Edge) blocks. We kept getting persistent "client is offline" errors and spent a while debugging before realizing it was a transport layer incompatibility. To fix this, we wrote a custom Firestore REST API helper (`src/lib/firestore-rest.ts`) that uses standard `fetch()` calls to the Firestore v1 REST endpoint (`https://firestore.googleapis.com/v1/projects/chea-new/databases/(default)/documents`). This module provides:

- **firestoreGetDoc(collection, docId)** — Retrieves a document, returning null for non-existent documents.

- **firestoreSetDoc(collection, docId, data)** — Creates or overwrites a document using PATCH.

We had to write field conversion functions (`toFirestoreFields`, `fromFirestoreFields`) to handle serialization between plain JavaScript objects and Firestore's typed field format (`stringValue`, `integerValue`, `booleanValue`, `arrayValue`, `mapValue`, `timestampValue`). Authentication for REST calls uses the Firebase ID token from `auth.currentUser.getIdToken()`.

For operations needing real-time updates or complex queries (like `addDoc` with `serverTimestamp`), we used the standard Firestore SDK with `experimentalForceLongPolling: true` enabled, which partially works around the WebView2 gRPC issue.

6.1.2.6 Theme System

We built a custom theme system using a React Context provider (`ThemeProvider`) that supports three modes:

- **Dark mode:** Activates the `.dark` CSS class on the document root, applying a cyberpunk-inspired palette with deep navy backgrounds (`#0A1128`), neon crimson accents (`#FF0A54`), and blue-tinted text (`#8AB4F8`).

- **Light mode:** Applies the `.light` class with a clean, professional palette using white backgrounds, violet accents (`#4D00FF`), and neutral gray tones.

- **System mode:** Automatically follows the operating system's `prefers-color-scheme` media query.

Theme preference is saved in `localStorage` under the key `chea-ui-theme`. The `useTheme()` hook exposes the current theme, resolved theme (always "dark" or "light"), and a `setTheme` setter. All components use the `isDark` boolean from `resolvedTheme` to conditionally apply Tailwind classes.

6.1.2.7 Key UI Components

We designed reusable UI components following consistent patterns:

- **Button:** A `forwardRef` component supporting multiple variants, sizes, and states (loading, disabled). Uses Tailwind for styling with hover/focus transitions.

- **Input / PasswordInput:** Form input components with label, error, and icon support. `PasswordInput` adds a visibility toggle.

- **Card:** A container component with consistent border, shadow, and padding styling.

- **TitleBar:** A custom window title bar integrated with Tauri's window management, replacing the native OS title bar for a unified aesthetic.

- **ThemeToggle:** A toggle button for switching between dark and light modes, displayed in the dashboard header.

- **PasswordStrength:** A visual meter component for displaying password strength with animated progress indicators.

- **ScanAIAnalysis:** A reusable AI-powered analysis card used by both Link Scanner and File Scanner. After a scan completes, it sends detection statistics to the AI service and renders a plain-language risk assessment in the user's active language.
- **AttackNarrative:** An AI-powered hacker narrative card for the Password Checker. It sends only password characteristics (never the actual password) to the AI and renders an educational "how a hacker would crack this" narrative.
- **SecurityBuddy:** A Dashboard widget displaying an AI-generated daily cybersecurity challenge with a title, description, recommended tool, and navigation button. Challenge data is cached in localStorage keyed by user ID, date, and language.

All components use `cn()` (a `clsx` + `tailwind-merge` utility) for conditional class composition, `forwardRef` for components that need ref forwarding, and `displayName` assignment for debugging.

6.1.2.8 AI Service Layer

We implemented a centralised AI service layer to support six AI-powered features (FR-11 through FR-16). All AI calls route through the OpenRouter API using the `x-ai/grok-4-fast` model, with the API key stored as a Vite environment variable (`VITE_OPENROUTER_API_KEY`).

Core AI Service (`src/services/aiService.ts`):

This module provides two functions shared across all AI features:

- **callNova(prompt, options)** - Sends a standard (non-streaming) request to the OpenRouter chat completions endpoint and returns the full response text. Used by ScanAIAnalysis (FR-11), AttackNarrative (FR-12), AI Phishing Generator (FR-13), Scenario Simulator (FR-14), and Security Posture Assessment (FR-15).
- **callNovaStreaming(prompt, options)** - Sends a streaming request and returns an async iterator for progressive rendering.

Each call includes the user's active language in the prompt, ensuring AI-generated content is always produced in the correct language (English or Arabic).

AI Phishing Generator (`src/services/aiPhishingGenerator.ts`):

Extends the core service for the Phishing Dojo AI Challenge mode (FR-13). It constructs a prompt instructing the AI to generate a realistic phishing or legitimate email and return a JSON object conforming to the existing `PhishingEmail` interface (`senderName`, `senderEmail`, `subject`, `body`, `isPhishing`, `redFlags`). A validation function checks that all required fields exist and have the correct types before returning the email to the UI.

Security Questions Data (`src/data/securityQuestions.ts`):

Contains 15 cybersecurity habit questions in both English and Arabic, used by the Security Posture Assessment (FR-15). Questions cover password security, phishing awareness, data protection, social media safety, and device security.

Language-Aware Hook (src/hooks/useSecurityQuestions.ts):

A custom hook that reads the active language from i18next and returns the appropriate question set. Language changes are reactive — switching language immediately returns the translated questions.

6.1.3 Backend Implementation

The backend is written in Rust as a set of Tauri command handlers, each exposed to the frontend through the Tauri IPC (Inter-Process Communication) system. The Rust codebase is organized into six modules under src-tauri/src/.

6.1.3.1 Command Registration and IPC Architecture

The lib.rs file is the application entry point. It initializes the Tauri builder with required plugins (tauri-plugin-dialog, tauri-plugin-opener), sets up the PTY state as managed state, and registers all command handlers in the invoke_handler:

```
.invoke_handler(tauri::generate_handler![
    greet,
    crypto::encrypt_text,
    crypto::decrypt_text,
    image_privacy::scan_image_metadata,
    image_privacy::strip_image_metadata,
    virustotal::scan_url,
    virustotal::scan_file,
    ai_agent::chat_with_ai,
    diagram::save_diagram,
    terminal::write_to_pty,
    terminal::resize_pty
])
```

Frontend-to-backend communication works through the Tauri invoke IPC mechanism. The frontend calls `invoke('command_name', { parameters })`, which serializes arguments as JSON, sends them to the Rust process via a platform-specific IPC channel, deserializes in Rust, executes the command, and returns the result. Commands marked `async` run on the Tokio runtime; synchronous commands run on the calling thread. Results are serialized as JSON and returned as a `Result<T, String>` where T: `Serialize`.

6.1.3.2 Cryptography Module (crypto.rs)

The cryptography module handles text encryption and decryption using three algorithms, all protected by Argon2id key derivation. We chose these specific algorithms because they cover different use cases: AES-256-GCM for maximum security on modern hardware, ChaCha20-Poly1305 for devices without AES-NI support, and AES-128-CBC as a legacy option for compatibility.

Key Derivation:

The `derive_key` function uses Argon2id (the memory-hard variant of Argon2) to derive encryption keys from user-provided passwords. A random 16-byte salt is generated for each encryption operation using the OS's cryptographically secure random number generator (`OsRng`). This means identical passwords produce different keys every time, which prevents rainbow table and precomputation attacks.

Supported Algorithms:

1. AES-256-GCM: The primary algorithm we use for vault encryption. A 32-byte key is derived, a random 12-byte nonce (IV) is generated, and AES-GCM authenticated encryption produces both ciphertext and a 16-byte authentication tag. The tag ensures ciphertext integrity, any modification to the ciphertext, IV, or additional authenticated data is detected during decryption, causing the operation to fail with "wrong password or corrupted data."

2. ChaCha20-Poly1305: An alternative AEAD (Authenticated Encryption with Associated Data) cipher. A 32-byte key and 12-byte nonce are used with the ChaCha20 stream cipher and Poly1305 MAC. This provides the same security guarantees as AES-256-GCM but is optimized for software implementation without AES-NI hardware acceleration.

3. AES-128-CBC: A legacy block cipher mode using a 16-byte key and 16-byte IV. PKCS#7 padding is applied to align plaintext to the AES block size (16 bytes). Unlike GCM and ChaCha20, CBC doesn't provide built-in authentication but PKCS#7 padding validation during decryption gives a limited integrity check.

Serialization:

The encrypted output is serialized as a JSON EncryptedPayload containing the algorithm identifier (alg), Base64-encoded salt, IV, and ciphertext. This payload is then Base64-encoded again for safe transport and storage. During decryption, the process is reversed: the outer Base64 layer is decoded, the JSON payload is deserialized, the key is re-derived from the password and salt, and the appropriate decryption algorithm is applied.

6.1.3.3 VirusTotal Module (virustotal.rs)

The VirusTotal module provides URL and file scanning using the VirusTotal v3 REST API:

URL Scanning (scan_url):

1. The target URL is submitted to VirusTotal's /urls endpoint as URL-encoded form data.
2. VirusTotal returns an analysis ID.
3. The poll_analysis function repeatedly queries the /analyses/{id} endpoint at 4-second intervals (up to 30 attempts, roughly 2 minutes) until the analysis status becomes "completed."
4. The result is parsed into a ScanResult containing the target URL, overall status (malicious/suspicious/clean), reputation score, per-engine detection details, and aggregate statistics (malicious, suspicious, harmless, undetected counts).

File Scanning (scan_file):

1. The file is hashed locally using SHA-256 via the sha2 crate, read in 8 KB chunks to handle large files without excessive memory consumption.

2. The hash is queried against VirusTotal's `/files/{hash}` endpoint. If the file has been previously scanned, the cached report is returned immediately, avoiding redundant uploads.
3. If the file is unknown to VirusTotal, the file data is read into memory and uploaded via multipart form data. Files larger than 32 MB request a special upload URL from the `/files/upload_url` endpoint.
4. The resulting analysis ID is polled the same way as URL scanning.

HTTP Client:

A request: Client is configured with a 120-second timeout for all VirusTotal API operations. The API key is included in the `x-apikey` header for authentication.

6.1.3.4 Image Privacy Module (`image_privacy.rs`)

The image privacy module handles EXIF metadata scanning and stripping for JPEG and PNG images:

Metadata Scanning (`scan_image_metadata`):

1. File system metadata (file size, creation/modification/access timestamps) is read using `std::fs::metadata`.
2. Image dimensions and color type are determined using the image crate's decoder.
3. EXIF data is parsed using the kamadak-exif crate (`exif::Reader`), which extracts fields from the image's EXIF segment.
4. Extracted metadata is organized into structured categories:
 - Camera Data: Make, model, software.
 - GPS Data: Latitude/longitude (converted from DMS to decimal degrees), Google Maps URL.
 - Datetime Data: Original capture date, digitization date, file system timestamps.
 - Camera Settings: Aperture, exposure time, ISO, focal length, flash, white balance, orientation, resolution.
 - File Properties: File type, MIME type, dimensions, megapixels, bit depth, color type.

Metadata Stripping (`strip_image_metadata`):

1. The image file is read into memory as raw bytes.
2. For JPEG files, the `img_parts::jpeg::Jpeg` parser decodes the image structure, calls `set_exif(None)` to remove all EXIF segments, and re-encodes the image without metadata.
3. For PNG files, the same process is applied using `img_parts::png::Png`.
4. The cleaned image is written to the specified output path. The pixel data remains unmodified — only metadata segments are removed.

6.1.3.5 AI Buddy Module (`ai_agent.rs`)

The AI buddy module provides a streaming chat interface to the OpenRouter API, which proxies requests to the `x-ai/grok-4-fast` language model:

System Prompt:

A detailed system prompt constrains the AI to answer only cybersecurity and AI-related questions. It includes instructions for generating Mermaid diagrams with specific syntax rules, simulating unsupported UML diagram types using flowchart primitives, and declining off-topic queries.

Streaming Architecture:

The `chat_with_ai` command uses Tauri's `Channel<String>` IPC mechanism for real-time streaming. The implementation:

1. Loads the OpenRouter API key from environment variables via `dotenvy`.
2. Prepends the system prompt to the user's message history.
3. Sends a streaming request to `https://openrouter.ai/api/v1/chat/completions` with `stream: true`.
4. Reads the Server-Sent Events (SSE) response, parsing data: lines for JSON chunks containing `choices[0].delta.content`.
5. Each content chunk is immediately sent to the frontend via `on_chunk.send(content)`, enabling character-by-character rendering.
6. The data: `[DONE]` marker signals stream completion.

Model Fallback:

A `MODEL_CHAIN` array defines a priority-ordered list of models. If the primary model fails (returns a non-200 status or empty response), the system automatically tries the next model in the chain. This gives us resilience against model outages without the user noticing anything.

6.1.3.6 Terminal Module (terminal.rs)

The terminal module provides an integrated terminal emulator using a pseudo-terminal (PTY):

1. On first use (`get_or_init_pty`), a native PTY pair is created using the `portable_pty` crate with an initial size of 80×24 characters.
2. On Windows, `powershell.exe` is spawned as the child process; on Unix systems, `bash` is used.
3. The PTY reader is cloned and moved to a dedicated thread that continuously reads output in 4 KB chunks.
4. Each chunk of output is emitted as a Tauri event (`pty-output`) using `app.emit()`, which the frontend `xterm.js` terminal subscribes to.
5. User input is forwarded to the PTY writer via `write_to_pty`, which writes the data bytes and flushes the buffer.
6. The PTY instance is stored in a `OnceLock<Arc<Mutex<Option<PtyInstance>>>>` to ensure a single, lazily-initialized terminal session per application lifecycle.

The frontend terminal (in `Terminal.tsx`) uses `@xterm/xterm v6` with the `FitAddon` for automatic resizing, and listens for `pty-output` events to render shell output in real-time.

6.1.4 Database Implementation

6.1.4.1 Firebase Project Configuration

We set up the Firebase project (chea-new) in the Firebase Console with the following services:

- **Firebase Authentication:** Enabled with email/password provider. Anonymous authentication and other providers are disabled.
- **Cloud Firestore:** Provisioned in production mode with security rules.
- **Firebase Analytics:** Enabled for usage tracking. The Firebase configuration is embedded in `src/lib/firebase.ts` with the project's API key, auth domain, project ID, storage bucket, messaging sender ID, and app ID.

6.1.4.2 Collection and Subcollection Structure

We organized Firestore data using a user-scoped hierarchy where all user data lives under the `users/{userId}/` path:

```
users/{userId}/
├── vaultConfig/main      # Vault master password verification hash
├── vault/{credentialId}  # Individual credential entries (encrypted)
├── progress/data         # User XP, level, streak, and score data
├── dailyTasks/{date}     # Daily task progress keyed by ISO date string
├── activities/{activityId} # Activity log entries (including ai_phishing_round,
scenario_simulator, security_posture)
├── chatSessions/{sessionId}/
│   └── messages/{messageId} # Individual chat messages within a session
```

Client-Side Cache (localStorage):

The Security Buddy widget (FR-16) caches its daily AI-generated challenge in `localStorage` using the key pattern `chea_buddy_{userId}_{date}_{language}`. This cache automatically invalidates at midnight and refreshes when the user switches language, ensuring only one AI API call per unique user/date/language combination per day.

Vault Configuration:

The `vaultConfig/main` document stores a single `encryptedVerifyHash` field. During vault setup, a fixed verification string ("chea-vault-verification-string") is encrypted with the user's master password using AES-256-GCM and stored. During vault unlock, this hash is decrypted and compared to the expected string — if decryption succeeds and the strings match, the master password is verified. This way, the master password is never stored in plaintext or reversible form.

Credential Storage:

Vault credentials go in the `vault` subcollection. Each document contains `name`, `username`, `domain`, `type` (login or card), and an `encryptedData` field. The `encryptedData` field holds the user's actual password or card details, encrypted client-side using the Tauri crypto module with the vault master password. Only encrypted ciphertext is transmitted to and stored in Firestore.

User Progress:

A single document at `progress/data` stores the user's XP total, current level (1-10), total score, streak day count, and the date of last activity. The level is recalculated client-side after each XP addition using the `LEVEL_THRESHOLDS` array.

Daily Tasks:

Daily tasks are stored in date-keyed documents (`dailyTasks/2025-04-10`). Each document contains an array of task objects with `id`, `type`, `description`, `target`, `current progress`, `points`, and `completed status`. A new task document is automatically created for each day via the `ensureDailyTasks` function.

Chat Sessions:

Chat sessions and their messages use a two-level subcollection structure. Session metadata (`title`, `timestamps`) is stored in `chatSessions/{sessionId}`, while individual messages are stored in `chatSessions/{sessionId}/messages/{messageId}`. The `deleteChatSession` function uses Firestore batch writes to atomically delete all messages and the session document.

6.1.4.3 Firestore REST API Workaround

As mentioned in 6.1.2.5 Firebase Integration, the Firestore Web SDK's gRPC-web transport doesn't work with Tauri's WebView2 runtime. Our custom REST API helper (`firestore-rest.ts`) provides an alternative access path using standard HTTPS `fetch()` calls. This workaround handles:

- **Authentication:** Each request includes the Firebase ID token in the `Authorization: Bearer` header.
- **Serialization:** A bidirectional converter maps between JavaScript types and Firestore's REST field format (e.g., `{ stringValue: "hello" }`, `{ integerValue: "42" }`).
- **Error Handling:** HTTP 404 responses are treated as "document not found" (returning null), while other non-200 responses throw descriptive errors.

6.1.4.4 Security Rules

Firestore security rules restrict all read and write operations to authenticated users only, with an expiration-based access window:

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /{document}/** {
      allow read, write: if request.time < timestamp.date(2026, 8, 5);
    }
  }
}
```

The current rules use a time-based access control that permits operations until August 5, 2026. For production deployment, these rules would need to be tightened to require authentication (`request.auth != null`) and restrict users to accessing only their own data (`request.auth.uid == userId`). We used time-

based rules during development to make testing easier, but this is one of the first things we'd fix before any real deployment.

6.1.4.5 Data Encryption Approach

We implemented client-side encryption for all sensitive data. The encryption pipeline works like this:

- 1. **Master Password:** The user sets a master password for their credential vault. This password never leaves the client device in plaintext.
- 2. **Key Derivation:** When encrypting or decrypting vault data, the master password is combined with a random salt and processed through Argon2id to produce a 256-bit (AES-256-GCM/ChaCha20) or 128-bit (AES-128-CBC) key.
- 3. **Client-Side Encryption:** All encryption and decryption happens in the Rust backend via Tauri IPC. The frontend sends plaintext to the Rust process, which returns encrypted ciphertext. This ensures unencrypted data is never transmitted over the network or stored in Firestore.
- 4. **Storage:** Only the Base64-encoded encrypted payload (containing the algorithm, salt, IV, and ciphertext) is stored in Firestore. The salt and IV are included in the payload to enable decryption, but the password-derived key is never persisted.

6.2 Evaluation and Testing

6.2.1 Functional Testing

We tested every feature manually to make sure it worked correctly. Each test case specifies the input, expected output, actual result, and pass/fail status. Testing was done on a Windows 11 machine with the app running in development mode (bun run tauri dev).

Table 14: Functional Testing Results

Test ID	Test Description	Input	Expected Output	Actual Result	Status
FT-01	User registration with valid credentials	Email: testuser@example.com, Password: Str0ngP@ss!	Account created successfully, user redirected to dashboard	Account created, redirected to dashboard	Pass
FT-02	User login with correct credentials	Email: testuser@example.com, Password: Str0ngP@ss!	User authenticated and redirected to dashboard	Authenticated, dashboard loaded	Pass
FT-03	User login with incorrect password	Email: testuser@example.com, Password: wrongpassword	Error message displayed, user	"Invalid email or password" error shown	Pass

Test ID	Test Description	Input	Expected Output	Actual Result	Status
			remains on login page		
FT-04	User logout	Click "Log Out" button in sidebar	User signed out, redirected to login page	Signed out, redirected to login	Pass
FT-05	Password reset flow	Email: testuser@example.com on forgot password page	Password reset email sent confirmation displayed	Confirmation message shown	Pass
FT-06	URL scan - clean URL	URL: https://www.google.com	Scan result: "clean" status, 0 malicious detections	Status: clean, 0 malicious, multiple harmless/undetected	Pass
FT-07	URL scan - known malicious URL	URL: known phishing test URL	Scan result: "malicious" status, ≥ 1 malicious detections	Status: malicious, multiple engines flagged	Pass
FT-08	URL scan - invalid URL	URL: not-a-url	Error message indicating invalid URL or scan failure	Error returned from VirusTotal API	Pass
FT-09	File scan - clean file	A safe PDF document (1.2 MB)	Scan result: "clean" status, SHA-256 hash displayed	Status: clean, hash matched local calculation	Pass
FT-10	File scan - EICAR test file	EICAR anti-malware test file	Scan result: "malicious" status, multiple detections	Status: malicious, 50+ engine detections	Pass
FT-11	Password generation - custom parameters	Length: 20, uppercase: on, lowercase: on, numbers: on, symbols: on	20-character password with all character types	20-char password generated with all types present	Pass

Test ID	Test Description	Input	Expected Output	Actual Result	Status
FT-12	Password strength check - strong password	Password: Kj7\$mP2!xR9@vL4n	Strength meter: "Strong" or "Very Strong"	Rating: Very Strong, estimated crack time displayed	Pass
FT-13	Password strength check - weak password	Password: 123456	Strength meter: "Weak" or "Very Weak"	Rating: Very Weak, immediate crack time	Pass
FT-14	Encryption- AES-256-GCM round-trip	Plaintext: Hello CHEA!, Password: test123, Algorithm: AES-256-GCM	Encrypted output produced; decrypted output matches original	Decrypted text: Hello CHEA! — matches original	Pass
FT-15	Encryption- ChaCha20-Poly1305 round-trip	Plaintext: Hello CHEA!, Password: test123, Algorithm: ChaCha20-Poly1305	Encrypted output produced; decrypted output matches original	Decrypted text: Hello CHEA! — matches original	Pass
FT-16	Encryption - AES-128-CBC round-trip	Plaintext: Hello CHEA!, Password: test123, Algorithm: AES-128-CBC	Encrypted output produced; decrypted output matches original	Decrypted text: Hello CHEA! — matches original	Pass
FT-17	Vault - setup master password	Master password: VaultP@ss123!	Verification hash stored in Firestore; vault unlocked	Vault setup succeeded, credentials accessible	Pass
FT-18	Vault - add and view credential	Name: "GitHub", Username: "user", Password: ghp_test123	Credential encrypted and stored; decrypts correctly on view	Credential saved, password decrypted and displayed	Pass

Test ID	Test Description	Input	Expected Output	Actual Result	Status
FT-19	Vault - delete credential	Delete an existing credential	Credential removed from list and Firestore	Credential removed from UI and database	Pass
FT-20	AI chatbot - send message and receive response	Message: "What is phishing?"	Streaming response about phishing received	Streaming response delivered with cybersecurity content	Pass
FT-21	AI chatbot - session management	Create new session, send messages, switch sessions	Multiple sessions maintained; messages load correctly	Sessions switch correctly, history preserved	Pass
FT-22	Photo metadata scan - JPEG with EXIF	Sample JPEG with camera and GPS data	EXIF fields displayed: camera make/model, GPS coordinates, timestamps	All metadata categories populated correctly	Pass
FT-23	Photo metadata strip - JPEG with EXIF	Same JPEG, strip metadata	Clean image written; re-scan shows no EXIF data	Stripped image has 0 EXIF fields, identical visual appearance	Pass
FT-24	Quiz Arena - answer questions	Complete a 5-question quiz round	Score calculated, XP awarded, activity logged	Score displayed, XP increased, activity appeared in dashboard	Pass
FT-25	Phishing Dojo - identify phishing emails	Classify 5 emails as phishing or legitimate	Feedback shown for each email; XP awarded	Correct/incorrect feedback displayed, XP earned	Pass

Test ID	Test Description	Input	Expected Output	Actual Result	Status
FT-26	Gamification - XP earning and level progression	Perform activities to earn ≥ 100 XP	XP counter increments, level increases from 1 to 2	XP updated in real-time, level changed to "Apprentice"	Pass
FT-27	Daily tasks - task completion	Complete a scan, generate a password	Task counter increments, completed tasks show checkmarks	Task progress updated, completed task shows green check	Pass
FT-28	Terminal - execute command	Type echo Hello and press Enter	Output Hello displayed in xterm.js terminal	Hello printed in terminal	Pass
FT-29	AI Scan Analysis- Link Scanner (FR-11)	Scan URL https://www.google.com , wait for results	AI analysis card appears below scan results with plain-language risk assessment	AI analysis card rendered with student-friendly explanation of scan statistics	Pass
FT-30	AI Scan Analysis -File Scanner (FR-11)	Scan a clean PDF file, wait for results	AI analysis card appears with file-specific risk assessment	AI analysis card rendered with file safety explanation	Pass
FT-31	AI Attack Narrative - Password Checker (FR-12)	Check password: mypassword123	"How a Hacker Would Crack This" card appears with AI-generated attack narrative	Narrative card rendered with educational hacker perspective explaining dictionary attack vulnerability	Pass
FT-32	AI Phishing Challenge -	Select "AI Challenge" mode in Phishing Dojo	AI-generated email displayed with realistic phishing	Unique AI-generated email shown,	Pass

Test ID	Test Description	Input	Expected Output	Actual Result	Status
	Phishing Dojo (FR-13)		indicators; classification works as expected	classification feedback displayed, +15 XP awarded	
FT-33	AI Scenario Simulator (FR-14)	Complete a 5-round AI Scenario Simulator session	AI-generated scenarios presented one at a time with feedback; XP awarded after completion	5 unique scenarios generated, feedback shown after each answer, +20 XP awarded, activity logged	Pass
FT-34	AI Security Posture Assessment (FR-15)	Complete 15-question assessment and submit	AI-generated security report card with grade, strengths, weaknesses, and recommendations	Personalised report card with grade (B+), category breakdown, and actionable tips displayed	Pass
FT-35	Security Buddy - daily challenge (FR-16)	Open Dashboard, observe Security Buddy widget	Daily AI-generated cybersecurity challenge displayed with title, description, and tool recommendation	Challenge card visible with unique daily task, cached in localStorage.	Pass

6.2.2 Security Testing

We tested the security of our cryptographic implementation, data protection mechanisms, and resistance to common attack vectors.

- Encryption Verification:** After conducting different tests in encryption verification and tamper- detection mechanisms, the results were all positive. The algorithms: AES-256-GCM, ChaCha20-Poly1305, AES-128-CBC all produced unique ciphertexts for the same inputs. Furthermore, the decryption failed and kept giving authentication errors when we did the tamper-detection tests. In this test, a single byte of ciphertext was modified to ensure that even the smallest modification results in failure of decryption. Another test that was conducted was "wrong password rejection" where the plaintext was encrypted with one password and attempted to decrypt it with another. This resulted in a failure of decryption with an error

message which is what we had expected. The tamper detection test (ST-04) was one we were particularly careful about. We manually modified bytes in the ciphertext to confirm that the GCM and ChaCha20-Poly1305 authentication tags actually catch tampering. Even changing a single byte caused decryption to fail, which is exactly what we wanted — it prevents chosen-ciphertext attacks where an attacker might try to modify encrypted data.

- **Vault Data Protection:** In addition to encryption verification, we also conducted a test regarding the protection of Vault Data. Our first test was to check whether the passwords stored in Firestore were plaintext or ciphertext. The results came out to be what was expected; the password fields contained Base 64- encoded ciphertext. Our next test was whether the master password was stored or not, and the results were that it wasn't. Instead, there was only encryption verification hash. Also, after entering the wrong master password, the system showed an error in authenticating, and the credentials were not decrypted.
- **API Key Exposure:** Our API Key test was also quite successful. Both the VirusTotal and OpenRouter API key were absent from the frontend. For the VirusTotal Key, we searched the compiled JavaScript bundle. The key was found in the Rust Binary. For the OpenRouter Key, we inspected the frontend and didn't find any. Instead, the key was loaded via dotenvy in Rust Backend. This test proved that both the VirusTotal and OpenRouter API keys are only used in the Rust backend (virustotal.rs and ai_agent.rs) as we had expected. The frontend talks to these services indirectly through Tauri IPC commands, so the keys never appear in the WebView's JavaScript context or network traffic.
- **Input Validation and Sanitization:** Since CHEA is a desktop app using WebView2, it's inherently more resistant to traditional web-based XSS and SQL injection attacks than a web app would be. We don't use SQL databases, and React's default JSX escaping prevents script injection in rendered content. User inputs are passed to backend commands as string parameters without HTML interpretation. We performed two kinds of XSS resistance test- URL input and Chat input. We entered malicious HTML and script payloads in both URL and AI chatbot chat which were never executed. This confirmed that any XSS vectors that are stored across the interface were eliminated.
- **Session Management:** At the end, we conducted a session management test. We wanted to check if the user remains logged in even after exiting the application. This showed that the session persisted through Firebase browserLocalPersistence. We also checked whether unauthenticated access will be prevented or not. We did this by trying to navigate to the dashboard without login. The result is what we expected ProtectedRoute guard redirected to the login page

6.2.3 Performance Testing

We tested performance on the following machine:

- Operating System: Windows 11 Pro (23H2)
- Processor: Intel Core i7-12700H (14 cores, 20 threads)
- RAM: 16 GB DDR5
- Storage: NVMe SSD
- Network: 100 Mbps fiber connection
- Runtime: Tauri development mode (bun run tauri dev)

Each operation was timed using browser developer tools and application-level timestamps. Results represent the average of 5 consecutive measurements.

Table 15: Performance Testing Results

Operation	Input Size / Parameters	Average Time	Notes
URL scan (clean URL)	https://www.google.com	12.4 seconds	Includes VirusTotal analysis polling (3-4 polls at 4-second intervals)
URL scan (malicious URL)	Known phishing test URL	14.8 seconds	Slightly longer due to larger result set
File scan - small file	PDF document (1.2 MB)	1.2 seconds (cached) / 18.6 seconds (new upload)	SHA-256 hashing: 0.08 seconds; cached result returned from VT database
File scan - medium file	ZIP archive (15 MB)	0.15 seconds (hash) + 22.3 seconds (upload + analysis)	Upload time proportional to file size
Encryption - AES-256-GCM	1 KB plaintext	< 50 ms	Includes Argon2id key derivation
Encryption - AES-256-GCM	100 KB plaintext	< 80 ms	Negligible increase for larger inputs

Operation	Input Size / Parameters	Average Time	Notes
Encryption - ChaCha20-Poly1305	1 KB plaintext	< 50 ms	Comparable to AES-256-GCM
Encryption - AES-128-CBC	1 KB plaintext	< 45 ms	Slightly faster due to simpler key size
Decryption - AES-256-GCM	1 KB ciphertext	< 50 ms	Includes Argon2id key derivation
Password generation	20 characters, all character types	< 5 ms	Cryptographically secure random generation
Photo EXIF scan	4 MB JPEG with GPS data	0.35 seconds	Includes image decoding and EXIF parsing
Photo EXIF strip	4 MB JPEG	0.42 seconds	Includes reading, parsing, stripping, and writing
App cold start	Launch from desktop	3.2 seconds	Time from double-click to interactive dashboard
App warm start	Reopen after closing	1.8 seconds	WebView2 runtime cached
Page navigation	Click sidebar item	< 200 ms	React Router navigation with Framer Motion transitions
AI chatbot - first token	Send message	2.1 seconds	Time to first streamed token from OpenRouter API
AI chatbot - full response	200-word response	8.5 seconds	Total streaming time depends on response length
Terminal - command execution	dir command	< 100 ms	PTY round-trip including rendering

Operation	Input Size / Parameters	Average Time	Notes
AI Scan Analysis (FR-11)	Scan statistics + URL	3.2 seconds	AI-generated plain-language risk assessment
AI Attack Narrative (FR-12)	Password traits (not actual password)	4.1 seconds	AI-generated hacker perspective narrative
AI Phishing Email (FR-13)	Single email generation	3.8 seconds	JSON-validated phishing or legitimate email
AI Scenario Simulator - 5 rounds (FR-14)	All 5 scenarios generated at once	8.7 seconds	Single API call, JSON-validated scenarios
AI Security Posture report (FR-15)	15 questions + answers	5.4 seconds	AI-generated personalised report card
Security Buddy daily challenge (FR-16)	Single challenge generation	2.9 seconds	Cached in localStorage; only 1 call per day

The main performance bottleneck is external API latency. VirusTotal scans take 12-18 seconds because of the polling-based analysis mechanism — there's not much we can do about that since it's how their free API works. AI chatbot and AI feature response times depend on OpenRouter's model inference speed, typically ranging from 2.9 to 8.7 seconds depending on response complexity. The Security Buddy widget mitigates this with localStorage caching (one AI call per day). On the other hand, all local operations (encryption, hashing, EXIF processing, password generation) complete in under 500 ms, which really shows the benefit of using Rust for compute-intensive tasks.

6.2.4 Usability Testing

6.2.4.1 Testing Approach

We ran usability tests with participants from our target demographic (students aged 9-15) along with a few adult evaluators. We used a task-based approach — participants were asked to complete specific security tasks in CHEA while an observer recorded completion time, errors, and qualitative feedback.

6.2.4.2 Task Scenarios

Participants were given these task scenarios:

- 1. Scan a Link:** "Your friend sent you this link: <https://example-suspicious-site.com>. Use CHEA to check if it is safe."
- 2. Generate a Strong Password:** "Create a strong password for your email account using CHEA."
- 3. Encrypt a Secret Message:** "Encrypt the message 'Meet me at the library at 3pm' with the password 'secret123' and copy the result."
- 4. Identify a Phishing Email:** "Open the Phishing Dojo and identify which of the 5 emails are phishing attempts."
- 5. Check a Photo for Hidden Data:** "Scan this photo to see if it contains any hidden location or camera information."
- 6. Save a Credential:** "Save your GitHub username and password in the vault so you don't forget them."
- 7. AI Scenario Simulator:** "Try the Scenario Simulator and work through 5 AI-generated security situations."
- 8. AI Security Posture:** "Complete the Security Posture Assessment to see how safe your online habits are."
- 9. Security Buddy Challenge:** "Check the daily challenge on the Dashboard and complete it."

6.2.4.3 Survey Results

After completing the tasks, participants rated the application on a 5-point Likert scale across multiple dimensions:

Table 16: Survey Results

Criterion	Average Rating (n=12)	Interpretation
Ease of use	4.5 / 5	Participants found the interface intuitive
Visual appeal	4.7 / 5	The cyberpunk theme was well-received by younger users
Task completion rate	4.3 / 5	Most tasks completed successfully on first attempt
Error recovery	3.8 / 5	Some participants needed guidance for vault setup

Criterion	Average Rating (n=12)	Interpretation
Learning value	4.6 / 5	Participants reported learning new security concepts
Willingness to use again	4.4 / 5	Strong interest in continued use

6.2.4.4 Key Findings

1. Intuitive Navigation: Participants found features easily using the categorized sidebar without any instructions. The icon-based navigation (Lucide React icons) was immediately understood.

2. Gamification Engagement: The XP and level system genuinely motivated repeated use. Several participants got excited about earning XP and leveling up, and a few asked to keep using the Quiz Arena even after finishing their assigned tasks. This was encouraging it suggested the gamification approach was working as intended.

3. Dark Mode Preference: Most younger participants (9 out of 12) preferred the dark/cyberpunk theme, calling it "cool" and "hacker-like." This was a pleasant surprise the theme choice seemed to genuinely increase their engagement with the security content.

4. Vault Complexity: The master password concept was initially confusing for participants aged 9-11. They needed an explanation of why a separate password was needed for the vault. Participants aged 13-15 understood the concept right away. In hindsight, we could have made the onboarding flow for the vault more guided.

5. AI Chatbot Engagement: Participants spent the most unassigned time interacting with the AI chatbot, asking cybersecurity questions and requesting explanations of concepts they'd encountered in the Quiz Arena and Phishing Dojo. The chatbot ended up being more popular than we expected.

6. Scan Results Comprehension: The VirusTotal scan results were generally understood, but the difference between "malicious" and "suspicious" categories needed explanation for younger participants. We could improve this by adding clearer visual indicators or tooltips.

6.2.4.5 User Feedback Summary

Feedback was collected from 5 children aged 10-17years:

- Participant 1: Boy, 13 years old
- Participant 2: Boy, 10 years old
- Participant 3: Girl, 13 years old
- Participant 4: Girl, 15 years old

- Participant 5: Girl, 11 years old
- Participant 6: Boy, 17 years old

Based on the feedback collected during usability testing, the following observations were recorded:

1. **Positive Reception:** Users appreciated the gamified learning approach and found the interface engaging and easy to navigate. They appreciated the fact that the application respects user's intelligence. It isn't one of those applications which are extremely "dumbed down" and yet so easy to navigate. The clean UI made the user feel like it was a "real pro tool." One of the users also said they would like to see it published in the future. Another described the application as "amazing" and "very beneficial for avoiding dangerous or harmful files online."
2. **Security Awareness Improvement:** After using the application, participants demonstrated increased awareness of cybersecurity best practices. The Phishing Dojo was specifically praised for teaching "what to look in the real world" which one of the users noted was crucial. A user applying to universities internationally confirmed that the dojo helps with detecting scam emails in real-life scenarios, demonstrating practical knowledge transfer beyond the application.
3. **AI Assistant Value:** The AI chatbot was frequently cited as a valuable feature for learning and getting instant answers to security questions. Users found the feature helpful for answering security-related questions in real-time. It not just provided the answers instantly, but also explained in terms that were easily understandable, which many users appreciated.
4. **Performance Satisfaction:** Most users found the application responsive with minimal loading times during normal operation. One user specifically noted that "seeing all these features to work together so smoothly is pretty impressive" indicating that the integration of multiple tools functioned properly without any issues.
5. **Improvement Suggestions:** Some users suggested adding more visual tutorials and expanding the quiz content library for variety. Some of the users suggested that we add leaderboards and features that will allow them to share their achievements with others in the future.

6.2.5 Strengths and Limitations

6.2.5.1 System Strengths

1. **Integrated Security Toolkit:** CHEA brings together 17 distinct security features in a single desktop application, including six AI-powered features (FR-11 through FR-16) that enhance existing tools with intelligent analysis and introduce new AI-driven educational experiences. Normally, a student would need multiple separate tools for URL scanning, file scanning, photo privacy, password management, encryption, and cybersecurity education. Having everything in one place makes it much more accessible for younger users.

2. Client-Side Encryption Architecture: Using the Rust backend for all cryptographic operations means sensitive data (vault passwords, encrypted messages) is processed entirely on the user's device. The Argon2id key derivation with per-operation random salts provides strong protection against brute-force attacks, and the AEAD modes (AES-256-GCM, ChaCha20-Poly1305) guarantee both confidentiality and integrity of encrypted data.

3. Gamification-Driven Learning: The XP, leveling, streak, and daily task systems create a motivation loop that encourages regular engagement. The tiered difficulty in the Quiz Arena and Phishing Dojo adapts to the user's progression, providing appropriately challenging content at each level. The usability tests confirmed this was working participants genuinely wanted to keep earning XP.

4. Age-Appropriate Design: The UI was designed specifically for the 9-15 age group, with friendly language ("Chat Buddy," "Treasure Box," "Secret Codes"), engaging animations (Framer Motion page transitions, animated XP badges), and a cyberpunk aesthetic that appeals to younger users while keeping things functional.

5. Lightweight Desktop Architecture: Tauri v2 gives us native desktop integration with a much smaller resource footprint than Electron. The compiled binary is under 10 MB compared to 100+ MB for equivalent Electron apps, and memory usage is lower since there's no bundled Chromium instance.

6. Real-World Security Tools: Unlike purely educational tools that simulate security operations, CHEA uses actual external services (VirusTotal for threat analysis, OpenRouter for AI responses) to provide real security analysis results. This gives students a more authentic experience that prepares them for real-world cybersecurity practices.

6.2.5.2 Known Limitations

1. No Automated Test Suite: This is probably the biggest gap. We didn't write unit tests, integration tests, or end-to-end tests, which increases the risk of regression if anyone continues developing the project. All testing was manual, and that's time-consuming and error-prone, especially for complex workflows like encryption round-trips and VirusTotal polling. If we had more time, setting up at least basic unit tests for the crypto module would be the first priority.

2. Firestore Security Rules: The current Firestore security rules use time-based access control instead of authentication-based rules. This means all data is accessible to anyone with the project ID until the expiration date. For any real deployment, we'd need to implement user-scoped rules (`request.auth.uid == userId`). We kept it simple during development for testing convenience, but it needs to be addressed.

3. VirusTotal API Rate Limiting: The free VirusTotal API allows 500 requests per day and 4 requests per minute. In a classroom with multiple students, this limit could be hit quickly. The polling-based analysis mechanism also introduces 12-18 second latencies for URL scans, which some users found a bit slow during testing.

4. Single-Platform Terminal: The terminal emulator only supports PowerShell on Windows and Bash on Unix. It doesn't support custom shell configurations, and the PTY resize functionality isn't fully implemented (the `resize_pty` command is a stub). This feature was added late in development and didn't get as much attention as the core security tools.

5. No Offline Mode: The app requires an internet connection for Firebase authentication, Firestore operations, VirusTotal scans, and the AI chatbot. Users without connectivity can't log in or access any features. Adding at least basic offline support for the vault would be a valuable improvement.

6. Vault Password Recovery: If a user forgets their master password, there's no recovery mechanism. The encrypted vault data can't be decrypted without the correct password, and there's no password reset option for the vault (this is separate from the Firebase account password reset). We considered adding a recovery key feature but ran out of time.

7. Limited File Size Support: While the VirusTotal module handles large file uploads (> 32 MB) via special upload URLs, the file is read entirely into memory before uploading. This could cause memory issues with very large files on systems with limited RAM.

8. Static Educational Content (Partially Addressed): The Quiz Arena (45 questions) has a fixed content set, and after completing everything, users will start seeing repeated content. However, the Phishing Dojo's new AI Challenge mode (FR-13) generates unique phishing emails on demand, providing virtually unlimited practice material. The AI Scenario Simulator (FR-14) also generates fresh scenarios each session. Expanding the AI-generated content approach to the Quiz Arena would further address this limitation.

6.2.5.3 Comparison with Existing Tools

Table 17: Comparison with existing tools

Feature	CHEA	Have I Been Pwned	VirusTotal (Web)	Bitwarden	CyberStart America
URL/Link Scanning	Yes	No	Yes	No	No
File Scanning	Yes	No	Yes	No	No
Password Vault	Yes	No	No	Yes	No
Password Generator	Yes	No	No	Yes	No
Encryption Tool	Yes (3 algorithms)	No	No	No	No
Photo Privacy	Yes	No	No	No	No
AI Chatbot	Yes	No	No	No	No

Feature	CHEA	Have I Been Pwned	VirusTotal (Web)	Bitwarden	CyberStart America
AI Scan Analysis (FR-11)	Yes	No	No	No	No
AI Attack Narrative (FR-12)	Yes	No	No	No	No
AI Phishing Generator (FR-13)	Yes	No	No	No	No
AI Scenario Simulator (FR-14)	Yes	No	No	No	No
AI Security Posture (FR-15)	Yes	No	No	No	No
Security Buddy (FR-16)	Yes	No	No	No	No
Quiz Games	Yes	No	No	No	Yes
Phishing Training	Yes (static + AI)	No	No	No	Limited
Gamification	Yes	No	No	No	Yes
Target Age	9-15	Adults	Adults	Adults	13-18
Desktop App	Yes	Web only	Web only	Yes	Web only

What sets CHEA apart from these tools is that it combines security utilities (which are usually spread across separate professional tools) with educational features (which are usually in separate learning platforms) and AI-powered intelligent analysis (FR-11 through FR-16) into a single, age-appropriate desktop application. Individual tools like VirusTotal's web interface provide more detailed scan results, and password managers like Bitwarden offer more mature vault features — but no single existing solution delivers the integrated, gamified, AI-enhanced, education-focused experience that CHEA provides for its target age group.

Chapter 7

Conclusion and Future Work

7.1 Introduction

This chapter concludes the Cyber Hygiene Educator & Assistant (CHEA) project by summarizing the work completed, presenting the main results and findings, discussing the significance of the project, acknowledging its limitation and outlining directions for future. This chapter ties together the design, implementation, and testing phases that were described in previous chapters to provide a comprehensive conclusion to this research project.

7.2 Summary of Project Work

The CHEA project successfully addressed the important cybersecurity vulnerability facing students aged 9-15 through the development of an integrated Windows desktop application that combines active protection with gamified education. The project followed a systematic development lifecycle, beginning with extensive literature review to identify research gaps, proceeding through requirements analysis and system design, and concluding in implementation and testing.

The development approach utilized modern technologies including React 18 with TypeScript for the frontend user interface, Rust with the Tauri v2 framework for the native desktop backend, and Firebase Firestore for cloud-based data storage. This architectural choice resulted in a lightweight application with a small executable size (under 10 MB) and low memory consumption (under 200MB), while maintaining strong security through client-side encryption using AES-256-GCM with Argon2id key derivation.

7.3 Key Features Implemented

The CHEA application includes ten main functional features organized into four categories:

- **Scanning Tools:** The Link Scanner check URLs against VirusTotal's database of 70+ security engineers, providing student-friendly risk assessments with detailed detection statistics. The File Scanner computes SHA-256 hashes and checks against VirusTotal's database, with the ability to delete detected threats.
- **Password Tools:** The Password Generator creates cryptographically secure passwords with configurable parameters including length (6-64 characters, character types, and exclusion options). The Password Strength Analyzer evaluates existing passwords by calculating entropy, estimating crack time, and detecting common weak patterns.
- **Encryption & Vault:** The Credential Vault provides encrypted storage for login credentials and credit card details, protected by user-defined master password with AES-256-GCM encryption. The Encryption Lab teaches cryptography concepts through hands-on

experimentation with three-encryption algorithms (AES-256-GCM, ChaCha20-Poly1305, AES-128-CBC).

- **Educational Games:** The Phishing Dojo presents gamified phishing identification exercises across three difficulty tiers with detailed feedback on red flags. The Quiz Arena tests cybersecurity knowledge with trivia questions and XP rewards. The AI Agent provides personalized security education through a conversational interface.

7.4 Testing Results and Findings

The testing phase verified that CHEA meets its functional and non-functional requirements across several categories.

Security Testing (6.2.2 Security Testing): All security tests have been passed. Input validation ensures that malicious input is treated as literal text, preventing XSS attacks. The encryption implementation correctly protects sensitive data with AES-256-GCM authenticated encryption. The vault master password verification mechanism works without storing plaintext passwords.

Table 18: Summary of Security Test Results

Test ID	Test Category	Test Description	Expected Result	Actual Result	Status
SEC-01	Input Validation	URL scanner rejects malformed input	No script execution	Pass	Pass
SEC-02	Input Validation	Chat input resists XSS	HTML sanitized	Pass	Pass
SEC-03	Encryption	Vault encryption protects credentials	Encrypted storage	Pass	Pass
SEC-04	Authentication	Master password verification	Password not stored	Pass	Pass

Performance Testing: Local cryptographic operations (encryption, decryption, hashing, EXIF scanning) completed in under 500 milliseconds, meeting the NFR-03 performance requirement. Application cold start completed in under 5 seconds. Page navigation between dashboard sections completed in under 300 milliseconds.

Usability Testing: Initial user testing with students aged 9-15 showed positive results. Participants navigated between tools easily without prior training. The gamification approach received enthusiastic feedback, with participants expressing excitement about the XP system and progression of mechanics. Visual indicators (color-coded strength meters, status badges) were intuitive for the target age group.

7.5 Significance and Impact

The significance of the CHEA project extends across practical and conceptual dimensions.

Practical Significance: The application provides students with a multi-layered defense mechanism against common cyber threats. The integrated approach consolidates tools that would otherwise require multiple separate applications, reduce cognitive load, and simplify the development of consistent security habits. The free, accessible design ensures that students from all economic backgrounds can benefit from cybersecurity education.

Conceptual Significance: The project challenges the prevailing assumption that security must compromise usability. CHEA demonstrates that enterprise-grade security can be made accessible to non-technical users without fundamental compromises. The combination of practical protection with educational content addresses the persistent gap between security awareness and practical skills that existing tools exhibit.

Impact on Education: CHEA offers educational institutions a supplementary tool for cybersecurity curriculum. The gamification elements provide measurable engagement metrics through the XP and progression system. The AI Agent offers personalized learning at a scale. The Phishing Dojo provides practical training that traditional textbook-based education cannot match.

7.6 Limitations

Despite its comprehensive design, the CHEA project has several limitations that should be acknowledged.

Scope Limitations: The application targets students aged 9-15 specifically and does not address other age groups or professional users. The desktop-only deployment (Tauri with WebView2) excludes mobile platform users.

Technical Limitations: The VirusTotal API free tier has rate limits that may affect high-volume usage scenarios. The AI chatbot depends on external API availability (OpenRouter), which may experience downtime. The credential vault requires users to remember a master password with no recovery mechanism; forgotten passwords result in permanent data loss.

Functional Limitations: The image metadata stripping only supports JPEG and PNG formats. The URL scanner does not detect all threat types (zero-day attacks may evade detection).

Resource Limitations: Security testing was conducted manually without automated penetration testing tools. Evaluation relied on informal user feedback rather than formal controlled studies. Long-term retention of cybersecurity habits was not measured due to project timeline constraints.

7.7 Future Work

Several directions for future development and research emerge from this project.

Feature Expansion: Future versions could extend the platform to mobile devices (iOS and Android) using React Native or Flutter. Additional educational content could include more phishing scenarios, malware analysis exercises, and social media safety modules. A browser extension could provide real-time URL protection during web browsing.

Research Opportunities: Long-term studies could evaluate the long-term effectiveness of gamified cybersecurity education. Comparative analysis could measure CHEA against traditional educational approaches. Investigation into optimal gamification parameters (XP thresholds, reward frequencies) could improve engagement metrics.

Technical Improvements: Blockchain-based password recovery could address the master password forgetting issue. Automated security testing using established frameworks could provide a more comprehensive vulnerability assessment. API caching could improve offline functionality and reduce rate limit dependencies.

Community Contributions: Open-sourcing the project could encourage community contributions to content and features. Integration with educational institution learning management systems could streamline classroom adoption. Localization efforts could extend accessibility to non-English speaking populations.

7.8 Conclusion

The Cyber Hygiene Educator & Assistant project successfully addresses the identified research gaps in K-12 cybersecurity education through an integrated, gamified, and accessible desktop application. The comprehensive implementation demonstrates that enterprise-grade security can be made accessible to young students without compromising fundamental security principles.

The project adds value to the field by presenting a working model of integrated security education that combines practical tools with educational content. The positive testing results and enthusiastic user feedback validate the approach of making security accessible through gamification and age-appropriate design. While limitations exist, the foundation established by CHEA provides a solid basis for continued development and research in cybersecurity education for younger populations.

Future work should focus on expanding platform support, conducting long-term effectiveness studies, and building community contributions to content and features. With continued development, CHEA is able to add impactful contributions to closing the cybersecurity skills gap among students aged 9-15 and establishing lifelong defensive digital habits.

References

- Alharbi, T. and Tassaddiq, A. (2021). Assessment of Cybersecurity Awareness among Students of Majmaah University. *Big Data and Cognitive Computing*, [online] 5(2), p.23. doi: <https://doi.org/10.3390/bdcc5020023>
- Alqahtani, M.A. (2022). Factors Affecting Cybersecurity Awareness among University Students. *Applied Sciences*, [online] 12(5), p.2589. doi: <https://doi.org/10.3390/app12052589>
- Abdulrahman Abdullah Arishia, Nazhatul Hafizah Kamarudinb, Azmi, K., Zarina Binti Shukurd and Hasan, M.K. (2024). Cybersecurity Awareness in Schools: A Systematic Review of Practices, Challenges, and Target Audiences. *International Journal of Advanced Computer Science and Applications*, [online] 15(12). doi: <https://doi.org/10.14569/ijacsa.2024.0151249>
- Ayeyemi, M. (2023). A Systematic Review of Cybersecurity Education in K-12 Context. [online] Erepo.uef.fi. Available at: <https://erepo.uef.fi/items/c2615283-de99-4980-a188-df7514d84318> [Accessed 28 Apr. 2026].
- Bitwarden. (n.d.). Bitwarden Security Whitepaper | Bitwarden Help Center. [online] Available at: <https://bitwarden.com/help/bitwarden-security-white-paper/>
- Bottyan, L. (2023). Cybersecurity awareness among university students. *Journal of Applied Technical and Educational Sciences*, [online] 13(3), pp.363–363. doi: <https://doi.org/10.24368/jates363>
- Check Point (2023). Check Point Research Reports a 38% Increase in 2022 Global Cyberattacks. [online] Check Point Software. Available at: <https://blog.checkpoint.com/2023/01/05/38-increase-in-2022-global-cyberattacks/>.
- Chen, W., He, Y., Tian, X. and He, W. (2021). Exploring Cybersecurity Education at the K-12 Level. [online] www.learntechlib.org. Available at: <https://www.learntechlib.org/primary/p/220175/>
- Cowling, M., Sim, K.N., Orlando, J. and Hamra, J. (2024). Untangling Digital Safety, literacy, and Wellbeing in School activities for 10 to 13 Year Old Students. *Education and Information Technologies*, 30. doi: <https://doi.org/10.1007/s10639-024-13183-z>

- Erendor, M.E. and Yildirim, M. (2022). CYBERSECURITY AWARENESS IN ONLINE EDUCATION: A CASE STUDY ANALYSIS. IEEE Access, pp.1–1. doi: <https://doi.org/10.1109/access.2022.3171829>
- Fakhrudin, A., and Haryanto (2023). Digital Literacy Analysis of Primary School Students. KnE Social Sciences. [online] doi:<https://doi.org/10.18502/kss.v8i8.13280>.
- Ibrahim, A., McKee, M., Sikos, L.F., and Johnson, N.F. (2024). A Systematic Review of K-12 Cybersecurity Education Around the World. IEEE access, pp.1–1. doi: <https://doi.org/10.1109/access.2024.3393425>
- Martinez, E. (2023). VirusTotal Release Notes. [online] Virustotal.com. Available at: <https://releases.virustotal.com/2023/07/>
- Zhang-Kennedy, L. and Chiasson, S. (2021). A Systematic Review of Multimedia Tools for Cybersecurity Awareness and Education. ACM Computing Surveys, 54(1), pp.1–39. doi: <https://doi.org/10.1145/3427920>
- Purnama, S., Ulfah, M., Machali, I., Wibowo, A. and Narmaditya, B.S. (2021). Does digital literacy influence students' online risk? Evidence from Covid-19. Heliyon, 7(6), p.e07406. doi: <https://doi.org/10.1016/j.heliyon.2021.e07406>
- Verizon (2021). 2021 Data Breach Investigations Report (DBIR). [online] Verizon Enterprise Solutions. Available at: <https://www.verizon.com/business/resources/reports/2021-data-breach-investigations-report.pdf>.
- Saglam, R.B., Miller, V. and Franqueira, V.N.L. (2023). A Systematic Literature Review on Cyber Security Education for Children. IEEE Transactions on Education, 66(3), pp.1–13. doi: <https://doi.org/10.1109/te.2022.3231019>
- Dhanya Pramod (2024). Gamification in cybersecurity education; a state of the art review and research agenda. Journal of Applied Research in Higher Education. doi: <https://doi.org/10.1108/jarhe-02-2024-0072>
- Liu, N., Long, S., and Martin, F. (2025). Systematic Review of Elementary Cybersecurity Education: Curriculum, Pedagogy, and Barriers. Journal of Cybersecurity Education, Research and Practice, 2025(1). doi: <https://doi.org/10.62915/2472-2707.1265>

- Mrđa, B., Soleša, D. and Krašna, M. (2025). Cybersecurity education for children: Development of an online application as an internet safety tool. *International Review*, (1-2), pp.80–94. doi: <https://doi.org/10.5937/intrev2501080m>
- Nagaraj, D.H., Shinu Abhi and Agarwal, R. (2025). Cybersecurity awareness: Gamified learning through phishing analysis. *AIP conference proceedings*, 3281, pp.020032–020032. doi: <https://doi.org/10.1063/5.0247129>
- Chiosea, F. and Getty (n.d.). Executive Summary The State of Cybersecurity Education in K-12 Schools Results of a National Survey. [online] Available at: <https://cyber.org/sites/default/files/2020-06/The%20State%20of%20Cybersecurity%20Education%20in%20K-12%20Schools.pdf>.
- Almomani, I., Ahmed, M. and Maglaras, L. (2021). Cybersecurity maturity assessment framework for higher education institutions in Saudi Arabia. *PeerJ Computer Science*, 7, p.e703. doi: <https://doi.org/10.7717/peerj-cs.703>
- Kaspersky Lab . (2023). Kaspersky Total Security Solutions. [online] Available at: <https://www.kaspersky.com>.
- Wijanarko, A. and Erlansari, A. (2025) “Gamification on Cybersecurity Awareness Training for Adolescents: A Systematic Literature Review”, *Indonesian Journal of Computer Science and Engineering*, 2(02), pp. 6–12. doi: <https://rumah-jurnal.com/index.php/ijcse/article/view/490>
- Khairallah, A.O. and Abu-Naseer, M. (2026). The effectiveness of gamification teaching method in raising awareness on Email Phishing: Controlled Experiment. [online] *Diva-portal.org*. Available at: <https://nu.diva-portal.org/smash/get/diva2:1872531/FULLTEXT01.pdf> [Accessed 28 Apr. 2026]